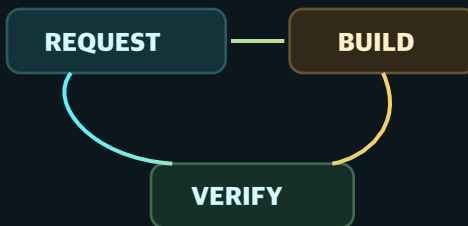
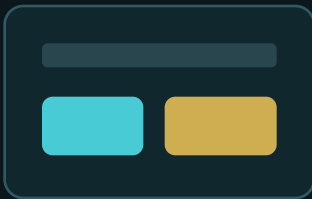


AI와 끝까지 완성하는 Tauri 2 데스크톱 앱

여섯 프로젝트로 익히는
요청·구현·검증·배포

Rust · SvelteKit · Tauri 2 · 실전 배포



Ted Park

요청 | 구현 | 검증 | 배포

AI와 끝까지 완성하는 Tauri 2 데스크톱 앱

여섯 프로젝트로 익히는 요청·구현·검증·배포

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Apple SD Gothic Neo | Menlo.

차례

머리말	1
교육용 자료와 제공 범위	1
이 장의 흐름	2
학습 목표	2
이 책을 쓰면서	2
검증의 기준을 옮기다	3
이 책이 다루는 것	3
왜 이 기술 스택인가	4
이 책이 다루지 않는 것	4
이 책이 다루는 기술 스택이 낫선 이유	5
여섯 프로젝트 한눈에 보기	5
이 책 읽는 법	6
전체 구성	7
사전 요구사항	8
기술 스택 한눈에 보기	8
개발 환경	9
저자의 노트	10
마무리 박스: 이 장에서 반드시 기억할 것	11
1장. 속도보다 중요한 건 검증이다—주도권을 쥐어야 하는 이유	12
이 장의 흐름	13
학습 목표	13
1.1 왜 계획 없는 AI 코딩은 오래 못 가는가	14
1.2 왜 하필 지금인가	14

1.3 개발, 그전에는: 직접 코드를 썼다	15
1.4 지금은: AI가 쓰고, 내가 결정한다	15
1.5 “읽기만 하면 된다”는 말이 반만 맞는 이유	17
1.6 역할 전환: 코드 검토에서 요구사항 정리로	17
1.7 한 장면—readbooks.ai의 PDF 파서 에피소드	18
1.8 이 책에서 계속 등장하는 여섯 가지 사례	19
1.9 이 책을 읽고 나면 남는 것	19
마무리 박스: 이 장에서 반드시 기억할 것	20
2장. Request, Build, Verify —루프의 형태	21
이 장의 흐름	22
학습 목표	22
2.1 하나의 루프, 크기만 달라질 뿐	23
2.2 1단계—요청	24
너무 짧은 요청	24
너무 긴 요청	24
2.3 2단계—구현	25
2.4 3단계—검증	25
2.5 4단계—반복: 검증 결과를 다음 요청으로 돌려보내기	26
2.6 “완벽한 프롬프트 하나로 앱 완성”이 허상인 이유	27
2.7 빠대 우선 전략—탑다운 AI 개발	27
1단계: 빠대—아키텍처 프롬프트	28
2단계: 세부 구현—한 번에 하나의 모듈	29
3단계: 버그 수정—컴파일, 실행, 수정	29
4단계: 기능 QA —각 기능을 처음부터 끝까지 검증	29
5단계: 다듬기—엡지 케이스, 성능, UX	30
이 순서가 중요한 이유	30
2.8 실제 사례—Mandai DB 스키마 루프	31
사이클 1—요청	31
사이클 1—구현	31
사이클 1—검증	32
사이클 2—수정 요청	32

사이클 2—구현	32
사이클 2—검증	33
2.9 기본 프롬프트 템플릿: 무엇을 / 왜 / 제약사항 / 형식	34
2.10 루프 돌릴 때 주의할 점	35
마무리 박스: 이 장에서 반드시 기억할 것	35
3장. 프롬프트 전용 개발 루프—도구보다 중요한 운용법	36
이 장의 흐름	37
학습 목표	37
3.1 이 장의 범위	37
3.2 프롬프트 전용 루프의 5단계	38
1단계: 작업 단위 정하기	38
2단계: 프롬프트 작성	38
3단계: 결과 받기	39
4단계: 검증	39
5단계: 수정 요청	39
3.3 프롬프트 패턴 5가지	40
패턴 1: 뼈대 먼저	40
패턴 2: 범위 고정	40
패턴 3: 부정 제약	40
패턴 4: 파일 참조	41
패턴 5: 검증 기준 선제 작성	41
3.4 짧은 실전 세션—노트 저장 기능	41
라운드 1—뼈대만 받기	42
라운드 2—명령 하나만 구현	42
라운드 3—프론트엔드 연결	43
라운드 4—실패 증거로 수정	43
세션에서 배울 것	44
3.5 4장으로 넘어가기 전에	44
마무리 박스: 이 장에서 반드시 기억할 것	44
4장. 도구와 환경—Claude Code, Cursor, Codex, GitHub Copilot	46
이 장의 흐름	47

학습 목표	47
4.1 네 가지 도구, 네 가지 접근	47
4.2 언제 무엇을 쓰는가	49
4.3 최소 Tauri 2 + SvelteKit + Rust 환경	49
버전 고정 전략	50
필수 설치 단계	50
AI 도구 설정	51
프로젝트 초기화	52
설치가 실패할 때—가장 흔한 3가지	53
4.4 처음 30분—스캐폴드에서 첫 커밋까지	54
즉시 확인 (5분)	54
프로젝트 설정 (10분)	54
첫 생산적 변경 (15분)	55
확인 체크리스트	56
4.5 Cursor Project Rules / CLAUDE.md —규칙 파일 활용	57
CLAUDE.md (Claude Code용) —실제 사례	57
Cursor Project Rules —같은 개념, 다른 형식	58
규칙 파일의 효과	58
4.6 AI에 줄 제약—스택과 버전 명시	58
4.7 더 알아보기: MCP, Agent, 슬래시 명령	59
4.8 도구 선택 시 따르는 5가지 원칙	59
마무리 박스: 이 장에서 반드시 기억할 것	60
5장. Tauri 2 기초—핵심 패턴과 AI 실수 방지	61
이 장의 흐름	62
학습 목표	62
5.1 Tauri 2 아키텍처 개괄—명확한 경계가 중요한 이유	63
네이티브 UI가 아닌 WebView인 이유	64
5.2 명령—#[tauri::command] 패턴	64
명령 이름을 짓는 원칙	64
가장 작은 예제	64
실패할 수 있는 명령	65
여러 필드를 가진 데이터	66

5.3 상태 관리— <code>tauri::State<T></code>	67
기본 패턴: <code>RwLock</code>	67
잠금 선택: 자료형보다 임계 구역을 먼저 본다	68
초기화 비용이 큰 자원	69
5.4 이벤트— <code>emit</code> 과 <code>listen</code>	69
백엔드에서 프론트엔드로 보내기	69
스트리밍 패턴	70
5.5 플러그인— <code>tauri-plugin-*</code> 생태계	71
AI 앱에 자주 쓰는 플러그인	71
5.6 권한과 기능—Tauri 2의 새로운 보안 모델	72
AI가 권한을 빠뜨리는 이유	73
5.7 프론트엔드— <code>SvelteKit 2 + Svelte 5 + Tauri</code>	74
<code>Svelte 5 rune</code> 과 Tauri 이벤트의 궁합	74
API 래퍼로 명령 호출 모으기	75
5.8 프로젝트 구조—참조 레이아웃	76
5.9 AI 개발을 위한 Tauri 2 멘탈 모델	77
프롬프팅 전략	77
5.10 AI가 Tauri 2에서 자주 저지르는 실수들	77
마무리 박스: 이 장에서 반드시 기억할 것	78
6장. Mandai 설계—AI에게 얼마나 큰 단위로 맡길 수 있나	81
이 장의 흐름	82
학습 목표	82
6.1 Mandai는 처음에 어떤 앱이었나	83
6.2 첫 번째 실수—“전체를 한 번에 만들어줘”	84
문제 1. 셀 ID로 UUID를 썼다	84
문제 2. Pomodoro 세션이 태스크와 연결되지 않았다	84
문제 3. 폴더 구조는 그렇듯하지만 내용이 비어 있었다	85
6.3 크기 조절—AI가 한 번에 처리할 수 있는 단위	85
셀 ID 체계—두 번의 반문으로 확정	86
Pomodoro 세션—분석 시나리오까지 따라간다	86
AI 채팅—선택지를 제시받기	87
6.4 AI의 반문을 설계 도구로 활용하기	87

6.5 세 계층으로 구성하기: 화면, 데이터, 명령	89
계층 1: 화면 정의	89
계층 2: 데이터 요구사항	90
계층 3: 명령 열거	90
6.6 AI 청크 크기 법칙	91
6.7 빌드 중간에 계획이 바뀔 때	93
피벗 1: 셀 ID 충돌 문제	93
피벗 2: 실시간 타이머 문제	94
피벗이 너무 잦아질 때	95
마무리 박스: 이 장에서 반드시 기억할 것	95
7장. DB 스키마—첫 번째 Request -> Build -> Verify 루프	97
이 장의 흐름	98
학습 목표	99
7.1 DuckDB를 선택한 이유	99
먼저 봐야 할 것—7개 테이블의 관계	99
왜 DuckDB인가	99
DuckDB vs SQLite —AI가 자주 틀리는 부분	100
7.2 첫 번째 루프—요청 초안 작성	100
연결은 한 번 열고 공유한다	101
스키마 요청 보내기	102
초안 검토—무엇이 빠졌나	102
7.3 두 번째 루프—구멍 메우기	103
수정된 핵심 테이블	103
인덱스—자주 실행할 쿼리부터 정한다	104
7.4 세 번째 루프—마이그레이션 시스템	105
7.5 스키마 테스트—AI로 품질 검증하기	106
7.6 한 번에 완성된 명령—db_query_json	107
7.7 Mandai를 위한 DuckDB 쿼리 패턴	108
소프트 삭제와 조인의 주의점	109
7.8 TypeScript 타입 레이어—AI가 확실히 도움이 된 곳	109
7.9 이 장의 루프 요약	110
DB 작업 패턴 분류	110

7.10 Rust 측 명령 레지스트리	111
마무리 박스: 이 장에서 반드시 기억할 것	111
8장. AI API 통합—AI를 사용해서 AI 코드 작성하기	113
이 장의 흐름	114
학습 목표	114
8.1 요구사항을 먼저 글로 정리하기	115
8.2 다중 Provider 추상화—Rust Trait	116
8.3 이벤트 구조	117
스트림 취소	119
8.4 SSE 파싱이 실패한 곳	119
8.5 에러 처리—Provider마다 다르게 실패할 때	120
8.6 응답 정규화—프론트엔드에 통일된 형식 제공	121
8.7 이 장의 루프 정리	121
마무리 박스: 이 장에서 반드시 기억할 것	122
9장. Mandai 개발 실수—AI가 틀린 곳들	123
이 장의 흐름	124
학습 목표	124
9.1 Pomodoro 타이머—상태가 두 곳에 산다	125
진짜 문제	125
잘못된 첫 시도	125
함정 1 —CHECK 제약 위반	126
함정 2 —과도한 정리 로직과 시간 기반 복구	126
함정 3 —백그라운드 스로틀링과 태스크 바인딩	127
수정된 프롬프트 템플릿	127
핵심 교훈	128
9.2 Mandal-Art Cell ID —일반 패턴이 도메인 규칙을 덮어쓰는 순간	128
어디서 어긋났나	129
왜 이런 일이 일어나는가	129
고친 방법	129
9.3 생산성 점수 계산—엣지 케이스를 명시하면 AI가 개선된다	130
루프마다 드러난 엣지 케이스	130

이 경험에서 얻은 것	131
9.4 버전 혼동—Tauri 1.x와 2.x API를 섞을 때	132
전형적인 예: 창 관리 API	132
버전 명시 프롬프트로 해결	132
v1 -> v2에서 자주 섞이는 패턴	132
9.5 막는 법: 프롬프트·리뷰·테스트 3종 세트	133
가짜 생산성	133
도구 1—프롬프트 체크리스트	134
도구 2—리뷰 프롬프트	135
도구 3—테스트 프로토콜	136
세 도구의 관계	137
마무리 박스: 이 장에서 반드시 기억할 것	137
10장. readbooks.ai—AI 출력 품질을 설계하는 프롬프트 엔지니어링	139
이 장의 흐름	140
학습 목표	141
10.1 배경	141
번역을 위한 Provider 추상화	141
온디바이스, 온디맨드, 오프라인은 다른 말이다	142
소형 모델이 데스크톱 번역 옵션이 된 이유	144
10.2 첫 번째 프롬프트—모든 것을 놓친 초안	146
왜 AI는 이렇게 동작하는가	146
실제 실패 사례—한 블록에 다섯 가지 오류	146
1차 수정—보존 범위부터 고정한다	147
2차 수정—“기호”와“내용”을 분리한다	147
10.3 언어별 분기—“36개 언어를 위한 하나의 프롬프트”환상	148
해결—기본 프롬프트 + 언어별 프로파일	149
실제 프로파일은 어떻게 생겼는가	149
언어 프로파일 구축: AI를 연구 어시스턴트로 쓰기	150
10.4 AI와 함께 프롬프트 설계—메타 루프	150
메타 프롬프트란	150
같은 AI를 검토자로 다시 쓰기	151
이중 에이전트—번역자와 검토자 분리	151

실패한 메타 프롬프트들	151
프롬프트 버전 관리	152
10.5 번역 품질 기준—AI에게 전달하는 방법	152
각 기준이 어떤 실패에서 나왔는가	153
도메인별 추가 기준	153
검증 파이프라인—자동 검사 + 위험 기반 사람 검토	153
10.6 이 장의 루프 정리	154
반복된 세 가지 패턴	155
마무리 박스: 이 장에서 반드시 기억할 것	155
11장. 번역 검증—AI 출력을 어떻게 신뢰할 것인가	157
이 장의 흐름	158
학습 목표	158
11.1 문제—“그렇듯하지만 틀린”번역	158
11.2 역번역—손으로 한 번 해보기	159
11.3 검증 명령 만들기—가장 단순한 골격	160
오프라인 검증에서 독립성을 지키는 방법	160
11.4 단일 점수의 한계—BLEU와 cosine 유사도	161
11.5 세 개의 축으로 나누기	161
결과 타입을 프론트엔드와 맞추기	161
날조 탐지 추가	162
3단계 배지로 정리	162
11.6 운영하며 다듬는 이야기	163
11.7 사용자 피드백 루프	163
기능	164
실제 데이터로 프롬프트 업데이트	164
피드백을 단순한 세 단계로 정렬	164
11.8 “AI 출력을 다른 AI로 검증하기”시각화	164
11.9 독자 앱에 적용하는 팁	165
팁 1. 단일 점수 금지. 축을 쪼개라.	165
팁 2. 생성 단계와 검증 단계를 독립적으로.	166
팁 3. 임계치는 3단계로.	166
팁 4. 사용자 피드백을 프롬프트에 반영.	166

11.10 Part II와 Part III를 돌아보며	166
마무리 박스: 이 장에서 반드시 기억할 것	166

머리말

이 책은 **본문의 설명·계산 예·상태 흐름으로 설계와 검증 원리를 이해하도록** 구성했다. 교육용 ZIP은 반복 실험을 위한 선택 자료다. `app.trading`이나 상용 앱 클래스 이름이 등장하는 코드는 구조를 설명하는 발췌일 수 있으며, 별도의 전체 제품 소스를 복원하라는 뜻은 아니다. 독립 실행 예제는 실행 조건과 기대 결과를 함께 표시한다.

교육용 자료와 제공 범위

교육용 파일 묶음 이름은 `tauri2-app-examples.zip`이다. 포함 범위는 `tauri/ch05-local-notes/`의 Tauri 최소 앱과 `flet/ear-loop/`의 19장 선택 실습다. 설치 명령·버전 잠금 파일·검증 범위는 각 README에서 확인한다. 전체 상용 앱이나 자동매매 운영 저장소의 접근권한을 제공하는 상품은 아니다.

자료는 위키독스의 책 사용법·전체 목차 안내에 기재된 제공 경로로 받는다. 해당 경로가 아직 표시되지 않은 판매 준비 단계라면 자료 배포가 완료된 것으로 해석하지 않는다. 링크나 첨부이 없을 때는 저자의 공개 안내 페이지에 문의한다. PDF의 코드 설명과 실행 가능한 교육용 묶음의 범위를 구분한다.

이 장의 흐름

이 머리말은 본문이 시작되기 전에 다음을 짚어둔다.

- 저자가 AI와 함께 앱을 개발하고 검증하는 방법
- 이 책이 다루는 것과 다루지 않는 것
- 책을 읽기 위해 알고 있어야 하는 것
- 이 책에서 만들 여섯 개의 프로젝트와 사용 기술
- 책을 어떤 순서로 읽으면 좋은지

처음부터 모두 이해할 필요는 없다. 본문을 읽다가 길을 잃었을 때 돌아오면 된다.

학습 목표

이 머리말을 읽고 나면 다음 세 가지만 이해하면 된다.

- 이 책이 AI 도구 사용법 자체보다 **검증하며 앱을 완성하는 방식**을 다룬다는 점
- Tauri 2, SvelteKit, Rust, DuckDB 같은 기술을 처음부터 모두 외울 필요는 없다는 점
- 본문을 어떤 순서로 읽고, 막힐 때 어디로 돌아오면 되는지

이 책을 쓰면서

2026년, AI 도구는 빠르게 바뀌었다. 하지만 내 작업 방식이 달라진 건 단순히 도구가 새로워졌기 때문이 아니었다. 내가 맡는 일의 범위 자체가 바뀌었다.

1월의 나는 몇 년 동안 그랬듯 코드를 직접 쓰는 개발자였다. 티켓을 열고, 에디터를 열고, 타이핑했다. 기능 하나에 이틀, 복잡하면 사흘. 그게 익숙한 속도였다.

이 책에서는 데스크톱 앱 네 개와 터미널 도구 두 개, 모두 여섯 개의 프로젝트를 사례로 다룬다. 생산성 앱과 트레이딩 시스템을 확인하는 TUI(터미널 화면에서 동작하는 앱) 등이 포함된다.

실력이 갑자기 는 게 아니다. 내가 하는 일과 AI가 하는 일의 역할이 근본적으로 바뀌었다.

검증의 기준을 옮기다

연초에는 다들 그러듯 AI를 더 빠른 자동완성처럼 썼다. 원하는 걸 설명하면 AI가 코드를 만들고, 나는 그걸 받아 작업을 이어갔다.

AI는 빨랐지만 나는 느렸다. AI가 3분 만에 만든 코드를 나는 한 시간 반 동안 리뷰했다. 꼼꼼한 리뷰가 좋은 습관이라고 생각했지만, 반복할수록 이상한 병목이 생겼다. **읽을 필요도 없는 코드를 읽으면서 내가 전체 작업을 막고 있었다.**

그때 스스로에게 물었다.

내가 해야 하는 일이 코드가 올바른지 확인하는 건가, 아니면 기능이 올바르게 동작하는지 확인하는 건가?

첫 번째는 코드를 읽어서 확인한다. 두 번째는 직접 실행해서 확인한다. 실제로 중요한 건 두 번째인데, 나는 첫 번째에 시간을 쓰고 있었다.

그 뒤로 작업 방식을 이렇게 바꿨다.

- AI에게 정확한 요청을 작성한다.
- 기능을 직접 실행해서 확인한다.
- 자주 발생하는 오류 상황을 체크리스트로 점검한다.
- 통과하면 커밋한다. 실패하면 그 내용을 그대로 다음 요청에 넣고 다시 반복한다.

이 변화는 코드를 안 보겠다는 게 아니다. **검증의 기준을 코드 모양에서 실제 동작으로 옮기겠다는 뜻이다.**

그 결과가 여섯 개 프로젝트와 이 책이다. 시간이 더 생긴 게 아니다. 중요하지 않은 곳에 시간을 쓰지 않기로 한 것뿐이다.

AI는 코드를 짜고, 판단은 당신이 한다. 무계획한 AI 코딩은 빠르게 시작할 수 있지만, 무엇이 잘못됐는지 모르면 AI에게 설명할 수도 없고 고칠 수도 없다. 왜 막히는지, 그리고 어떻게 검증의 주도권을 사람이 잡는지는 1장에서 자세히 다룬다.

이 책이 다루는 것

이 책 전체를 하나의 사이클이 관통한다.

Request(요청) -> Build(구현) -> Verify(검증)

앱을 만들 때 이 세 단계를 계속 반복한다. 검증을 통과하면 다음 요청으로 넘어가고, 실패하면 실패한 결과를 증거로 삼아 다시 요청한다. 각 단계가 정확히 무엇을 뜻하는지, 어디서 실수가 자주 생기지는 2장에서 설명한다. 이 사이클이 Tauri 2 앱에서 실제로 어떻게 돌아가는지를 여섯 개 프로젝트로 직접 보여준다.

아주 작은 예로 보면 이렇다. “버튼을 누르면 파일을 저장하라”고 요청한다. AI가 Tauri 명령과 Svelte 버튼 코드를 만든다. 그다음 코드를 읽으며 추측하지 않고, 실제로 버튼을 눌러 파일이 생기는지 확인한다. 파일이 생기지 않으면 오류 메시지와 기대 동작을 다음 요청에 넣는다. 이 짧은 흐름이 이 책 전체의 기본 단위다.

왜 이 기술 스택인가

이 스택의 출발점은 Tauri 2다. 단지 설치 파일이 작고 빠르기 때문만은 아니다. AI와 함께 개발할 때 사람이 주도권을 놓치지 않기 쉬운 구조였기 때문이다.

화면은 Svelte가 맡고, 파일 접근·프로세스 실행·시스템 권한처럼 위험한 일은 Tauri 명령이 맡는다. 역할이 나뉘어 있으니 문제가 생겼을 때 “화면 상태의 문제인지, 시스템 명령의 문제인지, 데이터 저장의 문제인지”를 사람이 직접 짚을 수 있었다.

이 구분은 AI에게도 중요하다. AI가 코드를 빠르게 만들수록, 개발자는 어디까지 말기고 어디서 검증 할지를 분명히 해야 한다.

Tauri 2는 프론트엔드와 백엔드의 경계가 드러나고, DuckDB는 로컬 파일 하나로 데이터를 확인할 수 있으며, Svelte 5는 화면 상태를 비교적 단순한 패턴으로 정리할 수 있다. 이 조합은 기술적 우월성을 주장하기 위해서가 아니라, 사람이 검증하고 통제하기 쉬운 개발 흐름을 만들기 위해 이 책의 중심 스택이 됐다.

이 책이 다루지 않는 것

이 책이 무엇을 다루지 않는지 미리 알아두면, 다른 자료를 함께 찾아야 할 때 헛갈리지 않는다.

- **Rust 문법 튜토리얼.** Rust 기초 문법은 설명하지 않는다.
- **Svelte 5 문법 튜토리얼.** Svelte 5의 새 문법은 코드에서 쓰지만, 처음부터 가르치지는 않는다.
- **공식 Tauri 2 문서 재설명.** 이 책은 공식 문서를 대체하거나 반복하지 않는다.
- **프롬프트 엔지니어링 일반론.** 글쓰기·이미지 생성 같은 코딩 외 영역은 다루지 않는다.

- **AI 모델 내부 구조.** Claude와 GPT가 내부적으로 텍스트를 어떻게 처리하는지는 비교하지 않는다.

이 책이 다루는 기술 스택이 낮은 이유

이 책이 다루는 기술들이 생소하게 느껴지는 것은 자연스러운 일이다.

Tauri 2는 Rust와 JavaScript/TypeScript를 함께 쓰는 프레임워크다. 두 언어는 서로 다른 환경에서 돌아가기 때문에, 처음에는 어떻게 연결되는지 파악하기 어렵다. 처음부터 모든 구조를 이해할 필요는 없다.

이 책은 각 기술을 개별 튜토리얼처럼 하나씩 설명하지 않는다. 실제 프로젝트를 진행하다 문제가 생기면 AI에게 물어보고, AI가 제안한 해결책을 직접 실행하며 확인한다. 배워야 할 것은 모든 기술의 세부 문법이 아니다. **모르는 기술 앞에서도 문제를 쫓개고 확인하는 방식이다.**

예를 들어 Mandai 앱(만다라트 할일·목표 관리 앱)을 만들면서 DuckDB(가벼운 분석용 내장 데이터베이스) 연동을 배우게 된다. 트레이딩 대시보드에서는 WebSocket(서버와 끊임 없이 양방향으로 데이터를 주고받는 통신 방식)으로 실시간 데이터를 받는다. 막히는 부분은 AI에게 넘기면 된다. 처음부터 모두 이해하고 시작할 필요는 없다.

AI는 구현의 많은 부분을 맡고, 한 방법이 막히면 다른 방법을 빠르게 시도한다. 그렇다고 모든 결정을 AI에게 넘기지는 않는다. AI가 선택지와 초안을 만들면 **최종 판단은 사람이 내린다.**

AI가 Rust 코드를 대신 작성해주지만, 생성된 코드에서 문제를 발견하려면 기본 문법 정도는 읽을 수 있어야 한다. 직접 쓸 줄 알 필요는 없다.

여섯 프로젝트 한눈에 보기

이 책에서 다루는 여섯 프로젝트는 설명을 위해 만든 예제가 아니다. 저자가 실제로 사용하는 앱들이다.

프로젝트	무엇을 만드는가	첫 회독 난이도
Mandai	AI 코치를 내장한 생산성 앱	중간
readbooks.ai	2025년 Next.js로 만든 PDF 번역 앱을 Tauri 2로 재구현	중간
Trading Dashboard	실시간 트레이딩 대시보드	어려움

프로젝트	무엇을 만드는가	첫 회독 난이도
RL & Backtest Lab	강화학습 훈련 제어	어려움
TUI Trading Monitor	터미널에서 동작하는 트레이딩 모니터	쉬움
TUI Log Viewer	터미널 로그 뷰어	쉬움

각 프로젝트가 구체적으로 무엇을 하는지, 그리고 이 책 전체에서 어떤 역할을 맡는지는 1.8절에서 자세히 다룬다.

이 책 읽는 법

Part I(1~5장)은 순서대로 읽는다. 책 전체의 전제, 루프 정의, AI 코딩 도구 워크플로우, 도구 설정, Tauri 2 기초를 다룬다.

Part II로 넘어가기 전에 **1.8절(여섯 가지 사례)**을 먼저 훑어본다. 이후 각 장에서 AI에게 무엇을, 왜 요청하는지 맥락이 훨씬 잘 잡힌다.

Part II_{VI}(6~17장)는 각 파트가 하나의 앱을 중심으로 독립 구성된다. 관심 있는 파트만 골라 읽어도 된다.

- Mandai 앱이 궁금하면 -> Part II
- AI 번역과 출력 품질이 궁금하면 -> Part III
- 트레이딩 대시보드가 궁금하면 -> Part IV
- 도메인 지식을 AI에게 주는 법이 궁금하면 -> Part V
- 터미널 앱이 궁금하면 -> Part VI

각 파트는“이 앱에서 배울 루프 패턴”소개로 시작한다. Part II 이후의 앱 사례는 서로 이어지기도 하지만, 반드시 모두 순서대로 읽어야 이해되는 구조는 아니다. 관심 있는 앱부터 읽어도 되도록 각 파트의 첫 장에서 필요한 맥락을 다시 잡아준다.

Part VII(18장)은 이 방법을 자신의 앱에 적용할 때 참고하는 장이다. 처음 읽을 때 한 번, 실제로 자신의 앱을 시작할 때 다시 읽는다.

Part VIII(19장)은 Python 중심 개발자를 위한 보충 경로다. Tauri 2 대신 Flet을 선택할 조건, 번역 모델까지 포함한 오프라인 데스크톱 앱을 만드는 흐름, AI가 오래된 Flet API를 만들지 않도록 검증하는

방법을 다룬다. Rust와 TypeScript의 경계가 아직 부담스럽다면 5장을 읽은 뒤 19장으로 건너가도 된다.

막혔을 때는 처음부터 다시 읽지 않아도 된다. 용어가 낯설면 “기술 스택 한눈에 보기”로 돌아오고, 개발 환경이 맞지 않으면 “개발 환경”표를 확인한다. 루프 자체가 헷갈리면 2장으로, 내 앱에 어떻게 적용할지 막히면 18장으로 돌아오면 된다.

전체 구성

이 책은 여덟 개의 파트로 구성된다. Part I~VII은 여섯 개의 실제 앱과 독자의 앱을 중심으로 진행하고, Part VIII은 같은 검증 루프를 Python 기반 Flet에 적용하는 보충 실습이다. **굵게 표시된 이름**은 직접 만들어 볼 프로젝트다.

Part	장	테마
I	1 -5	AI를 쓰는 이유, 피드백 루프, Copilot 워크플로, 개발 도구, Tauri 2 기초
II	6 -9	Mandai —설계, DB, AI API, AI 실수 대처
III	10 -11	readbooks.ai —프롬프팅, 결과 검증
IV	12 -14	Trading Dashboard —규모 확장, 실시간 처리, 배포
V	15	RL & Backtest Lab —AI에 전문 도메인 지식 주입하기
VI	16 -17	TUI Apps —터미널 UI 앱 빠르게 완성하기
VII	18	내 앱에 적용하기
VIII	19	Flet 앱 —검증 보드와 모델 포함 오픈라인 번역기 만들기

사전 요구사항

이 책을 완주하려면 다음 배경이 필요하다.

- **Rust와 TypeScript 코드를 읽을 수 있어야 한다.** 직접 짤 줄 몰라도 괜찮다. AI가 만들어준 코드를 보고 “이 코드가 뭘 하는 건지” 대략 파악할 수 있으면 충분하다.
- **Git과 터미널이 익숙해야 한다.** 브랜치를 만들고 커밋을 나누는 게 몸에 밴 상태라면, 이 책의 작업 흐름(요청 작성 -> 실행 검증 -> 수정 요청)을 바로 적용할 수 있다.
- **AI 코딩 도구를 한 번이라도 써봤어야 한다.** Claude Code, Cursor, GitHub Copilot, ChatGPT 중 하나라도 실제 작업에 써본 경험이 필요하다. 아직 없다면 토이 프로젝트로 일주일 정도 AI와 함께 작업해보고 돌아와라.

반대로 다음은 **없어도 된다**. Tauri 1 경험, SvelteKit 경험, Rust 경험—전부 몰라도 상관없다. 필요한 내용은 각 장 안에서 설명한다.

Rust가 처음이라면 공식 Rust Book을 전부 읽을 필요는 없다. 변수, 함수, Result, 소유권의 기본 감각 정도만 훑어도 충분하다. AI 코딩 도구를 아직 써본 적이 없다면, “텍스트 파일을 저장하는 작은 앱”처럼 하루 안에 끝나는 토이 프로젝트를 먼저 한 번 만들어보는 것이 좋다.

기술 스택 한눈에 보기

이 책에서 쓰는 핵심 기술이다. 각각을 깊이 알 필요는 없다. **지금은 무슨 역할을 하는지만 파악하고 넘어가면 된다**. 세부 버전을 외우기 위한 표가 아니라, 뒤에서 문제가 생겼을 때 기준으로 돌아오기 위한 표다.

기술	역할	쓰이는 곳
Tauri 2	웹 기술(HTML/CSS/JS)로 데스크톱 앱을 만드는 프레임워크. 설치 파일이 작다.	2~14장
SvelteKit 2 + Svelte 5	이 책의 화면(프론트엔드) 프레임워크.	2~14장

기술	역할	쓰이는 곳
Rust	Tauri 백엔드 언어. 직접 작성할 일은 거의 없다. 읽을 수 있는 정도면 충분하다.	2~17장
DuckDB	서버 없이 파일 하나로 동작하는 분석용 데이터베이스.	7~13장
ratatui	터미널 화면에 UI를 그리는 Rust 라이브러리.	16~17장
Python	강화학습 훈련 스크립트와 Flet 앱의 주 언어.	15장, 19장
Flet	Python 코드로 Flutter 기반 데스크톱·모바일·웹 UI를 만드는 프레임워크.	19장
shadcn-svelte	Tailwind 기반 Svelte UI 컴포넌트 모음.	6~14장
Claude Code	터미널에서 실행하는 AI 코딩 도구. 이 책에서 가장 자주 쓴다.	전 장
Cursor	AI 자동완성을 내장한 코드 에디터. 파일 하나를 빠르게 편집할 때 쓴다.	전 장

각 기술의 세부 동작 방식과 차이점(예: Tauri와 Electron 비교, Svelte 5의 rune 문법 등)은 본문에서 처음 사용할 때 다시 설명한다.

개발 환경

이 책의 모든 코드와 스크린샷은 다음 환경에서 제작되었다. 동일한 버전을 강제할 필요는 없지만, 막혔을 때 비교해 볼 기준점이 된다.

항목	버전
OS	macOS 15.7.5 (Sequoia)
Rust	1.95.0
Node.js	24.15.0 LTS
pnpm	10.33.2
Tauri	2.11.0
SvelteKit	2.59.0 + Svelte 5.55.0
DuckDB	1.5.2
Python	3.14.4 (강화학습 실습 장 한정)
Claude Code	2026년 5월 기준 최신
Cursor	3.0
VSCode	최신 (1.99+)

주의: pnpm 9.x는 2026년 4월 30일부로 지원이 끝났다. pnpm 10을 사용해야 한다. Node.js 22 LTS도 동작하지만, 이 책은 Node.js 24 LTS를 기준으로 작성됐다.

Windows와 Linux에서도 대부분의 예제가 그대로 동작한다. 플랫폼마다 다른 부분이 있으면 해당 장에서 별도로 표시한다.

저자의 노트

이 책은 내가 AI와 함께 개발한 경험을 순서대로 담았다.

프롬프트를 고치고 다시 시도하다가 코드를 직접 수정한 적도 있다. 직접 수정하는 것 자체가 문제는 아니다. 원인을 이해하지 못한 채 임시로 막고 변경 기록과 재검증을 생략하는 것을 경계해야 한다. AI가 그럴듯해 보이는 틀린 코드를 줬는데, 실행도 해보지 않고 그대로 커밋한 기억도 있다. 그렇게 하면 오류가 눈덩이처럼 쌓여 코드를 통째로 버려야 할 수도 있다. 이 책의 학습 방식은 그 과정에서 내가 실제로 생각한 흐름을 따라간다.

직장을 다니며 150일 된 아이를 키우면서 앱 여섯 개를 만든 작업 기록이다. 점심시간에 카페에서 프롬프트를 짜고, 퇴근길 전철에서 코드를 확인하고, 새벽 수유가 끝난 뒤 커밋을 날렸다.

이 책에 나오는 장면 중 일부는 흐름을 위해 약간 재구성했다. 교훈과 코드는 실제다.

시간이 있어서 쓴 책이 아니다. 시간이 없어도 AI 에이전트가 있으면 된다는 걸 말하고 싶어서 만든 기록이다.

마무리 박스: 이 장에서 반드시 기억할 것

머리말이 길었지만, 본문으로 넘어가기 전에 꼭 기억해야 할 것은 네 가지뿐이다.

- **핵심은 Request -> Build -> Verify 사이클이다.** 코드를 처음부터 끝까지 읽는 게 아니라, 실제로 실행해서 동작을 확인한다.
- **여섯 개의 실제 앱**으로 이 사이클을 연습한다. 모두 저자가 매일 쓰는 앱이다.
- **Part I은 순서대로**, Part II 이후는 관심 있는 앱부터 골라 읽어도 된다.
- **모든 기술을 미리 배울 필요는 없다.** 막히는 순간 AI에게 묻고, 직접 실행해 확인하는 방식이 곧 학습 과정이다.

이제 1장에서 “AI가 코드를 짜는데, 왜 막히는 건 사람인가”부터 살펴본다.

박승환 2026년

1장. 속도보다 중요한 건 검증이다—주도권을 쥐어야 하는 이유

Part I —기초 (1~5장) AI와 함께 개발하는 방법을 다룬다. 왜 AI를 쓰는가(1장), 어떤 루프로 움직이는가(2장), 어떤 도구를 어떻게 쓰는가(3~4장), Tauri 2의 핵심 패턴(5장). 이 다섯 장이 이 책 전체의 토대다. 첫 회독에서는 도구 이름을 모두 외우기보다 “요청하고, 만들고, 검증하고, 다시 요청한다”는 루프를 몸에 익히는 데 집중한다. 1장은 왜 검증이 주도권의 핵심인지 설명하고, 2장은 Request -> Build -> Verify 루프를 정의한다. 3~4장은 그 루프를 도구 사용 방식으로 옮기고, 5장은 이후 앱 개발에 필요한 Tauri 2의 기본 경계를 잡는다.

처음엔 AI에게 “만들어 달라”고 하면 끝이라고 생각했다. 나온 코드를 그대로 붙여 넣고, 화면이 뜨면 다음 기능으로 넘어갔다. 며칠 동안은 정말 빠른 것 같았다. 직접 타이핑하던 시간이 사라졌고, 몇 시간 걸리던 초안이 몇 분 만에 나왔다. 하지만 빨라진 속도만큼 잘못된 판단도 빠르게 쌓였다.

예를 들어 이런 한 줄짜리 요청만 던져도 돌아가는 코드가 나왔다.

트레이딩 대시보드 만들어줘. Python FastAPI 백엔드에 WebSocket으로 연결하고, 실시간 차트, Tauri 2로 만들어.

화면도 그럴듯했다. 문제는 그 다음이었다. 연결이 끊겼는데 다시 연결되지 않았다. 상태가 꼬였는데 어느 부분에서 잘못됐는지 찾을 수 없었다. AI에게 다시 고쳐달라고 하면 그럴듯한 코드가 또 나왔지만, 같은 종류의 문제가 모양만 바뀌서 반복됐다.

깨달은 것은 단순했다. 직접 확인하지 않은 코드는 내가 제어하는 코드가 아니었다. AI가 썼는지, 사람이 썼는지는 중요하지 않았다. 어떤 동작을 기대했는지, 어떤 조건에서 실패하는지 확인하지 않고 통합했다면, 그 코드는 시스템 안에 들어온 블랙박스나 다름없었다. 빠르게 생성된다고 해서 주도권을 쥔 것은 아니었다.

그래서 질문이 바뀌었다. “이걸 만들어줘”보다 “이 동작이 정말 맞는지 어떻게 확인할까”를 먼저 물었다. 결과를 실행하고, 실패하는 조건을 좁히고, 그 증거를 다시 요청에 담았다. 이 루프를 반복했다. 이 장은 그 전환에 대한 이야기다. AI가 코드를 빨리 써준다는 사실보다 더 중요한 것이 있다. 그 속도 속에서 개발자가 어떻게 주도권을 잃지 않는냐다.

이 장의 흐름

이 장은 AI 코딩의 속도보다 검증이 왜 중요한지, 그리고 개발자의 역할이 어떻게 바뀌는지 순서대로 본다.

- 왜 계획 없는 AI 코딩은 오래 못 가는가
- 왜 하필 지금인가
- 개발, 그전에는: 직접 코드를 썼다
- 지금은: AI가 쓰고, 내가 결정한다
- “읽기만 하면 된다”는 말이 반만 맞는 이유
- 역할 전환: 코드 검토에서 요구사항 정리로
- 한 장면—한국어 PDF 파서 에피소드
- 이 책에서 반복해서 등장하는 여섯 사례
- 이 책을 읽고 나면 남는 것

학습 목표

- AI가 왜 단순 자동완성이 아닌지 이해한다.

- 계획 없이 AI로 코딩할 때 어디서 막히는지 안다.
- “AI 코드는 읽기만 하면 된다”는 말이 어디까지 맞고 어디서 틀리는지 구분한다.
- 개발자 역할이 타이핑에서 검증과 판단으로 바뀐 이유를 이해한다.
- 이 책에서 다루는 여섯 가지 사례를 미리 살펴본다.

1.1 왜 계획 없는 AI 코딩은 오래 못 가는가

계획 없이 AI로 코딩하면 처음에는 잘 되는 것처럼 보인다. 화면이 뜨고, 버튼이 눌리고, API 응답이 콘솔에 찍힌다. 그래서 검증을 미루기 쉽다. “일단 다음 기능까지 붙이고 나중에 정리하자”는 생각이 자연스럽게 따라온다.

문제는 그렇게 쌓인 코드다. 어느 순간 뭔가 고장 났을 때, 어디가 문제인지 바로 보이지 않는다. 예를 들어 WebSocket(웹소켓, 실시간 양방향 통신) 재연결이 실패했다고 하자. 프론트엔드 상태 문제인지, Rust 쪽 문제인지, Python 백엔드 문제인지 한눈에 파악하기 어렵다. 나도 대시보드가 멈췄을 때 한참 엉뚱한 곳을 의심했다. 실제 원인은 단순했다. 실패 조건을 확인하지 않은 채 AI 결과물을 그냥 커밋한 작업 방식이 문제였다.

그 뒤로 방식을 바꿨다. AI가 코드를 작성하더라도, 다음 단계로 넘어갈지 결정하는 것은 개발자 몫이다. 그 결정은 감이 아니라 검증 결과에 근거해야 한다. 이 책에서 반복하는 **Request, Build, Verify 루프**(요청 -> 생성 -> 검증)는 그 주도권을 유지하기 위한 최소한의 장치다. 속도는 AI에게 맡기되, 판단은 개발자가 가져가는 구조다.

1.2 왜 하필 지금인가

AI는 예전에도 있었다. 자동완성도 있었다. 그렇다면 왜 하필 2026년인가. 세 가지가 같은 시점에 맞물렸다.

첫째, 모델이 달라졌다. 2024년 말부터 Claude 같은 AI 모델들은 여러 파일을 동시에 읽고 이해하는 능력을 갖추기 시작했다. 2026년에는 파일을 읽고, 코드를 고치고, 테스트를 실행하는 작업을 한 번의 대화 흐름 안에서 이어갈 수 있게 됐다. 단순한 자동완성이 아니라, 작은 개발 단위를 통째로 맡길 수 있는 도구가 된 것이다.

둘째, Tauri 2가 정식 출시(GA)됐다. Tauri는 운영체제 WebView를 재사용하므로 Chromium을 함께 번들하는 Electron 앱보다 기본 번들 크기가 작아질 가능성이 크다. 실제 설치 크기와 메모리는 포

함한 사이드카·모델·미디어·플러그인에 따라 달라지므로 프레임워크 이름만으로 수치를 단정하면 안 된다.

기술 세부사항은 5장에서 다루지만, 여기서 주목할 점은 Tauri의 구조다. 화면은 웹 기술로 만들고, 파일 접근이나 시스템 권한이 필요한 작업은 Tauri 명령(프론트엔드에서 호출하는 Rust 함수)으로 처리한다. 역할이 이렇게 나뉘어 있으면 AI에게도 어느 쪽을 고쳐야 하는지 명확히 지시할 수 있다.

셋째, Svelte 5 runes가 나왔다. runes(룬)는 `$state`, `$derived`처럼 `$`로 시작하는 Svelte 5의 새 반응형 문법이다. 자세한 내용은 5장에서 다룬다.

지금은 한 가지만 기억하면 된다. Svelte 5는 `$state`, `$derived`, `$effect`로 반응성의 의도를 코드에 직접 드러낸다. 이 책은 한 프로젝트 안에서 이 문법을 일관되게 사용해 AI가 Svelte 4 예제와 섞지 않도록 한다. 문법이 짧다는 사실만으로 코드 품질이 자동으로 높아지는 것은 아니므로 타입 검사와 동작 검증은 그대로 필요하다.

1.3 개발, 그전에는: 직접 코드를 쳤다

AI가 없던 시절, 개발은 대부분 손가락으로 했다. JIRA 티켓을 읽고 에디터를 열면 브라우저에는 Stack Overflow 탭이 여러 개 떠 있었다. 빠른 코드도 직접 쳐야 했고, 컴포넌트 하나 만드는 데 반나절이 걸리는 일이 흔했다. 그 시절 최고의 생산성 도구는 자동완성이었다. 앞글자 몇 개를 입력하면 IDE가 나머지를 채워주는 것, 그게 전부였다.

코드의 90%가 내 손을 통과했다. 함수명, 변수명, 반복되는 조건문, import 문까지 직접 입력했다. 이 방식에는 장점이 있었다. 손으로 쓰면 코드 구조를 더 오래 들여다보게 된다. 하지만 단점도 분명했다. 타이핑에 시간이 너무 많이 들어가다 보니, 설계를 충분히 생각하기 전에 구현부터 밀어붙이는 일이 잦았다. “일단 돌아가게 만들고 나중에 리팩터링하자”는 말은 계획이 아니라 시간이 없다는 고백이었다. 그리고 대부분의 경우, 그 나중은 오지 않았다.

1.4 지금은: AI가 쓰고, 내가 결정한다

2026년에 달라진 건 AI가 내 일을 대신하게 된 것이 아니다. AI가 타이핑을 맡고, 내가 방향을 결정하게 됐다. 손은 덜 바빠졌지만, 무엇을 요청하고 어디서 멈춰 검증할지는 계속 결정해야 한다. 판단의 빈도는 오히려 늘었다.

어떤 식으로 진행되는지 한 번의 작업으로 보자. 아침에 노트에 아이디어를 적었다. “Mandal-Art 목표 트리 + Pomodoro + AI 코칭 -> 로컬 DuckDB에 저장 -> 전부 Tauri 2로 감싸기.”점심 후에 Claude Code 터미널을 열고 첫 프롬프트를 던졌다.

이 프롬프트는“무엇을 만들지”와“어떤 제약을 따를지”를 짧게 적은 것이다.

Tauri 2 + SvelteKit 2 + Svelte 5로 Mandal-Art 9x9 그리드 컴포넌트를 만들어줘.
셀을 클릭하면 편집 모달이 열리게 해. 중앙 셀이 바뀌면
AI가 주변 빈 셀들의 후보를 제안하게 해.
Svelte 5 runes만 사용해. 파일별로 코드를 나눠줘.

3분 만에 컴포넌트, 스토어, AI 제안용 Tauri 명령 초안이 나왔다. 결과물을 실행해 보니 두 가지가 마음에 들지 않았다. 첫째, 새 문법을 쓰라고 했는데 옛날 문법이 섞여 있었다. 둘째, AI 호출이 프론트엔드에서 직접 일어났다—이러면 API 키가 노출된다.

관찰한 문제를 그대로 다음 프롬프트에 담았다.

두 가지 수정.

1. `$state`, `$derived`, `$effect`만 사용해. ``let``으로 선언하는 반응형 변수 없음.
2. AI 호출을 Tauri 명령으로 옮겨. API 키는 프론트엔드에 노출되면 안 됨.

3분 더 고치자 둘 다 해결됐다. 커밋했다. 예전 같으면 6시간 걸릴 작업이 20분 안에 끝났다.

그러나 그 20분이“AI가 알아서 다 했다”는 뜻은 아니다. 원하는 것을 말로 표현하고, 결과를 실행해보고, 틀린 지점을 찾아 다시 요청했다. 타이핑 양은 줄었지만, 무엇을 원하는지 정리하고 결과를 판단하는 건 여전히 내 몫이었다.

같은 작업에서 AI가 첫 시도에 맞힌 것, 틀린 것, 내가 고치게 한 것을 정리하면 이렇다.

카테고리	예시
맞혔음	9x9 그리드 레이아웃, Tailwind 클래스 조합, 키보드 네비게이션 골격
틀렸음	옛 Svelte 문법이 섞여 있음, 프론트엔드에서 직접 AI 호출, 타입 처리 허술
고치게 함	새 runes 문법으로 통일, AI 호출을 Tauri 명령으로 이동, 에러 타입 정리

이 3행 표가 기능마다 반복된다. 내 하루는 사실 이 표 채우기다.

핵심은 AI가 처음부터 완벽하다는 게 아니다. 짧게 요청하고, 짧게 수정하고, 짧게 검증하는 루프를 빠르게 반복할 수 있다는 점이다. 예전보다 빨라진 이유도 거기에 있다. 한 번에 맞히는 능력보다, 틀렸을 때 바로 좁혀서 다시 시도하는 능력이 더 중요했다.

1.5 “읽기만 하면 된다”는 말이 반만 맞는 이유

지금 돌아보면 버그를 찾으려고 코드를 한 줄씩 읽을 필요는 없었다. 하지만 처음엔 그걸 몰랐다.

이 말은 반은 맞고, 반은 틀리다.

맞는 부분부터 보자. 이제 Rust 문법을 통째로 외울 필요는 없다. AI가 코드를 쓴다. 나는 읽으면 된다. “여기 이 처리가 맞나?”라고 물을 수 있으면 충분하다.

하지만 거기서 끝나면 틀린다. “읽을 수 있다”의 기준이 예전보다 훨씬 높아졌기 때문이다. AI가 건넨 코드에 문제가 있는지 판단할 수 있어야 한다. 지금 당장 Rust를 몰라도 된다. 5장까지 따라오면 “이런 코드가 보이면 위험하다”는 신호 정도는 눈에 들어온다. 그 수준이면 충분하다.

필요한 능력은 두 가지다. 위험 신호를 알아보는 능력, 그리고 “이 결과가 내가 원하는 것인가”를 판단하는 능력이다. 위험 신호는 거창한 것이 아니다.

Tauri 명령 안에 `unwrap()` 이 남아 있어 오류가 앱 전체를 죽일 수 있는 코드, 프론트엔드에서 AI API를 직접 호출해 키가 노출되는 코드, Tauri 1 방식의 API를 섞어 쓰는 코드, Rust에서 보내는 이벤트 이름과 Svelte에서 듣는 이름이 다른 코드, DB 스키마에 외래 키나 CHECK 제약이 빠진 코드가 모두 신호다. 이런 패턴이 보이면 “이 줄이 틀렸다”가 아니라 “검증해야 할 조건이 생겼다”고 보면 된다.

주의: 이것은 Mandai 프로젝트를 시작할 때 내가 가진 생각이었다. 끝까지 해보니 모든 코드를 같은 깊이로 읽는 방식이 병목이었다. 컴파일러와 cargo clippy, 동작 테스트는 많은 오류를 빠르게 드러내지만, 테스트하지 않은 요구사항·보안 경계·데이터 손실까지 보장하지는 않는다. 그래서 생성된 코드를 무조건 한 줄씩 읽는 대신, 위험한 경계는 직접 검토하고 나머지는 실행 가능한 검증으로 확인했다. 2장에서는 동작 검증을 코드 리뷰와 함께 쓰는 루프를 보여준다.

그렇다면 검증은 어떻게 하나? 코드를 읽는 게 아니라 **실행 결과를 증거로 다음 요청에 넣는다**. 한국어 PDF가 깨진다면, 깨진 글자 몇 줄을 그대로 복사해서 다음 프롬프트에 붙여넣으면 된다. 1.7절에서 이 방식이 실제로 어떻게 동작하는지 보자.

1.6 역할 전환: 코드 검토에서 요구사항 정리로

2020년에 내가 하루 중 가장 많이 한 일은 **코드 쓰기**였다. 2026년에 가장 많이 하는 일은 **프롬프트 쓰기**다.

프롬프트를 쓴다는 건 요구사항을 쓴다는 뜻이다. 회의 후 짧게 남기는 메모와는 다르다. AI에게 주는 프롬프트는 훨씬 더 **정확해야** 한다. 사람은 “알아서 해”라는 말을 어느 정도 스스로 해석한다. AI는 그렇지 않다. 이해한 것처럼 보이다가 한참 후에 전혀 다른 결과물을 내놓는다.

예전에는 다 만든 뒤에 검토했다. 지금은 만들기 전에 모호함부터 없애야 한다. 요구사항과 내 판단을 먼저 구분해두지 않으면, AI는 빈칸을 자기 방식으로 채운다.

그래서 매일 이런 항목을 머릿속에 정리한다.

- **무엇을:** 함수인지 컴포넌트인지 앱 전체인지.
- **왜:** 더 큰 기능 안에서 어디에 들어가는지.
- **제약:** 언어, 라이브러리, 성능 목표, 파일 구조.
- **출력 형태:** 코드 블록, 파일 목록, 변경된 부분만, 설명 길이.

익숙해지면 네 항목을 한 문장에 담을 수 있다. 2장에서 루프의 일부로 다룬다.

예전의 **코드 리뷰**는 다른 사람이 쓴 코드를 사후에 읽는 일이었다. 지금의 **요구사항 정리**는 AI가 뭔가를 만들기 전에 머릿속을 정리하는 일이다. 잘못 만든 뒤 고치는 게 아니라, 잘못 만들 가능성을 처음부터 줄인다. 순서가 뒤바뀐 것—그게 가장 큰 변화고, AI와 일할 때 생산성이 오르는 이유다.

1.7 한 장면—readbooks.ai의 PDF 파서 에피소드

readbooks.ai를 Tauri 2로 다시 만드는 작업 중에 PDF에서 텍스트를 꺼내는 기능이 필요했다. 첫 번째 프롬프트는 짧았다.

PDF 텍스트를 추출하는 Rust 기반 Tauri 명령.

Claude는 pdf-extract 라이브러리를 쓰는 20줄짜리 코드를 내놔다. 실행해봤더니 영어 PDF는 잘 됐다. 한국어 PDF는 **깨진 글자 박스들**만 나왔다.

예전이라면 검색창에 “rust pdf korean encoding”을 쳤을 거다. 그 대신 관찰한 결과를 그대로 들고 다시 물었다.

한국어 PDF가 깨져서 나온다. 샘플 입력의 첫 페이지가 이거다:
<깨진 텍스트 30줄>

후보 원인 3개를 나열하고 각각의 간단한 수정 방법을 알려줘,
그 다음에 가장 현실적인 것을 고르고 수정된 명령을 줘.

Claude는 원인 후보 세 가지를 나열한 뒤, 그중 가장 가능성이 높은 것을 골라 다른 라이브러리를 쓰거나 외부 도구를 호출하는 방법을 제안했다. 정확한 진단이었다.

여기서 중요한 건 Claude가 처음에 틀렸다는 사실이 아니다. 내가 **직접 관찰한 결과를 다음 프롬프트에 다시 넣어서** 요청을 구체화했다는 점이다. 예전이라면 “이걸 어떻게 고치지”를 검색창에 쳤을 거다. 지금은 **내가 본 것을 AI에게 정확히 보여주고, 그 위에서 다음 가설을 만든다.**

이 책에서 반복되는 패턴이 바로 이거다. 짧은 첫 요청, 결과를 증거로 다시 제시하기, 후보 가설 나열, 현실적인 선택. AI에게 답을 일방적으로 기대하는 방식이 아니다. 이 흐름이 반복 가능한 작업 방식의 토대가 된다.

1.8 이 책에서 계속 등장하는 여섯 가지 사례

머리말에서 소개한 여섯 프로젝트—Mandai, readbooks.ai, Trading Dashboard, RL & Backtest Lab, TUI Trading Monitor, TUI Log Viewer —가 이 책 전반에 걸쳐 반복해서 등장한다. 여기서는 이 장의 주제와 이어지는 부분만 먼저 살펴본다.

여섯 프로젝트는 각각 다른 문제에서 시작했다. Mandai는 작업을 취소할 때마다 UI가 멈췄다. readbooks.ai는 앞서 살펴봤듯이 한국어 PDF를 처리하면 글자가 깨졌다. 트레이딩 대시보드는 실시간 연결이 끊긴 뒤 다시 연결되지 않았다. RL Lab은 프로세스가 멈춰도 메모리에서 정리되지 않고 계속 남아 있었다.

공통점은 하나다. 문제가 눈에 보일 때만 AI가 쓸모 있었다. 무엇이 잘못됐는지 모르면 AI에게 설명조차 할 수 없다.

이 책의 사례들은 성공담 모음이 아니다. 실패를 어떻게 좁히는지, 그것을 어떻게 요청으로 바꾸는지, 어디서 사람이 직접 판단해야 하는지를 보여주는 재료들이다.

1.9 이 책을 읽고 나면 남는 것

이 책을 끝내도 Tauri 2 API를 전부 외우지는 않는다. Rust 전문가가 되는 것도 아니다. 그게 목표가 아니다.

가져가는 건 **같은 방법으로 자신만의 앱을 만드는 능력**이다. 구체적으로 다섯 가지다.

1. **요청을 정확하게 쓰는 근육.** 머릿속의 “이런 느낌”을 AI가 실행할 수 있는 프롬프트로 바꾸는 능력이다.
2. **AI 출력을 판단하는 눈.** 맞는 코드와 틀린 코드를 구별하고, 맞지만 위험한 코드도 잡아낸다. Tauri 2에서 초보자가 흔히 부딪히는 함정들을 미리 알아둔다.

3. **개발 루프를 끊지 않는 리듬.** 첫 시도가 완벽하지 않다는 걸 받아들이고, 짧은 사이클로 끝낸다.
4. **여섯 개의 실제 참조 프로젝트.** AI가 쓴 부분, 내가 고친 부분이 모두 남아 있다. 비슷한 걸 만들 때 거의 그대로 쓸 수 있는 패턴들이다.
5. **인터페이스를 고르는 판단.** 사용자 경험이 중요하면 GUI, 속도가 중요하면 TUI(터미널 기반 UI)다. 어떤 방향을 선택하느냐에 따라 이후 모든 프롬프트 방향이 달라진다.

이 다섯 가지는 특정 기술 스택에 묶여 있지 않다. Tauri가 아니어도, Rust가 아니어도 남는 건 요청하고 판단하고 검증하는 방식이다.

마무리 박스: 이 장에서 반드시 기억할 것

- AI 코딩에서 중요한 건 “빨리 만든다”가 아니라 “직접 검증한다”는 태도다. 검증하지 않은 코드는 내 것이 아니다.
- 무계획한 AI 코딩은 처음에는 빠르지만, 문제가 생기는 순간 어디부터 의심해야 할지 모르게 된다.
- 2026년에는 모델, Tauri 2, Svelte 5가 함께 안정되면서 AI를 실제 개발에 끼워 넣을 조건이 갖춰졌다.
- 개발자의 일은 타이핑에서 **요청하기 -> 결과 확인하기 -> 다시 요청하기**로 이동했다.
- “AI 코드는 읽기만 하면 된다”는 말은 반만 맞다. 문법 외울 필요는 줄었지만, 실행 결과를 증거로 다음 요청에 다시 넣는 능력이 새로 필요해졌다.
- 이 책은 여섯 개 실제 사례를 따라가며 그 작업 방식이 어떻게 동작하는지 보여준다.

2장에서는 이 루프를 실제로 어떻게 실행하는지 단계별로 풀어낸다—각 단계의 시작 조건, 끝내는 조건, 그리고 반복하면 프로젝트를 흔드는 잘못된 습관까지.

2장. Request, Build, Verify —루프의 형태

이 책의 모든 장은 같은 루프 위에서 움직인다. 다루는 코드는 장마다 다르다. DuckDB 스키마를 만들기도 하고, WebSocket 재연결을 고치기도 하고, Python 프로세스를 Tauri 앱 안에서 실행하기도 한다. 하지만 작업 방식은 바뀌지 않는다. 요청하고, AI가 만든 결과를 받고, 실행해서 확인하고, 실패한 부분만 다시 요청한다.

1장에서 본 핵심 어려움은 **AI가 그럴듯한 결과를 사람이 검증할 수 있는 속도보다 빠르게 만들어낸다는 점**이었다. 이 격차를 그대로 두면 검증되지 않은 코드 위에 다음 기능이 쌓이고, 세 번째 기능을 엮는 순간 어디가 원인인지 추적하기 어려워진다.

이 장은 그래서 루프를 의도적으로 촘촘하게 정의한다. 단계마다 시작 조건과 끝내는 기준을 둔다. 뒤의 프로젝트 장들은 이 루프를 실제 사례에 적용한 기록이다. 루프를 먼저 이해해야, 이후 장에서 내가 왜 특정 요청을 하고 왜 특정 지점에서 멈춰 검증하는지 보인다.

이 장의 흐름

- 학습 목표
- 이 장에서 쓰는 기술
- 하나의 루프, 크기만 달라질 뿐
- 1단계—요청
- 2단계—구현
- 3단계—검증
- 4단계—반복: 검증 결과를 다음 요청으로 돌려보내기
- “완벽한 프롬프트 하나로 앱 완성”이 허상인 이유
- 뼈대 우선 전략 (탐다운 AI 개발)
- 실제 사례—Mandai DB 스키마 루프
- 기본 프롬프트 템플릿
- 루프 돌릴 때 주의할 점

학습 목표

- 이 책 내내 쓰는 핵심 루프 **요청 -> 구현 -> 검증**을 파악하고, 검증 결과를 다음 요청으로 되돌리는 방식을 익힌다.
- 각 단계에서 지켜야 할 원칙과 자주 발목을 잡는 안티패턴을 짚는다.
- “완벽한 프롬프트 하나로 앱 완성”이 왜 허상인지 본다.
- **뼈대 우선** 전략을 배운다: 전체 구조부터 잡고, 세부 기능으로 내려가며, 버그를 고치고, 기능을 검증한 뒤 다듬는다.
- Mandai의 초기 DB 스키마로 루프를 처음부터 끝까지 실제로 돌려본다.
- **무엇을 / 왜 / 제약사항 / 형식** 프롬프트 템플릿을 직접 써본다.

이 장에서 쓰는 기술

기술 / 라이브러리	버전	이 장에서의 역할
Tauri	2.11.0	뼈대 우선 전략 예제에서 Rust 모듈·명령 구조 설계의 대상 프레임워크

기술 / 라이브러리	버전	이 장에서의 역할
SvelteKit	2.59.0	빠대 프롭트에서 라우트와 컴포넌트 구조를 설계하는 프론트엔드 프레임워크
DuckDB	1.5.2	Mandai DB 스키마 루프 실습에서 SQL 스키마 설계의 대상 데이터베이스
tokio	-	비동기 명령 예제에서 이름만 언급. 실제 코드 작성은 5장 이후에서
thiserror	-	빠대 우선 전략 예제에서 이름만 언급. 실제 코드 작성은 5장 이후에서

2.1 하나의 루프, 크기만 달라질 뿐

이 책 전체에서 AI와 작업하는 방식은 하나로 정리된다.

요청 → 구현 → 검증
 ↓
 (필요하면) 다시 요청 → ...

이 한 줄이 책 전체의 작업 모델이다. 함수 하나를 작성할 때도, 앱 전체 아키텍처를 설계할 때도 똑같이 쓴다. 달라지는 것은 한 번 도는 데 걸리는 시간이다. 작게 돌리면 몇 분, 크게 돌리면 하루 이상. 어느 쪽이든 **검증 없이 다음 단계로 넘어가지 않는다**는 규칙은 같다.

루프의 크기는 대략 세 단계로 나눌 수 있다.

- **작은 루프 (5-15분):** “이 Svelte 컴포넌트에 키보드 단축키를 추가해줘.”
- **중간 루프 (30분-2시간):** “readbooks.ai의 PDF 업로드 파이프라인을 Tauri 명령으로 옮겨줘.” (readbooks.ai는 저자가 만든 PDF 번역 앱이다. **Tauri 명령(Command)**은 프론트엔드에서 `invoke("이름")`을 호출하면 Rust 함수가 실행되는 통로다.)
- **큰 루프 (하루-일주일):** “Trading Dashboard 19개 화면의 공통 레이아웃과 차트 컴포넌트 설계를 확정해줘.”

구분 기준은 시간만이 아니다. **작은 루프**는 실패했을 때 한 파일이나 한 함수만 다시 보면 된다. **중간 루프**는 화면, 명령, DB 중 두 계층 이상이 연결되므로 통합 검증이 필요하다. **큰 루프**는 여러 화면이나 공통 구조를 바꾸기 때문에, 먼저 설계 검증을 하고 그 안을 작은 루프로 쪼개야 한다.

크기가 커질수록 한 단계에 더 오래 걸리고, 큰 루프 안에는 작은 루프들이 중첩된다. 지금 돌리는 루프가 어떤 크기인지 파악해두면 전체 흐름을 잃지 않는다. 실패했을 때 원인을 한 문장으로 설명할 수 없다면, 루프가 너무 큰 것이다.

2.2 1단계—요청

요청은 AI에게 맥락, 제약사항, 목표를 전달하는 일이다. 자연어로 쓰지만 담아야 할 내용은 기술 명세서에 가깝다. 좋은 요청은 “무엇을 만들 것인가”만 말하지 않는다. 왜 필요한지, 어디에 붙는지, 어떤 방식은 피해야 하는지, 결과물이 어떤 형태여야 하는지도 함께 담는다. 이 정보가 빠지면 AI는 빈칸을 스스로 채운다. 그 가정이 실제 프로젝트와 어긋나면, 다음 루프 두세 번은 빠진 맥락을 메우는 데 쓰인다. 좋은 요청은 보통 세 가지 조건을 만족한다.

1. **맥락을 포함한다.** 이 기능이 코드베이스 어디에 붙는가? 주변 코드가 뭘 하는가?
2. **제약사항을 명시한다.** “Svelte 5 runes만 사용.” “모든 것을 한 파일에 묶여넣지 말 것.”
3. **결과물 형태를 정한다.** 코드 블록인가? 파일 세 개인가? “코드만, 설명은 없이”가 그 예다.

이 세 조건이 빠질 때 어떤 모습이 되는지 두 가지 안티패턴으로 보자.

너무 짧은 요청

가장 자주 보이는 실수다.

Tauri에서 DB 명령을 써줘

이런 요청을 받으면 AI는 비어 있는 정보를 알아서 채운다. DB 종류, 명령 이름, 오류 처리 방식까지 전부 AI가 결정한다. 문제는 그 결정이 틀렸을 때도 겉으로는 그럴듯해 보인다는 점이다. 프로젝트가 DuckDB를 쓰는데 SQLite 예제가 나오는 식이다. 다음 몇 라운드는 기능 개발이 아니라, 처음에 빠진 맥락을 뒤늦게 보충하는 데 쓰인다.

너무 긴 요청

반대로, 한 번에 너무 많은 일을 시키는 것도 위험하다.

Tauri 2 앱을 처음부터 만들어줘. SvelteKit(Svelte 5)를 사용하고, DuckDB를 임베드하고, OpenAI를 연결하고, Mandal-Art UI를 만들고, Pomodoro를 추가하고, GTD 워크플로우를 포함시켜줘. 모든 파일을 짧게 유지하고 주석을 추가해줘. Tailwind도 사용해줘...

열 가지를 한 번에 요청하면 AI는 대개 열 가지를 모두 알게 처리한다. 각 기능은 단독으로 보면 그럴싸하지만 서로 맞물리지 않는다. DB 스키마는 UI 상태와 맞지 않고, Tauri 명령 이름은 프론트엔드 호출부와 다르게 된다. 다섯 개의 작은 요청으로 나뉘었다면 단계마다 검증할 수 있었다.

2.3 2단계—구현

구현은 AI가 돌려주는 결과물이지만, 아직“이렇게 하면 될 것 같다”는 주장에 가깝다. 코드일 수도 있고, 설계 다이어그램이나 파일 트리일 수도 있다. 결과물이 나왔다고 루프가 끝난 게 아니다—검증 단계가 그 주장을 사실로 확정해주기 전까지는.

확인 방법은 산출물 종류에 따라 다르다. 아키텍처 스케치, DB 스키마, 파일 트리 같은 설계 산출물은 먼저 **읽는다**. 모듈 하나를 추가하거나 빼는 일은 코드 200줄을 되돌리는 것보다 훨씬 싸다. 반면 구현 코드는 줄 단위로 오래 읽기보다 먼저 **실행한다**. 겉으로 자연스러워 보이는 코드가 런타임에서 틀리는 경우가 많기 때문이다.

“실행 전에 모두 읽는다”는 습관은 머리말에서 말한 코드 리뷰 병목을 만든다. AI가 코드를 생성하는 속도는 내가 줄 단위로 읽는 속도보다 빠르다. 컴파일러와 동작 테스트는 내가 수동으로 찾으려던 버그를 더 빠르고 일관되게 잡아낸다. 코드를 이해하는 능력은 여전히 필요하다. 다만 모든 줄을 먼저 읽을 필요는 없다. **먼저 실행하고, 실패한 지점을 중심으로 읽는다**. 이“실행으로 주장을 확정하는”단계를 다음 절에서 자세히 본다.

2.4 3단계—검증

검증은 실행, 테스트, 리뷰다. 보통 다음 순서로 진행한다.

1. **컴파일과 타입 체크**. Rust는 cargo check(컴파일은 하지만 실행 파일은 만들지 않는 빠른 검사), TypeScript는 tsc --noEmit(타입 검사만 하고 파일을 출력하지 않음). 실패하면 컴파일러 출력을 그대로 붙여 요청 단계로 돌아간다.
2. **제약사항이 지켜졌는가**. 내가 지정한 동작을 직접 실행해본다.
3. **기존 기능이 멀쩡한가**. 인접 기능을 실행해서 회귀가 없는지 확인한다.

4. **엣지 케이스.** 잘못된 입력, 경계값, 오류 경로 세 가지를 직접 시도한다. 중요한 기능은 AI에게 단위 테스트를 작성하게 한다.

체크를 먼저 하는 이유는 속도다. 앱을 실행하기 전에 컴파일러와 타입 검사기가 틀린 이름, 빠진 import, 맞지 않는 반환 타입을 빠르게 잡아준다. 실행 검증은 그다음이다. **체크는 코드가 돌아갈 자격을 확인하고, 실행은 실제로 원하는 일을 하는지 확인한다.**

루프가 작으면 수동 실행 한 번이면 충분하다. 루프가 크면 테스트를 먼저 쓰고 AI가 그에 맞춰 구현하게 한다. 도구 사용법(cargo check, tsc --noEmit, 테스트 구성)은 4~5장에서 환경 설정과 함께 자세히 다룬다.

코드를 훑어보고 “맞는 것 같다”며 넘기는 건 검증이 아니다. 눈으로는 문제없어 보여도, 권한 파일이 빠졌거나 이벤트 이름이 프론트엔드와 다른 경우는 직접 실행해야만 드러난다.

검증을 건너뛰고 구현 두 개를 연달아 넣으면 첫 번째 오류가 두 번째 변경 아래에 묻힌다. 세 번째 기능까지 없으면 어떤 문제가 어느 변경에서 생겼는지 추적하는 데 기능 구현보다 더 많은 시간이 든다. **검증은 마지막에 한 번 하는 절차가 아니라, 구현 사이마다 루프를 닫는 동작이다.**

2.5 4단계—반복: 검증 결과를 다음 요청으로 돌려보내기

반복은 별도의 산출물을 만드는 단계가 아니다. 검증에서 발견한 실패, 로그, 화면 증상, 빠진 제약을 다음 요청에 넣어 루프를 다시 여는 동작이다. **반복할 때는 틀린 부분만 다시 요청한다.** 처음부터 전부 다시 써달라고 하지 않는다.

위 코드에서 다음 두 가지만 고쳐줘.

1. <구체적인 포인트 1>
2. <구체적인 포인트 2>

나머지는 그대로 뒀.

“나머지는 그대로 뒀”가 핵심이다. 이 한 줄이 없으면 AI는 건드리지 않아도 될 부분까지 바꾼다. 그러면 방금 확인한 코드를 처음부터 다시 검증해야 한다.

같은 요청으로 세 번 실패하면 반복을 멈추고 요청을 처음부터 다시 쓴다. 여기서 “세 번”은 단순히 메시지 횟수가 아니다. **같은 오류가 반복되거나, 새 오류가 계속 같은 영역에서 생기거나, AI가 이미 통과한 코드를 다시 혼드는 경우**를 말한다. 원인은 보통 셋 중 하나다.

- 제약사항을 빠뜨렸다.
- 맥락이 부족하다.
- 한 번에 너무 많은 것을 요청했다.

에러를 계속 붙여 넣기보다 요구사항과 제약을 다시 정리하는 편이 빠르다.

2.6 “완벽한 프롬프트 하나로 앱 완성”이 허상인 이유

루프를 작게 쪼개야 하는 이유는, 한 번에 끝내려는 시도가 실제로 잘 안 되기 때문이다. X(구 Twitter)에는 “프롬프트만 잘 쓰면 앱이 똑딱 완성된다”는 글이 넘쳐난다. 실제로 해보면 다르다. 이유는 세 가지다.

1. **맥락의 한계.** 프롬프트가 아무리 길어도 앱 전체를 한 번에 담을 수 없다.
2. **모호함이 누적된다.** 자연어로 쓴 요구사항은 조건이 열 개 쌓이면 앞쪽 조건이 다르게 해석될 수 있다.
3. **검증이 어렵다.** 한 번에 나온 결과물은 처음부터 끝까지 통째로 확인해야 하고, 문제가 생기면 어디서 틀렸는지 찾기 어렵다.

이 세 문제는 요청을 작게 나누면 대부분 해결된다. 그래서 나는 **짧은 요청을 여러 번** 사용한다. 요청 하나에 넣는 요구사항은 세 개를 넘기지 않는다. 다음 절의 “빠대 우선 전략”은 이 원칙을 더 큰 규모, 앱 전체 수준에서 적용한 결과다.

2.7 빠대 우선 전략—탑다운 AI 개발

이 책에서 가장 중요한 아이디어다. 나머지는 전부 이것의 세부사항이다.

여기서 말하는 다섯 단계는 2.2~2.5절의 “요청 -> 구현 -> 검증”한 바퀴와 다르다. 한 바퀴의 루프를 프로젝트 전체에 여러 번 배치하는 큰 작업 순서다. 먼저 빠대를 검증하고, 그 안에서 작은 루프를 여러 번 돌린다고 이해하면 된다.

Tauri 앱이든, TUI 도구든, 백엔드 서비스든—AI로 무엇을 만들든 세부사항보다 빠대를 먼저 잡는다. 전체 전략은 다섯 단계다.

- 1단계: 빠대 —아키텍처, 파일 구조, 인터페이스, 데이터 흐름
- 2단계: 세부 구현 —각 모듈을 하나씩 구현
- 3단계: 버그 수정 —컴파일, 실행, 모든 에러 수정
- 4단계: 기능 QA —각 기능을 처음부터 끝까지 검증
- 5단계: 다듬기 —옛지 케이스, 성능, UX

큰 결정에서 작은 결정으로 내려가는 깔때기 모양이다.

AI는 이 순서를 모른다. 기능을 요청하면 한 번에 구조·로직·앳지 케이스·에러 처리를 전부 만들려 한다. 결과물은 완성된 것처럼 보이지만 취약하다. 한 곳을 바꾸면 세 곳이 깨진다.

개발자가 각 단계의 범위를 직접 제한하면 결과 품질이 올라간다. 뼈대를 설계할 때는 코드를 쓰지 못하게 하고, 모듈을 구현할 때는 다른 모듈을 건드리지 못하게 한다.

1단계: 뼈대—아키텍처 프롬프트

첫 번째 프롬프트는 절대 “X를 만들어줘”가 아니다. “X의 뼈대를 설계해줘”다.

아래는 Mandai 프로젝트에 실제로 사용한 뼈대 프롬프트다. Mandai는 GTD(할 일을 수집·처리·정리하는 생산성 방법론) 태스크 관리와 DuckDB(앱 안에 내장하는 경량 데이터베이스)를 결합한 Tauri 앱이다. **코드 없이 구조만 받는 것이** 이 프롬프트의 목적이다.

[맥락]

Mandai라는 Tauri 2 + SvelteKit(Svelte 5 runes) 앱을 만들고 있다.
Mandal-Art (9×9 목표 그리드), GTD 태스크, 포모도로 타이머,
멀티 AI 챗(OpenAI, Claude, Gemini)을 결합한다.

[요청]

프로젝트 뼈대를 설계하라.

- src-tauri/의 모든 Rust 모듈과 각 모듈의 역할 (한 줄씩)
- src/routes/의 모든 Svelte 라우트와 렌더링하는 화면
- 모든 Tauri 명령 이름, 입력, 반환 타입
- DuckDB 테이블 (이름 + 컬럼, SQL은 아직 쓰지 말 것)

[제약사항]

- 구현 코드를 작성하지 말 것
- SQL을 작성하지 말 것
- 구조만 출력할 것

가장 중요한 줄은 제약사항이다. “구현 코드를 작성하지 말 것”이 빠지면 AI는 묻지 않아도 코드를 함께 토해낸다. 그러면 다시 코드 검토 모드로 끌려가고 뼈대를 따지는 단계의 이점을 잃는다.

나오는 건 **설계도**다. 제품이 아니다. 설계도는 읽기 쉽고, 바꾸기 쉽다. AI가 원치 않는 모듈을 추가하면 한 줄 지우면 된다—코드 200줄이 아니라.

뼈대 수준의 변경은 싸다. 코드 수준의 변경은 비싸다. 코드를 작성하기 전에 항상 뼈대를 확정한다.

2단계: 세부 구현—한 번에 하나의 모듈

뼈대가 확정되면 모듈 하나를 골라서 깊이 들어간다. 두 개가 아니다. 핵심은 **전체 뼈대를 맥락에 넣되, 요청은 한 조각에 한정**한다는 점이다.

포 모도로 기능의 최소 요청은 다음처럼 쓸 수 있다. “start_pomodoro(minutes)와 stop_pomodoro(session_id)를 구현하라. 0분과 중복 시작은 거부하고 세션 ID·시작 시각·종료 예정 시각을 돌려줘라. stop은 같은 ID로 두 번 호출해도 한 번만 종료되게 하라. 타이머는 tokio::time을 쓰고 오류는 thiserror로 정의하라. 정상 종료·중복 시작·잘못된 ID·앱 재시작의 기대 동작을 먼저 표로 작성하라.” 함수 수보다 입력·실패·복구 조건이 명확해야 한다.

이 모듈이 완성되고 검증되면 다음 모듈로 넘어간다. 같은 패턴을 반복한다. **모듈 하나, 깊이 있게, 검증 후 다음으로.**

3단계: 버그 수정—컴파일, 실행, 수정

모듈 몇 개가 구현되면 전체 프로젝트를 컴파일한다. 이 시점에 빌드가 실패하는 건 정상이다. AI가 각 모듈을 분리해서 작성했기 때문에 import 경로가 틀리고, 모듈 경계에서 타입이 안 맞고, 함수 시그니처가 바뀌어 있다.

버그 수정 프롬프트는 기능 구현 프롬프트와 다르다. **정밀하고 짧다.** 핵심은 두 가지다.

- **컴파일러 출력을 그대로 붙여넣는다.** 의역하지 않는다.
- **“이 두 에러만”이라고 못 박는다.** 빠지면 AI는 멀쩡한 함수까지 손대기 시작한다.

프로젝트가 다음 에러들로 컴파일에 실패한다:

```
error[E0308]: mismatched types
--> src/pomodoro.rs:42:12
   expected `i64`, found `String`
```

이 에러만 수정하라. 다른 것은 건드리지 마라.

4단계: 기능 QA—각 기능을 처음부터 끝까지 검증

컴파일되고 실행된다고 끝이 아니다. 기능 하나하나를 실제 사용자처럼 직접 써본다.

QA 루프는 이렇다:

1. 기능 하나를 고른다 (예: “태스크에 연결된 포모도로 세션 시작”)

2. 수동으로 실행한다. 동작하는가?
3. 안 되면: 정확한 실패 지점을 파악하고, 증상과 함께 AI에 프롬프트
4. 수정 -> 재검증 -> 다음 기능으로

버그 리포트 프롬프트의 포인트는 **기대 동작과 실제 동작을 한 줄씩** 명확히 적는 것이다.

[버그 리포트]

task_id = 5로 포모도로 세션을 시작한 후 중지하면,
세션은 DuckDB에 저장되지만 태스크의 accumulated_minutes가
업데이트되지 않는다.

기대 동작: task.pomodoro_minutes가 세션 시간만큼 증가

실제 동작: task.pomodoro_minutes가 0에 머무름

stop_pomodoro 함수를 보여달라. 태스크 업데이트는 어디서 일어나야 하는가?

마지막 줄이 핵심이다. AI에게 코드를 곧장 고치라고 시키지 않고 **잘못된 위치를 먼저 짚게** 만든다. 위치를 잘못 짚으면 그 자리에서 멈추고 다시 요청하면 된다.

5단계: 다듬기—엣지 케이스, 성능, UX

다듬기는 마지막 단계다. 정상 경로가 커버하지 못한 엣지 케이스, 실제 데이터에서만 드러나는 성능 이슈, 한 시간 써보고 나서 발견하는 UX 불편함을 여기서 처리한다. 프롬프트는 보통 엣지 케이스를 번호로 정리한다(예: 포모도로 타이머의 (1) 앱 재시작 시 중단된 세션 복구, (2) 중복 시작 방지, (3) 0분 세션 저장 금지). 번호로 매기면 응답이 빠뜨린 항목을 곧바로 찾을 수 있다.

이 순서가 중요한 이유

다섯 단계는 **앞 단계에 의존**한다. 앞 단계가 안정되지 않으면 뒤 단계가 흔들린다.

- 버그를 잡기 전에 코드를 다듬으면, 곧 바뀔 코드를 다듬는 셈이다.
- 모든 모듈을 구현하기 전에 버그를 잡으려 하면, 다음 모듈이 추가되면서 그 버그가 저절로 사라질 수 있다.
- 뼈대를 확정하기 전에 세부 기능을 구현하면, 아키텍처가 바뀔 때 모듈을 처음부터 다시 써야 한다.

AI 보조 개발에서 시간을 가장 많이 낭비하는 원인은 단계를 뒤섞는 것이다. 순서를 지키면 단계마다 빨라지는 게 아니라 **재작업 자체가 줄어든다.**

주의: “이것만 빨리 고치자”는 유혹은 반드시 찾아온다. 그 자리에서 고치지 말고 목록에 적어두었다가 맞는 단계에서 처리한다. AI 보조 개발에서 컨텍스트 스위칭 비용은 생각보다 크다.

2.8 실제 사례—Mandai DB 스키마 루프

이론은 충분히 봤다. 실제 사이클을 살펴보자. Mandai의 초기 DB 스키마를 확정하던 과정이다. 두 번의 사이클, 약 15분이 걸렸다.

DB 스키마를 첫 사례로 고른 이유는 단순하다. **코드 분량이 작아 한 화면에 들어오기 때문에, 루프 한 바퀴 전체가 한눈에 잡힌다.** 요청과 결과물, 검증과 수정이 같은 페이지에서 보이는 작업은 루프를 처음 익히기에 가장 좋다.

사이클 1 —요청

스키마 전체를 짧은 요청에 담았다. 핵심은 **9×9 격자의 셀 구조, 필수 컬럼 목록, DuckDB 유효 SQL**이다.

Mandai용 초기 DuckDB 스키마를 초안으로 작성해줘.

Mandal-Art는 9x9 격자다. 중앙 3x3의 각 셀이 하위 3x3의 중심이 된다.

각 셀은 title, description, done flag를 가진다.

누적 Pomodoro 분과 created/updated timestamps도 포함한다.

DuckDB 유효한 SQL만 사용. 인덱스 포함. 파일 하나에.

사이클 1 —구현

Claude가 생성한 초안에서 핵심 부분만 추렸다. 이 코드는 **셀과 부모-자식 관계, 기본 메타데이터 컬럼**을 정의한다.

```
CREATE TABLE cells (
  id INTEGER PRIMARY KEY,
  parent_id INTEGER REFERENCES cells(id),
  position INTEGER NOT NULL,
  title TEXT, description TEXT,
  done BOOLEAN DEFAULT false,
  pomodoro_minutes INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT now(),
```

```
updated_at TIMESTAMP DEFAULT now()
);
CREATE INDEX idx_cells_parent ON cells(parent_id);
```

읽을 때 짚을 줄은 셋이다. `parent_id`(격자의 계층 구조를 만든다), `position`(셀의 위치, 0-8이어야 한다), `updated_at`(기본값이 `now()`로 한 번만 설정된다—자동 갱신이 아니다).

사이클 1 —검증

검토하니 세 가지 문제가 보였다.

1. `position` 컬럼에 범위 제약이 없다. 0-8만 유효한데 스키마가 강제하지 않는다.
2. DuckDB는 SQLite와 자동 증가 ID 문법이 다르다. 초안에는 SQLite 문법이 섞여 들어갔다.
3. `updated_at`이 자동으로 채워지지 않는다. 앱 코드에서 직접 값을 넣어야 한다.

이 시점에서 사이클 1은 끝난다. 결과물은 버리지 않는다. 맞는 구조는 유지하고, 실패한 검증 결과만 다음 요청의 재료로 넘긴다.

사이클 2 —수정 요청

전체를 다시 요청하지 않았다. 맞는 부분은 그대로 두고, 틀린 부분만 짚었다.

그 스키마에서 세 가지를 고쳐줘.

1. `id`를 DuckDB 문법에 맞게 고쳐라.
 2. `position`에 `0..=8` 범위 제약을 추가해라. 중앙은 4.
 3. `updated_at`은 앱이 매 쓰기마다 채운다 —그것을 주석에 넣어라.
- `parent_id` 구조는 그대로 뒀.

여기서 중요한 문장은 마지막 줄이다. “`parent_id` 구조는 그대로 뒀”라고 고정하지 않으면, AI는 이미 통과한 구조까지 다시 손댈 수 있다. 그러면 사이클 1에서 끝낸 검증을 다시 해야 한다.

사이클 2 —구현

수정본은 다음 방향으로 돌아왔다.

```
CREATE SEQUENCE cell_id_seq;
CREATE TABLE cells (
  id INTEGER PRIMARY KEY DEFAULT nextval('cell_id_seq'),
  parent_id INTEGER REFERENCES cells(id),
  position INTEGER NOT NULL CHECK (position BETWEEN 0 AND 8),
  title TEXT,
  description TEXT,
  done BOOLEAN DEFAULT false,
  pomodoro_minutes INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT now(),
  -- updated_at은 앱 코드가 INSERT/UPDATE 시점마다 명시적으로 채운다.
  updated_at TIMESTAMP NOT NULL
);
CREATE INDEX idx_cells_parent ON cells(parent_id);
```

수정본은 세 가지 지적을 모두 반영했다. id는 DuckDB에서 쓸 수 있는 시퀀스 방식으로 바뀌었고, position에는 CHECK 제약이 들어갔고, updated_at은 자동 갱신이 아니라 앱 책임이라는 점이 주석으로 드러났다. DuckDB의 SEQUENCE·CHECK 같은 구체적 문법은 7장에서 따로 다룬다.

사이클 2—검증

마지막으로 처음 문제 세 가지를 다시 대조했다.

1. position 범위가 스키마에서 강제되는가? CHECK 제약으로 확인했다.
2. DuckDB 문법에 맞는가? SEQUENCE와 DEFAULT nextval 형태로 확인했다.
3. updated_at 책임이 분명한가? 주석과 NOT NULL로 앱에서 반드시 값을 넣어야 함을 드러냈다.

두 사이클, 약 15분. 스키마 v0.1 완료. 첫 시도에서 완벽한 결과를 기대하지 않았다. 빠르게 초안을 받고, 문제를 구체적으로 지적했다. “parent_id 구조는 그대로 뒀다”라고 손대지 말 부분도 명확히 짚었다. 도메인 지식은 코드를 처음부터 혼자 쓸 때만 필요한 게 아니다. AI가 잘못 가정한 지점을 좁혀줄 때도 필요하다.

이 사례에서 중요한 것은 SQL 문법 자체가 아니다. AI가 만든 초안을 그대로 믿지 않고, 검증 결과를 다음 요청으로 넘기는 방식이다. 맞는 구조는 고정하고, 틀린 부분만 다시 요청하면 루프는 짧아진다. 반대로 매번 전체를 다시 만들게 하면 검증했던 부분까지 흔들린다. AI와 함께 개발할 때 속도를 지키는 방법은 더 많이 말기는 것이 아니라, 무엇을 유지하고 무엇을 바꿀지 좁혀서 요청하는 것이다.

이 패턴을 모든 작업에 쓸 수 있도록 정리한 것이 다음 템플릿이다.

2.9 기본 프롬프트 템플릿: 무엇을 / 왜 / 제약사항 / 형식

매일 쓰는 네 덩어리 템플릿이다. 빈 칸을 채우는 식으로 쓰면 빠진 항목이 없는지 점검할 수 있다.

[무엇을]

한 줄: 무엇을 만들지.

[왜]

왜 중요한지. 포함하는 기능과 사용자 흐름.

[제약사항]

- 언어 / 버전
- 금지사항
- 성능 / 보안 조건
- 기존 코드와의 인터페이스

[형식]

- 결과물 형태 (코드만 / 설명과 함께 / diff / 파일 트리)
- 파일과 함수 명명 규칙

Tauri 명령 예시로 채워보면 이렇다. 이 요청이 만드는 것은 **PDF 업로드를 받아 로컬에 저장하는 Rust 명령 하나**다.

[무엇을]

PDF 업로드를 받아서 로컬에 저장하는 Tauri 명령.

[왜]

readbooks.ai 번역 파이프라인의 1단계. 2-4단계 (파싱, 번역, 저장)가 여기에 붙는다. 사용자가 PDF를 드래그앤드랍한다.

[제약사항]

- Rust, tauri 2.x, async
- app_data_dir/uploads/<uuid>.pdf에 저장
- 50MB보다 큰 파일은 거부
- thiserror로 커스텀 에러 타입 정의

[형식]

- Rust 코드 블록 하나, 30줄 미만
- 설명은 코드 위에 2-3줄로 제한

이 구조대로 쓰면 요청이 짧아져도 결과 품질은 그대로다. 손에 익으면 네 항목을 문단 하나로 압축해도 된다. 단, AI와 새 작업 세션을 시작할 때는 네 항목을 나눠 쓰는 게 낫다.

2.10 루프 돌릴 때 주의할 점

- AI의 “베스트 프랙티스” 주장은 한 번 더 의심한다. 학습 시점 이후 API가 바뀐 경우가 많다. 현재 공식 문서로 교차 확인한다.
- 같은 수정 지시로 두 번 같은 결과가 나오면, 지시 자체가 잘못된 것이다. 표현을 바꿔 다시 요청한다.
- 중요한 프롬프트는 저장해둔다. 애플마다 `prompts.md`에 적어두면 두 달 뒤 비슷한 작업에 그대로 재할용할 수 있다.

마무리 박스: 이 장에서 반드시 기억할 것

- 이 책의 모든 작업은 **요청 -> 구현 -> 검증** 루프를 따른다. 실패한 검증 결과는 다음 요청으로 돌아가며, 작업 규모에 따라 루프의 단위만 바뀐다.
- **빠대 우선 전략**을 쓴다: 코드를 짜기 전에 아키텍처를 확정하고, 모듈을 하나씩 구현하고, 전체가 들어간 뒤 버그를 잡고, 기능을 끝까지 검증한 다음 다듬는다. 단계를 오가면 재작업이 빠르게 늘어난다.
- “완벽한 프롬프트 하나”는 없다. **짧은 요청을 여러 번** 하는 게 품질도 좋고 속도도 빠르다.
- 검증 없이 구현하면 반복이 쌓일수록 기술 부채도 쌓인다.
- 모든 요청에는 **무엇을 / 왜 / 제약사항 / 형식**을 넣는다. 익숙해지면 문단 하나로 압축해도 된다.

다음 장부터는 이 루프를 실제 코드 작업에 적용해본다. 요청하고, 구현 결과를 받고, 검증한 뒤, 실패한 증거를 다음 요청으로 되돌리는 모습이 매번 반복되는 것을 보게 될 것이다.

3장. 프롬프트 전용 개발 루프—도구보다 중요한 운용법

2장에서는 Request -> Build -> Verify 루프를 정의했다. 이 장은 그 루프를 AI 코딩 도구를 사용할 때 실제로 어떻게 돌리는지 다룬다. Copilot, Cursor, Claude Code, Codex처럼 도구 이름은 계속 바뀐다. 하지만 AI에게 일을 맡기는 방식은 크게 바뀌지 않는다.

처음에는 GitHub Copilot을 자동완성 도구로만 썼다. 코드를 몇 글자 입력하면 이어지는 코드를 제안하고, Tab 키로 수락하는 방식이었다. 이 방식은 반복 코드에는 좋다. 하지만 Tauri 앱처럼 프론트엔드, Tauri 명령, 데이터베이스, 권한 파일이 함께 맞아야 하는 작업에는 부족했다. 자동완성은 “다음 줄”을 잘 맞히지만, “지금 이 기능을 어떤 순서로 만들어야 하는가”는 정해주지 못한다.

그래서 도구를 쓰는 순서를 바꿨다. 코드를 받기 전에 먼저 세 가지를 정한다.

1. 지금 AI에게 맡길 작업 단위는 무엇인가.
2. AI가 절대 바꾸면 안 되는 제약은 무엇인가.
3. 결과를 받은 뒤 무엇으로 검증할 것인가.

이 세 가지가 정리되어 있으면 어떤 도구를 써도 루프가 흔들리지 않는다. 이 장은 특정 도구 사용법이 아니라, **AI에게 맡기는 작업을 프롬프트로 통제하는 방법**을 다룬다.

이 장의 흐름

- 3.1 이 장의 범위
- 3.2 프롬프트 전용 루프의 5단계
- 3.3 프롬프트 패턴 5가지
- 3.4 짧은 실전 세션—노트 저장 기능
- 3.5 4장으로 넘어가기 전에

학습 목표

- 도구가 달라져도 유지되는 프롬프트 운용 원칙을 익힌다.
- 코드 생성 전에 작업 범위와 검증 기준을 먼저 정하는 습관을 만든다.
- AI가 이미 검증된 코드를 흔들지 않도록 보존 지시를 쓰는 방법을 배운다.
- 4장의 도구 설정과 5장의 Tauri 기초로 넘어가기 전에, 프롬프트 루프의 기본 자세를 잡는다.

3.1 이 장의 범위

이 장은 Copilot 사용법이나 Cursor 단축키를 설명하지 않는다. 도구별 설치, 선택 기준, 규칙 파일은 4장에서 다룬다. 여기서는 도구가 무엇이든 그대로 적용되는 프롬프트 루프만 본다.

역할을 분명히 나누면 이렇다.

장	중심 질문	결과물
2장	AI와 개발할 때 어떤 루프를 돌리는가	Request -> Build -> Verify 모델
3장	그 루프를 프롬프트로 어떻게 운영 하는가	작업 단위, 제약, 검증 프롬프트
4장	어떤 도구와 설정으로 반복할 것인가	Claude Code, Cursor, 규칙 파일
5장	Tauri 2에서 무엇을 조심해야 하는가	명령, 상태, 이벤트, 권한 경계

이 장의 예시는 가볍게 읽으면 된다. 아직 Tauri 2 내부 구조를 전부 알 필요는 없다. 중요한 것은 코드 세부가 아니라, AI에게 요청을 나누고 결과를 검증하는 방식이다.

3.2 프롬프트 전용 루프의 5단계

프롬프트 전용 루프는 코드를 전혀 직접 쓰지 않는다는 뜻이 아니다. 더 정확히는 **코드를 치는 시간보다 요청 작성과 검증에 시간을 더 쓰는 방식**이다. AI가 코드를 쓰더라도 방향을 정하고 결과를 승인하는 사람은 개발자다.

흐름은 다섯 단계다.

작업 단위 정하기 → 프롬프트 작성 → 결과 받기 → 검증 → 수정 요청

1단계: 작업 단위 정하기

가장 먼저 할 일은 “무엇을 만들까”가 아니라 **어디까지 말길까**를 정하는 것이다. 작업 단위가 너무 크면 AI가 빈칸을 스스로 채운다. 반대로 너무 작으면 왜 이 코드가 필요한지 설명할 맥락이 사라진다.

좋은 작업 단위는 한 문장으로 설명된다.

- 좋음: “노트를 로컬 텍스트 파일로 저장하는 Tauri 명령 하나”
- 좋음: “저장 버튼을 누르면 save_note 명령을 호출하는 Svelte 컴포넌트”
- 나쁨: “노트 앱 전체”
- 나쁨: “저장, 검색, AI 요약, 설정 화면까지 한 번에”

프롬프트를 쓰기 전에 이 문장을 먼저 적는다. 한 문장으로 설명이 안 되면 아직 작업 단위가 큰 것이다.

2단계: 프롬프트 작성

프롬프트는 대화가 아니라 작업 지시서다. 반드시 네 가지를 포함한다.

[맥락] 지금 프로젝트의 상태
 [목표] 이번 요청으로 만들 것
 [제약] 지켜야 할 규칙과 금지할 것
 [검증] 결과가 맞는지 확인할 방법

2장에서 본 “무엇을 / 왜 / 제약사항 / 형식”과 같은 원리다. 3장에서는 여기에 **검증 방법**까지 붙인다. 검증 방법을 쓰지 않은 프롬프트는 아직 완성된 요청이 아니다.

3단계: 결과 받기

AI의 첫 답변은 완성품이 아니라 초안이다. 방향이 맞는지 먼저 본다. 파일이 엉뚱한 곳에 생겼는지, 이미 정한 제약을 어겼는지, 출력 형식을 지켰는지 확인한다.

이 단계에서 모든 줄을 읽을 필요는 없다. 먼저 큰 구조를 본다.

- 요청한 파일만 바뀌었는가.
- 금지한 방식을 쓰지 않았는가.
- 검증할 수 있는 형태로 결과가 나왔는가.
- 설명만 있고 실행 가능한 코드가 빠지지 않았는가.

구조가 틀렸으면 바로 거절하고 다시 요청한다. 구조가 맞고 세부가 틀렸으면 다음 단계로 간다.

4단계: 검증

검증은 감상이 아니다. 실제 명령을 실행하고, 예상한 결과와 비교하는 일이다.

작은 작업에서는 다음 세 가지면 충분하다.

```
cargo check --manifest-path src-tauri/Cargo.toml
pnpm check
pnpm tauri dev
```

그리고 기능을 직접 눌러본다. 예를 들어 “저장 버튼”을 만들었다면 버튼을 클릭했을 때 실제 파일이 생겨야 한다. 콘솔에 성공 로그만 찍히는 것은 검증이 아니다. **기대했던 외부 결과가 생겼는지** 확인해야 한다.

5단계: 수정 요청

검증이 실패하면 전체를 다시 만들게 하지 않는다. 실패한 증거만 다음 요청에 넣는다.

방금 코드에서 저장 버튼을 눌렀지만 파일이 생기지 않는다.

확인한 내용:

- `cargo check --manifest-path src-tauri/Cargo.toml` 통과
- `pnpm check` 통과
- 버튼 클릭 시 콘솔에는 "saved" 로그가 나옴
- 앱 데이터 디렉터리 아래 notes 폴더에는 파일이 없음

요청:

파일 생성 경로와 실제 `write` 호출만 점검해줘.

UI 구조와 버튼 컴포넌트는 바꾸지 마.

마지막 줄이 중요하다. 이미 검증한 부분을 보호해야 한다. AI에게 “고쳐줘”라고만 하면 작동하던 UI까지 다시 쓸 수 있다.

3.3 프롬프트 패턴 5가지

매번 새 프롬프트를 처음부터 만들 필요는 없다. 실전에서 자주 쓰는 패턴은 다섯 가지다.

패턴 1: 뼈대 먼저

구현 전에 구조를 먼저 받는다.

코드는 쓰지 말고 구조만 제안해줘.

노트 저장 기능에 필요한 파일과 `Tauri` 명령 이름을 정리해줘.

프론트엔드 호출 위치도 목록에 포함해줘.

각 항목은 한 줄 설명만 붙여줘.

이 패턴은 처음 만드는 기능에 쓴다. 구조 단계에서 틀린 파일 하나를 지우는 건 쉽다. 구현 뒤에 잘못된 구조를 고치는 건 비싸다.

패턴 2: 범위 고정

수정할 때는 범위를 좁힌다.

아래 오류만 수정해줘.

다른 파일은 건드리지 마.

함수 이름과 반환 타입은 유지해.

AI는 문제를 해결하려고 주변 코드까지 고칠 때가 많다. 개발자에게는 그 수정이 비용이 된다. 방금 검증한 부분까지 다시 확인해야 하기 때문이다.

패턴 3: 부정 제약

AI가 자주 틀리는 방향은 미리 막는다.

금지:

- 프론트엔드에서 직접 파일 시스템 접근 금지
- `unwrap()` 사용 금지
- Tauri 1 API 사용 금지
- 기존 컴포넌트 이름 변경 금지

“좋게 만들어줘”보다 “이 네 가지는 하지 마”가 더 잘 작동한다. 특히 Tauri 2처럼 버전 차이가 큰 프레임 워크에서는 부정 제약이 효과적이다.

패턴 4: 파일 참조

프로젝트 관례를 설명하기보다 기존 파일을 보여준다.

`src/lib/api/notes.ts`의 호출 패턴을 따라가.
`src-tauri/src/commands/settings.rs`의 예러 처리 방식을 참고해.
 새 명령은 `save_note`로 만들고, 같은 스타일로 `Result`를 반환해.

도구마다 파일 참조 문법은 다르다. Cursor는 `@파일명`, Copilot Chat은 `#file:` 경로, Claude Code는 파일명을 직접 적거나 함께 열어둔 파일을 읽게 할 수 있다. 문법은 달라도 목적은 같다. **말로 설명하지 말고, 이미 맞는 예를 보여주는 것이다.**

패턴 5: 검증 기준 선제 작성

프롬프트 끝에는 항상 성공 기준을 붙인다.

성공 기준:

1. ``(cd src-tauri && cargo check)`` 통과
2. 저장 버튼 클릭 시 앱 데이터 디렉터리 아래 ``notes/note_*.txt`` 생성
3. 빈 내용 저장 시 예러 메시지 표시
4. API 키나 파일 경로가 프론트엔드에 하드코딩되지 않음

성공 기준은 AI에게만 주는 것이 아니다. 나중에 내가 결과를 통과시킬지 말지 판단하는 기준이다. 프롬프트 전에 기준을 쓰면, AI의 답변에 맞춰 기준을 바꾸지 않게 된다.

3.4 짧은 실전 세션—노트 저장 기능

무거운 Tauri 내부 코드는 5장에서 다룬다. 여기서는 프롬프트 루프가 어떻게 돌아가는지만 본다. 예시는 노트 저장 기능이다.

라운드 1 —빠대만 받기

첫 요청은 구현이 아니다.

[맥락]

Tauri 2 + SvelteKit 2 + Svelte 5로 간단한 노트 앱을 만들고 있다.
현재는 빈 화면과 텍스트 입력창만 있다.

[목표]

노트를 로컬 파일로 저장하는 기능의 구조를 잡고 싶다.

[제약]

코드는 쓰지 마.
파일 목록, Tauri 명령 이름, 프론트엔드 호출 위치만 제안해.

[검증]

내가 보고 "명령 하나 + 버튼 하나 + 저장 경로 하나"로 설명할 수 있어야 한다.

좋은 응답은 보통 이렇게 생긴다.

위치	역할
src-tauri/src/commands/notes.rs	save_note 명령
src/lib/api/notes.ts	프론트엔드 호출 래퍼
src/routes/+page.svelte	입력창과 저장 버튼
앱 데이터 디렉터리	실제 노트 파일 저장 위치

이 정도면 충분하다. 아직 코드는 없다. 하지만 어디를 만들지 정해졌다.

라운드 2 —명령 하나만 구현

이제 Rust 명령만 요청한다.

[목표]

save_note Tauri 명령 하나를 작성해줘.

[제약]

- 입력: title: String, body: String
- 빈 body는 에러
- 저장 경로는 앱 데이터 디렉터리 아래 notes/

- `unwrap()`, `expect()` 금지
- 프론트엔드 코드는 아직 쓰지 마

[검증]

``(cd src-tauri && cargo check)``가 통과해야 한다.
body가 비어 있으면 에러를 반환해야 한다.

여기서 AI가 프론트엔드까지 함께 만들면 실패다. 요청 범위를 벗어났기 때문이다. 그때는“프론트엔드 코드는 제외하고 Rust 명령만 다시 보여줘”라고 돌려보낸다.

라운드 3 —프론트엔드 연결

Rust 명령이 통과한 뒤에 버튼을 붙인다.

[목표]

기존 입력창 옆에 저장 버튼을 추가하고 `save_note`를 호출해줘.

[제약]

- `src/lib/api/notes.ts`에 `invoke` 래퍼를 뒤
- `+page.svelte`에서는 래퍼만 호출해
- 저장 성공/실패 메시지를 화면에 표시해
- Rust 명령 코드는 바꾸지 마

[검증]

버튼 클릭 시 실제 파일이 생성되어야 한다.
빈 내용이면 화면에 에러 메시지가 보여야 한다.

중요한 문장은“Rust 명령 코드는 바꾸지 마”다. 2라운드에서 이미 검증한 부분을 보호한다.

라운드 4 —실패 증거로 수정

실행해보니 파일이 생기지 않았다고 하자. 이때 다시 전체를 만들게 하지 않는다.

검증 실패:

- ``cargo check --manifest-path src-tauri/Cargo.toml`` 통과
- ``pnpm check`` 통과
- 버튼 클릭 시 성공 메시지는 보임
- 하지만 앱 데이터 디렉터리 아래 `notes` 폴더에 파일이 없음

추정하지 말고 저장 경로와 `write` 호출만 확인해줘.

프론트엔드 UI와 Rust 함수 시그니처는 바꾸지 마.

이 프롬프트는 짧지만 충분하다. 실패 증거, 수정 범위, 보존할 부분이 모두 들어 있다.

세션에서 배울 것

이 예제의 핵심은 노트 앱이 아니다. **한 번에 전체 기능을 만들지 않았다는 점**이다. 구조를 먼저 받고, 명령을 만들고, 프론트엔드를 붙이고, 실패 증거로 수정했다. 각 라운드마다 검증 기준이 있었다. 이것이 도구가 바뀌어도 유지되는 프롬프트 전용 개발 루프다.

3.5 4장으로 넘어가기 전에

3장에서 만든 것은 도구 사용법이 아니라 작업 방식이다. 이 방식이 있어야 4장의 도구 설정이 의미를 가진다. CLAUDE.md, Cursor Project Rules(.cursor/rules/*.mdc), 슬래시 명령은 모두 같은 목적을 가진다. 매번 프롬프트에 쓰던 제약을 파일에 기록해, AI가 같은 실수를 반복할 가능성을 줄이는 것이다.

4장에서는 이 장의 패턴을 실제 환경에 옮긴다.

- 어떤 작업은 Claude Code에 맡길지
- 어떤 작업은 Cursor에서 처리할지
- 프로젝트 규칙을 어디에 저장할지
- 첫 Tauri 앱 골격을 어떻게 만들지

도구를 먼저 고르면 도구에 끌려간다. 루프를 먼저 정하면 도구는 그 루프를 빠르게 돌리는 수단이 된다.

마무리 박스: 이 장에서 반드시 기억할 것

- 3장의 핵심은 Copilot이나 Cursor 사용법이 아니라 **프롬프트로 작업 범위를 통제하는 방법**이다.
- 프롬프트 전용 루프는 **작업 단위 정하기 -> 프롬프트 작성 -> 결과 받기 -> 검증 -> 수정 요청**으로 돈다.
- 좋은 프롬프트에는 맥락, 목표, 제약, 검증 기준이 함께 들어간다.
- AI가 이미 검증한 부분을 다시 흔들지 않도록 “이 파일은 바꾸지 마”, “이 함수 시그니처는 유지해” 같은 보존 지시를 쓴다.
- 실패하면 전체를 다시 요청하지 말고, 실패 증거와 수정 범위를 좁혀서 다시 보낸다.

4장에서는 이 루프를 실제 도구 설정으로 옮긴다. 5장에서는 그 도구들이 만들어낼 Tauri 2 코드의 기본 경계를 배운다.

4장. 도구와 환경—Claude Code, Cursor, Codex, GitHub Copilot

3장에서 프롬프트 루프와 5가지 패턴을 다뤘다. 이 장은 그 패턴들을 실제 도구에 연결한다. 어떤 도구를 쓰느냐보다 그 도구로 어떻게 루프를 유지하느냐가 핵심이다.

도구 선택은 비교표만으로 결론 내리기 어렵다. “Claude Code와 Cursor 중 무엇이 더 좋은가”같은 질문은 실제 개발을 나아지게 하지 않는다. 더 좋은 질문은 따로 있다. 지금 작업의 규모와 위험도를 보고, 어떤 도구가 막히지 않고 결과까지 데려다주는가.

같은 AI라도 함수 하나를 고칠 때와 저장소 전체 구조를 바꿀 때 필요한 환경이 다르다. 여기서 **피드백 루프**란 AI에게 요청하고, 결과를 확인하고, 다시 수정 요청하는 반복 주기를 말한다. 루프가 짧을수록 실수를 빨리 잡을 수 있다. 좋은 도구는 “가장 똑똑한 도구”가 아니라 지금 작업의 피드백 루프를 가장 짧게 만들어 주는 도구다.

이 장을 다 따라 하면 손에 남는 것: 동작하는 Tauri 앱 골격, 프로젝트 규칙 파일(CLAUDE.md 또는 .cursor/rules/*.mdc), 그리고 두 개의 git 커밋. 즉, 5장부터 본격적으로 AI와 함께 개발할 때 바로 쓸 수 있는 환경이다.

AI 코딩 도구는 실행 위치에 따라 크게 세 종류로 나뉜다. **터미널(CLI)**에서 돌리는 도구, **IDE 안에서** 돌리는 도구, 그리고 **클라우드**에서 돌아가는 도구다. 터미널 도구는 명령어 한 줄로 저장소 전체를 다루기 좋고, IDE 도구는 코드 옆에서 짧은 편집을 빠르게 한다. 클라우드 도구는 내 컴퓨터가 아닌 외부 환경에서 작업하므로 로컬을 건드리지 않고 위임할 때 쓴다.

이 장의 흐름

- 네 가지 도구, 네 가지 접근
- 언제 무엇을 쓰는가
- Tauri 2 + SvelteKit + Rust 최소 환경
- 처음 30분—스캐폴드에서 첫 커밋까지
- 규칙 파일(CLAUDE.md, .cursor/rules/*.mdc)
- AI에 줄 제약—스택과 버전 명시
- 더 알아보기: MCP, Agent, 슬래시 명령

학습 목표

- 네 도구(Claude Code, Cursor, Codex, GitHub Copilot)의 차이를 안다.
- 로컬에 Tauri 2 + SvelteKit + Rust가 동작하는 최소 환경을 만든다.
- CLAUDE.md / .cursor/rules/*.mdc 규칙 파일을 작성해 AI 출력 품질을 높인다.

상호 참조: 이 장은 3장의 프롬프트 루프와 5가지 패턴을 이어받아, 5장 Tauri 2 기초로 이어진다.

4.1 네 가지 도구, 네 가지 접근

2026년 기준, 실무에서 자주 비교되는 AI 코딩 도구는 네 가지다. 이름이 비슷해 보여도 차이는 모델보다 **실행 위치**와 **작업 방식**에서 먼저 드러난다.

도구	실행 환경	모델	주요 강점
Claude Code	터미널(로컬)	Claude (Anthropic)	긴 컨텍스트, 저장소 전체 탐색
Cursor	IDE(VS Code 포크)	선택 가능	빠른 인라인 편집과 자동완성
OpenAI Codex	데스크톱 앱·터미널·IDE·클라우드	OpenAI	로컬 저장소 작업과 격리된 클라우드 위임
GitHub Copilot	IDE + 클라우드 + CLI	선택 가능	GitHub 이슈/PR 통합, 인라인 자동완성

도구마다 잘 맞는 자리가 다르다. 하나씩 살펴보자.

Claude Code는 터미널에서 실행되고 프로젝트 폴더 전체를 컨텍스트로 쓸 수 있다. 긴 대화와 여러 파일을 동시에 다루는 작업에 강하다.

Cursor는 VS Code를 기반으로 만든 IDE다. 에이전트 작업과 자동완성을 한 화면에서 함께 쓸 수 있고, 짧은 편집에서 빠르다.

OpenAI Codex는 현재 하나의 실행 형태만 가리지 않는다. 데스크톱 앱, 로컬 저장소를 직접 다루는 CLI와 IDE 확장, 격리된 환경에서 병렬로 실행하는 Codex cloud가 있다. 로컬 도구는 내 컴퓨터의 파일과 개발 도구를 사용하고, cloud는 별도로 구성한 환경에서 작업한 뒤 변경 내용을 검토하게 한다.

공식 문서 확인: Codex CLI는 로컬 저장소의 파일을 수정하고 설치된 도구를 실행하며, Codex cloud는 격리된 클라우드 환경에서 병렬 작업을 수행한다. 제품 형태가 바뀔 수 있으므로 출간 후에는 **Codex CLI**와 **Codex cloud**를 기준으로 확인한다.

GitHub Copilot은 원래 자동완성 도구였다가 에이전트 플랫폼으로 확장됐다. IDE 안의 Agent 모드, GitHub 이슈를 자동으로 처리하는 Cloud Agent, 터미널용 Copilot CLI, 그리고 기존 인라인 자동완성까지 네 가지 사용 방식이 있다. 처음에는 이름과 역할만 기억해두면 충분하다.

이 책에서는 Claude Code + Cursor를 주로 쓴다. 이유는 단순하다. Claude Code는 로컬 저장소 전체를 긴 맥락으로 다루기 좋고, Cursor는 한 파일의 짧은 편집을 빠르게 처리한다. 큰 루프는 Claude Code로, 작은 편집은 Cursor로 나누면 피드백 루프가 짧아진다. GitHub Copilot과 Codex는 특정 상황에서 대안으로 언급한다.

주의: 같은 작업에 여러 에이전트를 동시에 쓰면 충돌이 나기 쉽다. 같은 파일을 두 에이전트가 동시에 건드릴 때 특히 그렇다. 작업마다 도구 하나를 정하고, 전환은 깔끔하게 하는 게 좋다.

4.2 언제 무엇을 쓰는가

도구 선택은 취향이 아니라 **작업 단위**의 문제다. “이 작업을 한 번에 얼마나 넓게 봐야 하는가”로 나눈다.

Claude Code가 맞는 상황

여러 파일에 걸친 새 기능 추가, 대규모 리팩토링, 프로젝트 구조를 처음 잡는 작업. 디버깅에서 로그·스택 트레이스·소스를 함께 봐야 할 때도 마찬가지다. 전체 파일 구조를 보기 때문에 원인을 추적하기 쉽다. 단, 범위가 넓은 만큼 **검증 기준**을 명확히 줘야 한다. 그렇지 않으면 많은 파일을 바꾸고도 요구사항과 다른 방향으로 흘러갈 수 있다.

Cursor가 맞는 상황

한 파일 안의 함수 하나, 특정 코드 조각 설명, 한 줄 자동완성처럼 **수정 위치가 이미 분명한** 작업. 반대로 변경 범위가 여러 디렉터리로 퍼지면 커서 중심 맥락이 오히려 좁게 느껴진다.

OpenAI Codex가 맞는 상황

CLI나 데스크톱 앱으로 로컬 저장소를 탐색·수정·테스트하고 싶을 때, 또는 Codex cloud에 오래 걸리는 작업을 격리해 위임하고 싶을 때다. 같은 이름 아래 로컬 실행과 클라우드 위임이 모두 있으므로, 프롬프트를 보내기 전에 작업 위치와 권한 범위를 먼저 확인한다.

GitHub Copilot이 맞는 상황

반복적인 보일러플레이트를 입력할 때 인라인 자동완성이 가장 빠르다. GitHub 이슈를 자동으로 PR로 만들고 싶을 때는 Cloud Agent를 쓴다.

내 경우 하루 작업의 절반 이상은 Claude Code, 그다음으로 Cursor가 많고, 나머지는 상황에 따라 보조적으로 쓴다. 새로 처음부터 만드는 앱일수록 Claude Code 비중이 올라가고, 기존 화면을 다듬는 작업이 많을수록 Cursor 비중이 올라간다.

4.3 최소 Tauri 2 + SvelteKit + Rust 환경

이 절은 Tauri 2 개발 환경을 처음 세팅할 때 따라 치는 명령 모음이다. 각 명령이 왜 필요한지 함께 설명한다.

버전 고정 전략

설치 전에 버전을 정확히 고정해야 한다. **AI 개발에서는 특히 중요하다.** AI는 다양한 버전의 코드를 학습했기 때문에, 버전을 명시하지 않으면 Tauri 1.x 코드(현재 동작 안 함), Svelte 4 스토어, 옛 SvelteKit 라우팅 같은 오래된 코드를 생성할 수 있다. 그래서 이 책은 다음 버전을 기준으로 한다.

- Tauri CLI: 2.11.x
- SvelteKit: 2.70.x
- Svelte: 5.56.x (rune 문법)
- Node.js: 24.x LTS
- Rust: stable 채널, 1.95.0

이 숫자는 2026년 8월에 이 책의 예제를 검증한 기준선이다. 새 프로젝트에서는 잠금 파일(pnpm-lock.yaml, Cargo.lock)을 함께 커밋하고, 패치 버전이 올라가더라도 한 번에 하나씩 업데이트한다. SvelteKit 2와 Svelte 5는 서로 다른 패키지이므로 둘을 같은 major version처럼 부르면 안 된다.

필수 설치 단계

각 단계는 한 번씩만 실행한다. 명령이 멈추지 않고 끝나면 성공이다.

1. Rust 툴체인

```
curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source ~/.cargo/env
rustup default stable
```

설치 후 `rustc --version`이 책의 검증 기준인 1.95.0 이상이면 정상이다. 이것은 Tauri의 영구적인 최소 버전 선언이 아니라 이 책의 재현 기준이다.

2. Node.js (버전 관리용 fnm)

```
curl -fsSL https://fnm.vercel.app/install | bash
source ~/.zshrc
fnm install 24
fnm use 24
fnm default 24
```

3. 패키지 매니저 (pnpm 권장)

```
npm install -g pnpm@latest
```

4. Tauri CLI

이 책은 프로젝트에 설치되는 JavaScript CLI 하나를 사용한다. 전역 `cargo tauri`와 로컬 `pnpm tauri`를 동시에 설치하면 어느 버전이 실행되는지 헷갈릴 수 있다.

```
# 프로젝트를 만든 뒤 그 안에서 실행한다
pnpm add -D @tauri-apps/cli@2.11.4
```

5. 플랫폼별 요구사항

플랫폼	설치 명령
macOS	<code>xcode-select --install</code>
Linux (Ubuntu/Debian)	아래 명령으로 WebKitGTK와 빌드 도구 설치
Windows	Microsoft C++ Build Tools, WebView2 (Win 11엔 보통 미리 설치)

Ubuntu/Debian에서는 다음 명령을 실행한다.

```
sudo apt install libwebkit2gtk-4.1-dev build-essential curl wget \
  file libxdo-dev libssl-dev libayatana-appindicator3-dev \
  librsvg2-dev patchelf
```

AI 도구 설정

```
# Claude Code CLI
npm install -g @anthropic-ai/claude-code
claude --version

# Codex CLI
npm install -g @openai/codex
codex --version
```

Cursor IDE는 cursor.com에서 다운로드한다. 기존 VS Code 설정을 자동으로 가져온다.

두 도구 모두 계정 로그인 방식을 사용할 수 있다. 별도로 API 과금 방식을 선택했을 때만 **API 키 환경 변수**를 셸 설정 파일에 추가한다.

```
export ANTHROPIC_API_KEY="your-key-here"
export OPENAI_API_KEY="your-key-here"
```

주의: API 키는 코드에 직접 넣거나 git에 커밋하면 안 된다. 환경 변수나 `.gitignore`에 등록된 `.env.local` 파일을 사용한다. AI 도구가 키를 코드에 그대로 넣으라고 제안할 때는 항상 거부한다.

프로젝트 초기화

`create-tauri-app`의 버전과 Tauri CLI의 버전은 서로 다르다. 특히 `create-tauri-app@2.11.0`은 존재하지 않으며, 현재 스캐폴더에는 Svelte 템플릿은 있어도 이 책이 쓰는 SvelteKit 템플릿은 없다. 따라서 SvelteKit 프로젝트를 먼저 만들고 Tauri를 붙인다.

```
pnpm dlx sv create my-app
cd my-app
pnpm install
pnpm add @tauri-apps/api
pnpm add -D @tauri-apps/cli@2.11.4 @sveltejs/adapter-static
pnpm tauri init
```

`sv create` 질문에서는 **SvelteKit minimal + TypeScript**를 고른다. `tauri init`에서는 개발 서버를 `http://localhost:5173`, 프론트엔드 개발·빌드 명령을 각각 `pnpm dev`, `pnpm build`, 빌드 결과 경로를 `../build`로 입력한다.

Tauri의 WebView에는 SvelteKit 서버를 함께 넣을 수 없으므로 정적 어댑터와 SPA 모드를 사용한다. `svelte.config.js`의 `kit.adapter`를 다음처럼 설정한다.

```
import adapter from '@sveltejs/adapter-static';
import { vitePreprocess } from '@sveltejs/vite-plugin-svelte';

const config = {
```

```
preprocess: vitePreprocess(),
kit: {
  adapter: adapter({ fallback: 'index.html' })
}
};

export default config;
```

루트 레이아웃에서 서버 사이드 렌더링을 끈다.

```
// src/routes/+layout.ts
export const ssr = false;
```

마지막으로 `src-tauri/tauri.conf.json`에서 `build.frontendDist`가 `../build`인지 확인하고 실행한다.

```
pnpm tauri dev
```

확인할 출력: SvelteKit 첫 화면이 Tauri 창 안에 열리고 콘솔에 오류가 없으면 정상이다.

설치가 실패할 때—가장 흔한 3가지

창이 안 뜨거나 빌드가 실패하면, 입문자 단계에서 거의 다음 세 가지 중 하나다.

- **Rust 경로 미설정:** `rustc --version`이 동작하지 않으면 `source ~/.cargo/env`를 다시 실행하거나 새 터미널을 연다.
- **WebView2 누락 (Windows 10):** Tauri는 시스템 웹뷰를 쓴다. Win 11엔 보통 깔려 있지만 Win 10에는 따로 설치해야 한다. Microsoft 공식 사이트에서 “Evergreen Bootstrapper”를 받는다.
- **Linux 라이브러리 누락:** `libwebkit2gtk-4.1-dev` 같은 패키지가 빠지면 `pnpm tauri dev` 실행 중 컴파일 에러가 난다. 위의 `apt install` 명령을 다시 확인한다.

참고: 데스크톱 앱은 운영체제별 번들러·서명 도구가 달라 일반적으로 각 대상 OS의 CI 러너에서 빌드한다. 배포 자동화는 14장에서 따로 다룬다. 지금은 **내 컴퓨터에서 일단 동작**하는 것까지만 확인하면 된다.

4.4 처음 30분—스캐폴드에서 첫 커밋까지

앞 절의 `sv create`와 `tauri init`을 마치면 프로젝트 기본 구조가 생긴다. 이 구조를 **AI 개발에 바로 쓸 수 있는 환경**으로 만드는 체크리스트다.

즉시 확인 (5분)

1. 개발 서버 테스트

```
cd your-app-name
pnpm install
pnpm tauri dev
```

앱이 열리고 “Welcome to Tauri!”가 보여야 한다. 브라우저 콘솔에 에러가 없어야 한다.

2. 빌드 테스트

```
pnpm tauri build
```

처음 실행하면 Rust 컴파일에 2~3분 걸린다. 에러 없이 완료되면 `src-tauri/target/release/bundle/`에 실행 파일이 만들어진다.

3. VS Code 통합 확인

Cursor나 VS Code에서 프로젝트를 연다. 다음 세 가지가 보여야 한다. - `src-tauri/` 폴더에서 Rust 구문 강조 - `src/` 폴더에서 Svelte 구문 강조 - TypeScript 에러 없음

프로젝트 설정 (10분)

4. 규칙 파일 만들기

먼저 프로젝트 루트에 `CLAUDE.md`를 만든다. Claude Code는 세션을 시작할 때 작업 폴더에 있는 `CLAUDE.md`를 **자동으로 읽어 시스템 컨텍스트에 포함**한다. 다시 말해 매번 프롬프트에 같은 규칙을 붙여 쓰지 않아도, 이 파일에 적어둔 내용이 기본 약속으로 따라붙는다.

프로젝트명

스택

- Tauri 2.11.x, SvelteKit 2, Svelte 5, TypeScript, DuckDB
- Svelte 5 rune 문법만 (\$state, \$derived, \$effect)

구조

- 프론트엔드: SvelteKit (src/)
- 백엔드: Rust Tauri 명령 (src-tauri/src/commands/)
- 데이터베이스: Tauri 명령을 통해서만 DuckDB 접근

규칙

- 프론트엔드에서 직접 API 호출 없음
- 모든 외부 데이터는 Tauri 명령을 통해
- 모든 명령 반환에 Result<T, AppError> 사용
- 프로덕션 코드에 console.log 없음

금지

- Svelte 스토어나 레거시 반응형 문 없음
- Rust에서 unwrap() 없음 (적절한 예외 처리 사용)
- 하드코딩된 파일 경로 없음 (Tauri path API 사용)

Cursor를 함께 쓴다면 .cursor/rules/tauri.mdc도 만든다. 최신 Cursor의 프로젝트 규칙 위치는 .cursor/rules/이며, 각 .mdc 파일에 적용 범위와 규칙을 나눠 적을 수 있다. 예전 프로젝트의 루트 .cursorsrules도 호환 목적으로 남아 있을 수 있지만 새 프로젝트의 기본 선택으로 삼지는 않는다. 내용은 CLAUDE.md와 핵심 제약이 어긋나지 않게 맞춘다.

5. git 초기화 및 첫 커밋

```
git init
git add .
git commit -m "Initial Tauri + SvelteKit scaffold"
```

첫 생산적 변경 (15분)

6. 첫 커스텀 Tauri 명령 만들기

직접 코드를 짜기 전에, AI가 Tauri 패턴을 제대로 따르는지 먼저 확인한다. 다음 프롬프트를 AI에 보낸다.

기본 앱 메타데이터를 반환하는 "get_app_info"라는 간단한 Tauri 명령을 만들어줘.

반환 타입: `Result<AppInfo, AppError>` 여기서 `AppInfo`에는 다음 포함:

- `app_name: String`
- `version: String`
- `platform: String`

CLAUDE.md의 프로젝트 규칙을 따라.

7. 새 명령 테스트

```
pnpm tauri dev
```

프론트엔드에서 이 명령을 호출했을 때 에러가 없어야 한다.

AI 코드가 동작하지 않을 때 가장 먼저 볼 곳 1. **파일 위치**: 새 파일이 CLAUDE.md에 명시한 폴더(`src-tauri/src/commands/`)에 있는가? 2. **반환 타입**: `Result<T, AppError>` 형태로 끝나는가? `unwrap()`이 섞여 있지 않은가? 3. **바인딩**: 프론트엔드용 TypeScript 호출 코드가 `src/lib/api/`에 함께 생성되었는가? 4. **버전**: AI가 Svelte 4 스토어나 Tauri 1.x 패턴을 섞지 않았는가? 이 네 가지 중 하나만 어긋나도 빌드가 실패한다. 코드를 다시 짜기 전에 위 항목을 먼저 본다.

8. 작동하는 명령 커밋

```
git add .
git commit -m "Add get_app_info Tauri command"
```

확인 체크리스트

30분이 지나면 아래 항목을 점검한다.

- ☒ dev 서버와 build 명령이 모두 작동함
- ☒ AI 동작을 제한하는 규칙 파일이 존재함
- ☒ 커밋이 두 개 있음 (스캐폴드 + 첫 기능)
- ☒ AI가 규칙을 따른다는 것을 확인함
- ☒ TypeScript와 Rust 컴파일 에러가 없음

결과: 이후 AI 작업은 여기서 잡은 패턴을 그대로 이어간다.

주의: 검증 단계를 건너뛰지 마라. 설정 문제는 코드 500줄을 쌓기 전에 잡아야 한다. AI가 컴파일도 실행도 안 되는 코드를 계속 생성하면, 나중에 원인을 찾는 데 더 많은 시간이 든다.

4.5 Cursor Project Rules / CLAUDE.md —규칙 파일 활용

이 규칙 파일에는 **AI가 기본 지침으로 참고할 내용**을 적어둔다. 매번 프롬프트에 같은 설명을 반복하는 일을 줄여주지만 강제 장치는 아니므로 결과 검증은 계속해야 한다. 4.4에서 새 프로젝트용 기본 템플릿을 봤다. 여기서는 실제 출시한 프로젝트의 규칙 파일이 어떻게 다듬어졌는지 본다.

CLAUDE.md (Claude Code용) —실제 사례

저장소 루트에 두면 Claude Code가 세션 시작 시 자동으로 읽는다. 다음은 Mandai 프로젝트의 CLAUDE.md 발췌다.

```
# Mandai

## 스택
- Tauri 2, SvelteKit 2, Svelte 5, TypeScript, DuckDB (duckdb crate)
- AI 프로바이더: OpenAI, Anthropic, Gemini (통합 명령)

## 규칙
- Svelte 5 rune만 ($state, $derived, $effect). 레거시 스토어 문법 없음.
- Tauri 명령은 src-tauri/src/commands/*.rs 아래에 있음.
- 에러는 thiserror 기반 MandaiError로 수렴.
- 프론트엔드에서 AI 호출 금지. 항상 Tauri 명령을 통해야 한다.

## 금지
- 커밋에 console.log 없음.
- 프로덕션 코드에 `unwrap()` 없음 (테스트는 괜찮음).
```

이 파일이 있으면 “Svelte 5 rune만 써라” 또는 “unwrap 쓰지 마라” 같은 말을 프롬프트마다 반복하지 않아도 된다.

Cursor Project Rules —같은 개념, 다른 형식

개념은 CLAUDE.md와 비슷하지만 형식과 적용 방식은 다르다. Cursor는 `.cursor/rules/` 아래의 `.mdc` 파일을 프로젝트 규칙으로 사용하고, Claude Code는 CLAUDE.md를 읽는다. 두 도구를 함께 쓰는 프로젝트라면 양쪽에 필요한 규칙을 두고 핵심 제약을 일치시킨다. 과거 사례에서 `.cursorrules`가 나오면 당시 사용한 레거시 단일 파일이라고 이해하면 된다.

앱마다 추가 규칙이 필요할 수 있다. 예를 들어 실시간 데이터를 다루는 앱이라면 “WebSocket 연결은 Tauri 백엔드에서 관리”, “프론트엔드는 Tauri 이벤트로 데이터 수신”같은 앱 고유 규칙을 덧붙인다. 공통 규칙은 새 프로젝트 템플릿에서 그대로 가져오고, 앱 고유 패턴만 더하면 된다.

규칙 파일의 효과

- 프롬프트가 짧아진다. “이 저장소에서는…”같은 전제 설명이 사라진다.
- 루프 반복 횟수가 줄고 AI 오류도 줄어든다.
- 저장소를 팀원에게 넘기면 **AI 규칙이 곧 팀 규칙**이 된다.

규칙 파일은 의존성을 올리거나 구조를 바꿀 때 함께 수정한다. 파일이 낡으면 AI가 구식 패턴으로 코드를 작성한다.

4.6 AI에 줄 제약—스택과 버전 명시

규칙 파일이 자리잡으면 그다음 단계는 **프롬프트마다 어떤 제약을 명시할 것인가**다. 핵심 규칙은 하나로 요약된다.

AI에 작업을 시킬 때는 **사용할 기술과 버전, 그리고 가정해야 할 패턴**을 명시한다.

제약을 따로 주지 않으면 AI는 학습 데이터에서 가장 흔한 패턴(예: Electron, React, PostgreSQL)으로 코드를 만든다. “Tauri 2를 쓴다”, “SvelteKit 2와 Svelte 5 rune 문법을 쓴다”, “백엔드는 Rust”같은 한두 줄을 프롬프트 윗부분에 덧붙이는 것만으로도 잘못된 기본값을 막을 수 있다.

작업별 제약은 구체적인 완료 조건으로 바꾼다. DB라면 “두 번째 시작에도 같은 migration을 다시 실행하지 않는다”, WebSocket이라면 “세 번 끊겨도 활성 연결은 하나다”, UI라면 “키보드만으로 저장과 오류 확인이 가능하다”처럼 적는다. 7장은 DB, 13장은 연결 복구, 14장은 배포 경계를 이어 설명한다. 도구 이름보다 무엇을 관찰해 성공으로 판단할지가 중요하다.

주의: 도구·기술 선택 제약은 **명시적으로, 반복적으로** 줘야 한다. 계속 상기시키지 않으면 AI는 가장 흔한 패턴을 기본으로 쓴다.

4.7 더 알아보기: MCP, Agent, 슬래시 명령

AI 도구 주변에는 새로운 용어가 빠르게 늘고 있다. 첫 회독에서는 이름과 역할만 알아두고, 실제로 만질 일이 생길 때 다시 보면 된다.

MCP(Model Context Protocol) —AI가 외부 도구나 데이터(파일 시스템, git, DB, 웹 검색 등)에 연결할 때 쓰는 표준 어댑터다. 연결해두면 “이 프로젝트의 cells 테이블 스키마 확인해줘”처럼 짧게 요청해도 AI가 직접 읽어온다. 스키마를 복사·붙여넣기 할 일이 줄어든다.

Agent —여러 단계를 스스로 계획하고 실행하는 AI다. Claude Code의 기본 동작도 이 방식이다. “이 저장소에 테스트 추가해줘”라고 하면 계획·실행·검증을 알아서 한다. 단, 단계가 한꺼번에 돌면 어디서 실수했는지 보기 어려우므로 출력이 다 보이는 모드에서 쓰는 게 안전하다.

슬래시 명령 —/commit, /review-pr처럼 **이름 붙여 저장해둔 프롬프트**다. 같은 요청을 다섯 번 이상 반복하게 되면 만들 가치가 있다. 한 번만 쓸 요청에는 만들지 않는다.

저장소 구조(모노레포 vs 분리)와 같은 결정도 첫 회독에서는 신경 쓰지 않아도 된다. 첫 앱은 단일 저장소로 시작하고, 공유할 코드가 명확해진 다음에 모노레포를 고민해도 늦지 않다. 이 주제는 책 후반의 운영 장에서 다시 다룬다.

4.8 도구 선택 시 따르는 5가지 원칙

1. **규칙 파일을 먼저 작성한다.** 새 저장소를 만들면 코드 전에 CLAUDE.md나 도구별 프로젝트 규칙부터 작성한다. 이 파일은 반복되는 제약을 안정적으로 전달한다.
2. **한 달에 한 번 설정을 점검한다.** AI 도구는 빠르게 바뀐다. 지금 최적이라 여기는 설정이 한 달 뒤엔 낡아 있을 수 있다.
3. **버전은 반드시 명시한다.** 버전을 쓰지 않으면 AI가 알아서 버전을 골라 코드를 생성한다. 그 버전이 프로젝트와 맞지 않으면 버그가 생긴다.
4. **설정을 즉시 테스트한다.** 초기 설정이 잘못되면 문제가 쌓인다. 기능을 개발하기 전에 dev, build, AI 연동이 제대로 동작하는지 먼저 확인한다.
5. **규칙 파일도 함께 관리한다.** 의존성을 올리거나 구조를 바꿀 때 CLAUDE.md와 Cursor Project Rules도 같이 수정한다. 파일이 낡으면 AI가 구식 패턴으로 코드를 작성한다.

마무리 박스: 이 장에서 반드시 기억할 것

- **도구마다 잘하는 일이 다르다.** 큰 구조 작업은 Claude Code, 한 파일 안 편집은 Cursor, 반복 입력은 자동완성. 경쟁 관계가 아니라 역할 분담이다.
- **AI에 무엇을 쓸지 명시해야 한다.** 아무 말도 안 하면 AI는 학습 데이터에서 가장 흔한 패턴 (Electron, React, PostgreSQL)을 기본으로 쓴다.
- **버전을 고정해야 한다.** 이 책의 기준인 Tauri CLI 2.11.x, SvelteKit 2.70.x, Svelte 5.56.x처럼 서로 다른 패키지의 버전을 구분해 적어야 오래된 코드가 섞이는 일을 줄일 수 있다.
- **CLAUDE.md와 .cursor/rules/*.mdc가 반복 제약을 줄여준다.** 규칙 파일은 도움을 주지만 검증을 대신하지 않는다.
- **30분 안에 환경 구축, 규칙 파일 작성, 첫 커밋까지 끝내라.** 이 패턴이 이후 모든 작업의 기준이 된다.
- MCP, Agent, 슬래시 명령은 지금은 이름과 역할만 알아도 충분하다. 필요할 때 다시 본다.

한 줄로 요약하면 이렇다. **AI 코드 품질은 모델뿐 아니라 어떤 제약과 검증 기준을 주느냐에 크게 좌우된다.**

이제 할 수 있는 것: - 작업 유형별로 AI 도구를 고른다 (구조적 작업 vs 세밀한 편집 vs 반복 코드) - 버전을 고정해서 완전한 Tauri 2 + SvelteKit 개발 환경을 설정한다 - AI 코드 품질을 높이고 수정 횟수를 줄이는 규칙 파일을 작성한다 - 가장 흔한 실수를 피한다: 버전 불일치, 제약 누락, 잘못된 스캐폴드 - 새 프로젝트 첫 30분에 생산적인 패턴을 잡는다

다음: 5장에서는 Tauri 2의 기초를 깊이 다룬다. AI 기반 워크플로우를 뒷받침하는 아키텍처, 명령 시스템, 상태 관리 패턴을 살펴본다.

상호 참조: 이 장에서 잡은 워크플로우 패턴은 6장의 빠른 프로토타이핑, 12장의 대규모 앱 분해 전략, 14장의 배포 전략으로 이어진다.

5장. Tauri 2 기초—핵심 패턴과 AI 실수 방지

Tauri 2는 이 책에서 AI에게 지시하기가 가장 까다로운 프레임워크다. 겉으로 보면 웹 프론트엔드와 Rust 백엔드를 붙인 단순한 구조처럼 보이지만, 실제로는 두 런타임 사이의 경계를 얼마나 정확히 지킵느냐가 앱의 안정성과 보안을 결정한다.

Tauri는 두 개의 독립된 런타임 위에서 동작한다. Rust가 백엔드 프로세스를 담당하고, SvelteKit을 구동하는 WebView가 프론트엔드를 맡는다. 둘은 메모리를 공유하지 않으며, **IPC 브리지(프로세스 간 통신 통로)**를 통해서만 메시지를 주고받는다. 데이터는 경계를 넘을 때마다 JSON 형태로 변환되어 전달된다(직렬화/역직렬화).

이 경계는 단순한 구분이 아니라 보안 설계의 핵심이고, AI가 가장 실수를 많이 내는 지점이기도 하다. AI는 이 경계를 무시하고 함수를 직접 부르려 하거나, 비밀 키를 프론트엔드에 노출하거나, 컴파일은 되지만 부하가 걸리면 멈춰버리는 비동기 코드를 작성하기도 한다.

이 장은 Tauri 문법 요약이 아니라 **경계 지도**에 가깝다. 어느 일이 어느 쪽에 속하는지는 결국 개발자가 판단해야 한다.

Tauri를 처음 접할 때는“작은 웹 앱 + 안전한 시스템 백엔드”로 이해하면 편하다. 화면은 웹 기술로 만들고, 파일·DB·프로세스·키체인처럼 민감한 작업은 Rust 명령으로 처리한다. 프론트엔드는 요청만 보내고, 실제 권한은 백엔드가 갖는다.

경계를 구분하는 기준은 하나다. **사용자 화면을 바꾸는 일은 프론트엔드, 운영체제나 데이터에 손대는 일은 Rust 명령이다.** 버튼 색을 바꾸거나 목록을 필터링하는 일은 Svelte에서 끝나야 한다. 반대로 파일을 저장하거나, DuckDB를 열거나, API 키를 키체인에서 읽는 일은 Rust 명령으로 처리해야 한다.

식당으로 비유하면 프론트엔드는 손님이 보는 홀이고, Rust 백엔드는 주방과 금고다. 손님은 메뉴를 고르고 요청을 보낼 수 있지만, 냉장고를 직접 열거나 금고에서 카드를 꺼내지는 못한다. 주문서는 정해진 창구를 통해 주방으로 들어가고, 완성된 음식만 다시 홀로 나온다. Tauri의 `invoke()`가 바로 그 주문서다.

이 장의 흐름

- 5.1 아키텍처 개괄—왜 경계가 중요한가
- 5.2 명령(command)—프론트엔드가 백엔드를 부르는 방식
- 5.3 상태(State)—명령 사이에 데이터를 공유하는 방법
- 5.4 이벤트—백엔드에서 프론트엔드로 데이터 흘려보내기
- 5.5 플러그인—자주 쓰는 기능 묶음
- 5.6 권한과 기능—Tauri 2의 새로운 보안 모델
- 5.7 프론트엔드—SvelteKit 2·Svelte 5와의 연결
- 5.8 프로젝트 구조
- 5.9 멘털 모델과 프롬프팅 전략
- 5.10 AI가 자주 저지르는 실수
- 마무리 박스와 빠른 참조

학습 목표

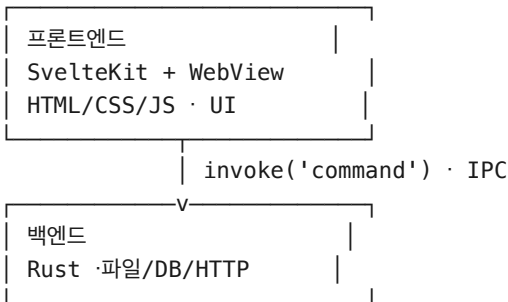
- Tauri 2의 두 런타임 구조(Rust 백엔드 + WebView 프론트엔드)를 이해한다.
- `#[tauri::command]`로 백엔드 함수를 프론트엔드에 노출하는 패턴을 익힌다.
- `tauri::State<T>`로 명령 사이에 데이터를 공유한다.
- 이벤트 시스템으로 백엔드의 진행 상황을 실시간으로 화면에 흘려보낸다.

- Tauri 2의 권한 모델(capabilities)을 이해하고, 권한 누락 에러를 빠르게 잡아낸다.
- AI가 Tauri 2 코드에서 자주 틀리는 패턴을 안다.

이 장을 다 읽고 나면 최소한 하나는 만들 수 있어야 한다. 프론트엔드 버튼을 누르면 Rust 명령이 실행되고, 결과 문자열이 화면에 표시되는 가장 작은 Tauri 앱이다. 뒤의 장들은 모두 이 흐름을 확장한다.

5.1 Tauri 2 아키텍처 개괄—명확한 경계가 중요한 이유

두 프로세스가 어떻게 연결되는지 그림으로 먼저 보자.



프론트엔드에서 `invoke('get_user_data')`를 호출하면 다음 순서로 처리된다.

1. `invoke()`가 함수 이름과 인수를 JSON으로 묶는다.
2. IPC 레이어가 그 데이터를 백엔드로 전달한다.
3. Rust가 `get_user_data` 명령을 실행한다.
4. Rust가 결과를 돌려보낸다.
5. 프론트엔드가 응답을 Promise로 받는다.

흐름은 HTTP API 호출과 비슷하지만 네트워크를 거치지 않는다. 같은 앱 안에서 WebView와 Rust 프로세스가 내부 통로로 메시지를 주고받는 구조다.

이 아키텍처가 AI 개발에 잘 맞는 이유는 **경계가 딱 떨어지기** 때문이다. “사용자 설정을 저장하는 백엔드 명령을 작성해”라고 하면 AI가 만들어야 할 것이 명확하다. Electron은 메인/렌더러 둘 다 JavaScript라 AI가 프론트엔드와 백엔드를 뒤섞기 쉽지만, Tauri 2는 Rust와 JavaScript라는 언어 차이가 분리를 강제한다.

네이티브 UI가 아닌 WebView인 이유

Tauri 2는 시스템에 내장된 WebView를 쓴다(macOS의 WebKit, Windows의 WebView2, Linux의 WebKitGTK). 덕분에 UI의 대부분을 HTML/CSS/JavaScript로 공유할 수 있다. 다만 WebView 엔진, 파일 경로, 권한, 서명 방식은 운영체제마다 다르므로 배포 전 플랫폼별 테스트는 필요하다.

5.2 명령—#[tauri::command] 패턴

명령은 프론트엔드가 백엔드 함수를 호출하는 방법이다. 파일 시스템 접근, 데이터베이스 쿼리, HTTP 요청은 모두 명령으로 처리한다.

명령 이름을 짓는 원칙

명령 이름을 UI 이벤트에 그대로 묶지 않는다. onSaveButtonClick은 특정 버튼에 묶인 이름이고, save_user_settings는 버튼이 사라져도 그대로 쓸 수 있다. **명령은 업무 동작을 표현해야지, 화면 이벤트를 표현하면 안 된다.**

가장 작은 예제

이 코드의 역할: 프론트엔드에서 이름을 보내면 Rust가 인사 문자열을 만들어 돌려준다. Tauri 명령에 필요한 세 가지(#[tauri::command], invoke_handler 등록, 프론트엔드 invoke() 호출)가 모두 들어 있다.

```
// src-tauri/src/lib.rs
#[tauri::command]
pub fn greet(name: String) -> String {
    format!("안녕, {}!", name)
}

pub fn run() {
    tauri::Builder::default()
        .invoke_handler(tauri::generate_handler![greet])
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}
```

프론트엔드는 같은 이름으로 호출한다.

```
// src/lib/api.ts
import { invoke } from '@tauri-apps/api/core';

export async function greet(name: string): Promise<string> {
  return await invoke('greet', { name });
}
```

확인할 점: 세 곳의 이름이 모두 일치해야 한다. Rust 함수명, generate_handler![] 등록, 프론트엔드 invoke(). 어느 하나가 어긋나면 실행 시점에 command not found가 난다. **명령을 추가할 때 가장 자주 잡히는 버그가 이 이름 불일치다.**

인수 이름도 확인해야 한다. 프론트엔드에서는 보통 { filePath }처럼 camelCase를 쓰고, Rust에서는 file_path처럼 snake_case를 쓴다. Tauri가 변환해주지만, 이름이 서로 대응되지 않으면 값이 None처럼 비어 들어온다. 명령이 호출되는데 값만 이상하다면 이름 변환을 먼저 의심한다.

실패할 수 있는 명령

실제 앱의 명령은 대부분 비동기이고 실패할 수 있다. 파일이 없거나, 권한이 없거나, 네트워크가 끊길 수 있다. 성공값만 반환하지 말고 Result<T, String>으로 실패를 명시한다.

```
#[tauri::command]
pub async fn read_config(path: String) -> Result<String, String> {
  if !std::path::Path::new(&path).exists() {
    return Err(format!("파일을 찾을 수 없습니다: {}", path));
  }
  tokio::fs::read_to_string(&path).await
    .map_err(|e| format!("파일 읽기 실패: {}", e))
}
```

이 예제는 Result 흐름만 보여주는 최소 코드다. 실제 앱에서 프론트엔드가 임의의 절대 경로를 넘기게 두면 안 된다. 허용된 앱 데이터 디렉터리를 기준으로 경로를 정규화하고, ..나 심볼릭 링크를 거친 최종 경로가 허용 범위 밖으로 나가지 않는지 확인한다.

프론트엔드에서는 이 실패를 try/catch로 받는다.

```
try {
    const text = await invoke<string>('read_config', { path });
} catch (error) {
    console.error('설정 읽기 실패:', error);
}
```

여러 필드를 가진 데이터

설정, 주문, 세션처럼 여러 필드를 가진 데이터는 구조체로 정의하고 Serialize/Deserialize를 붙인다. Tauri가 자동으로 변환해 주므로, 프론트엔드와 백엔드가 같은 데이터 모양을 공유한다.

```
use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize)]
pub struct UserSettings {
    theme: String,
    auto_save: bool,
    recent_files: Vec<String>,
}

#[tauri::command]
pub async fn save_user_settings(settings: UserSettings) -> Result<(),
String> {
    // 파일이나 DB에 저장
    Ok(())
}
```

확인할 점: 새 명령(save_user_settings)도 등록 목록에 추가해야 한다.

```
.invoke_handler(tauri::generate_handler![
    greet,
    save_user_settings,
])
```

AI 프롬프팅 팁: “사용자 설정을 관리하는 명령을 만들어줘”보다는 구조체 정의(필드 이름과 타입)를 함께 주면 결과가 훨씬 일관적이다.

5.3 상태 관리—`tauri::State<T>`

여러 명령이 데이터를 공유해야 할 때는 Tauri의 내장 상태 관리를 쓴다. 전역 변수보다 수명 주기와 주입 경로가 분명하다. 여러 스레드에서 공유하려면 상태 타입 자체도 `Send + Sync` 조건을 만족해야 하며, 내부 가변 데이터에는 용도에 맞는 잠금이나 메시지 채널을 둔다.

상태(State)는 앱 안의 공용 보관함이라고 보면 된다. 여러 명령이 같은 설정, DB 연결, 캐시를 써야 할 때 매번 새로 만들지 않고 보관함에서 꺼내 쓴다. 단, 동시에 여러 명령이 손을 뻗을 수 있으므로 잠금장치도 필요하다.

기본 패턴: `RwLock`

이 코드의 역할: 명령들이 공유하는 캐시를 앱 시작 시 한 번 등록하고, 각 명령은 `State<'_, AppState>`로 그 값을 꺼내 쓴다.

```
use std::sync::RwLock;
use std::collections::HashMap;
use tauri::State;

pub struct AppState {
    pub cache: RwLock<HashMap<String, String>>,
}

#[tauri::command]
pub async fn set_cache_value(
    key: String, value: String, state: State<'_, AppState>
) -> Result<(), String> {
    let mut cache = state.cache.write()
        .map_err(|e| format!("Lock 에러: {}", e))?;
    cache.insert(key, value);
    Ok(())
}

// 앱 시작 시 상태 등록
fn run() {
    tauri::Builder::default()
        .manage(AppState { cache: RwLock::new(HashMap::new()) })
        .invoke_handler(tauri::generate_handler![set_cache_value])
        .run(tauri::generate_context!())
}
```

```

        .expect("error");
    }

```

RwLock은 데이터를 안전하게 공유하기 위한 잠금장치다. 쓸 때는 `write()`로 하나만 들어가고, 읽을 때는 `read()`로 여러 곳이 동시에 들어갈 수 있다.

잠금 선택: 자료형보다 임계 구역을 먼저 본다

`std::sync::RwLock`과 `tokio::sync::Mutex` 중 하나가 언제나 정답인 것은 아니다. 캐시처럼 잠깐 읽거나 쓰고 즉시 놓는 동기 작업에는 표준 `RwLock`이 단순하다. 반대로 잠금을 잡은 상태에서 비동기 작업을 기다려야 한다면 Tokio 잠금이 필요할 수 있다. 가장 중요한 규칙은 **필요하지 않은 락을 `.await` 너머까지 들고 가지 않는 것이다**.

먼저 위험한 예를 보자. 다음 코드는 공유 데이터를 잠근 채 네트워크 요청까지 기다린다. 그동안 같은 상태가 필요한 다른 명령은 모두 대기한다.

```

// ☒ 잠금을 잡은 채 느린 비동기 작업을 기다림
use tokio::sync::Mutex;
pub struct BadState { data: Mutex<Vec<String>> }

#[tauri::command]
pub async fn bad_command(state: State<'_, BadState>) -> Result<(),
String> {
    let mut data = state.data.lock().await; // 비동기 락
    data.push("item".to_string());
    send_to_server(&data).await?;           // 락이 여기까지 유지됨
    Ok(())
}

```

수정 코드가 맡는 일은 공유 상태를 짧게 갱신한 뒤 복사본만 비동기 작업에 넘기는 것이다. 종괄호 블록이 끝나는 지점에서 잠금이 먼저 풀리는지 확인한다.

```

// ☑ 짧은 동기 임계 구역: 수정한 뒤 락을 바로 놓음
use std::sync::RwLock;
pub struct GoodState { data: RwLock<Vec<String>> }

#[tauri::command]

```

```
pub async fn good_command(state: State<'_, GoodState>) -> Result<(),
String> {
    let snapshot = {
        let mut data = state.data.write()
            .map_err(|e| format!("Lock poisoned: {}", e))?;
        data.push("item".to_string());
        data.clone()
    }; // 여기서 락 해제
    send_to_server(&snapshot).await?;
    Ok(())
}
```

읽기가 쓰기보다 훨씬 많을 때는 RwLock, 한 번에 한 작업만 접근해야 하는 자원에는 Mutex가 자연스럽다. 동기 DB 쿼리처럼 오래 걸리는 작업은 비동기 실행기 안에서 락으로 버티기보다 전용 작업 스레드나 spawn_blocking으로 분리하는 편이 안전하다.

초기화 비용이 큰 자원

DuckDB 연결처럼 만드는 데 시간이 걸리는 자원은 앱 전체에서 한 번만 만들고 싶다. 이런 경우는 7장에서 DuckDB 예제와 함께 다시 다룬다. 지금은 “한 번만 만들고 여러 명령이 공유하는 패턴이 따로 있다”정도만 기억하면 된다.

5.4 이벤트—emit과 listen

명령은 프론트엔드가 백엔드에 묻는 방식이다. 그런데 백엔드가 먼저 프론트엔드에 알려야 할 때도 있다. AI 응답 스트리밍, 장기 실행 프로세스의 진행 상황 같은 경우다. 프론트엔드가 “끝났나?”를 계속 묻는 대신, 백엔드가 이벤트를 밀어낸다(push).

백엔드에서 프론트엔드로 보내기

이 코드의 역할: 10초 동안 매초 진행 상황을 프론트엔드로 보내고, 마지막에 완료 이벤트를 보낸다.

```

use tauri::{AppHandle, Emitter}; // emit()은 Emitter 트레이트이 제공

#[tauri::command]
pub async fn start_long_process(app: AppHandle) -> Result<(), String> {
    tokio::spawn(async move {
        for i in 0..10 {
            tokio::time::sleep(tokio::time::Duration::from_secs(1)).await;
            let _ = app.emit("process_progress", (i + 1) as f64 / 10.0);
        }
        let _ = app.emit("process_complete", ());
    });
    Ok(())
}

```

실패하면 볼 것: use tauri::Emitter를 빠뜨리면 app.emit()이 컴파일되지 않는다.

프론트엔드는 listen으로 받는다. 리스너는 더 이상 필요 없을 때 해제해야 메모리 누수가 없다.

```

import { listen } from '@tauri-apps/api/event';

$effect(() => {
    const unlistenPromise = listen<number>('process_progress', (event)
    => {
        console.log('진행:', event.payload);
    });

    // 등록이 끝나기 전에 컴포넌트가 사라져도 정리된다.
    return () => { void unlistenPromise.then((unlisten) => unlisten());
    };
});

```

스트리밍 패턴

AI 응답을 화면에 흘려보낼 때도 같은 패턴이다. 모델이 토큰을 하나 만들 때마다 이벤트로 흘리고, 다 끝나면 완료 이벤트를 보낸다.

```
#[tauri::command]
pub async fn stream_ai_response(
    prompt: String,
    app: AppHandle,
) -> Result<(), String> {
    let mut stream = client.chat().create_stream(prompt).await?;
    while let Some(chunk) = stream.next().await {
        let chunk = chunk?;
        if let Some(content) = chunk.choices.first()
            .and_then(|choice| choice.delta.content.clone())
        {
            let _ = app.emit("ai_chunk", content);
        }
    }
    let _ = app.emit("ai_complete", ());
    Ok(())
}
```

이 패턴은 6장(타이머 알림), 8장(AI 응답 스트리밍), 15장(프로세스 출력 캡처)에서 반복해서 나온다.

처리량 주의: Tauri 공식 문서도 이벤트를 작은 JSON 메시지에 적합한 통로로 설명한다. 토큰이나 프로세스 출력을 매우 빠르게 보낼 때는 여러 조각을 20~50ms 단위로 묶거나, 순서와 처리량에 맞게 설계된 Tauri 채널(Channel)을 사용한다. provider가 보낸 토큰마다 전역 이벤트를 하나씩 발생시키면 렌더링과 직렬화 비용이 커질 수 있다.

5.5 플러그인—tauri-plugin-* 생태계

Tauri 2는 자주 쓰는 기능을 플러그인으로 제공한다. 각 플러그인은 Rust 명령과(필요하면) 프론트엔드 JavaScript API를 함께 묶어준다.

AI 앱에 자주 쓰는 플러그인

플러그인	사용 사례
tauri-plugin-shell	프로세스 생성, Python 스크립트 실행

플러그인	사용 사례
<code>tauri-plugin-store</code>	사용자 선호도 같은 설정 저장
<code>tauri-plugin-dialog</code>	파일/폴더 선택 다이얼로그
<code>tauri-plugin-fs</code>	파일 읽기/쓰기
<code>tauri-plugin-http</code>	프론트엔드에서 권한·범위를 제한한 HTTP 요청
<code>tauri-plugin-websocket</code>	WebSocket 연결

플러그인 추가는 두 단계다. `Cargo.toml`에 의존성을 더하고, Rust 코드에서 등록한다.

```
fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_store::Builder::new().build())
        .plugin(tauri_plugin_dialog::init())
        // ...
}
```

보안 주의: `tauri-plugin-store`는 데이터를 디스크에 JSON으로 저장한다. API 키처럼 노출되면 안 되는 값에는 쓰지 않는다. `tauri-plugin-stronghold`는 IOTA Stronghold 기반의 암호화 금고이지 운영체제 키체인 자체는 아니다. macOS Keychain·Windows Credential Manager·Linux Secret Service를 쓰려면 Rust의 `keyring` 같은 별도 크레이트를 선택한다.

5.6 권한과 기능—Tauri 2의 새로운 보안 모델

Tauri 2의 `capability`는 **WebView** 프론트엔드가 Tauri의 코어·플러그인 API 중 무엇을 호출할 수 있는지 제한한다. 창이나 WebView별로 허용할 권한과 경로·URL 범위를 나눌 수 있다. 반면 `generate_handler!`에 등록한 사용자 정의 Rust 명령은 기본적으로 모든 앱 창에서 호출할 수 있고, 명령 안에서 직접 작성한 Rust 코드는 `capability`가 자동으로 검사하지 않는다. `capability`는 악성·취약한 Rust 코드를 대신 막아주는 장치가 아니다.

```
{
  "$schema": "../gen/schemas/desktop-schema.json",
  "identifier": "main",
  "windows": ["main"],
  "permissions": [
    "core:default",
    "fs:allow-read-text-file",
    "fs:allow-write-text-file",
    "dialog:allow-open",
    "dialog:allow-save",
    "http:default",
    "store:default"
  ]
}
```

default 권한 묶음의 내용은 플러그인마다 다르다. 이름만 보고 넓거나 좁다고 단정하지 말고 생성된 스키마나 공식 권한 표에서 실제 포함 항목을 확인한다. 보안 경계가 중요한 기능은 fs:allow-read-text-file처럼 필요한 동작만 고르고, 경로·URL scope도 함께 좁힌다.

AI가 권한을 빠뜨리는 이유

Claude Code로 프론트엔드의 파일 플러그인 호출을 만들면 capability 파일 업데이트를 빠뜨릴 때가 많다. 예를 들어 다음 JavaScript 코드는 tauri-plugin-fs를 통해 파일을 읽으므로 해당 플러그인 권한과 경로 scope가 필요하다.

```
import { readTextFile } from '@tauri-apps/plugin-fs';
import { BaseDirectory } from '@tauri-apps/api/path';

const text = await readTextFile('notes/today.md', {
  baseDir: BaseDirectory.AppData
});
```

권한이 없으면 호출 Promise가 권한 오류로 거절된다.

```
[ERROR] fs.read_text_file not allowed by capability 'main'
```

실패하면 볼 것: 플러그인 등록, permissions 항목, 허용 경로 scope를 함께 확인한다. 에러 문자열은 플러그인 버전에 따라 달라질 수 있으므로 메시지를 그대로 권한 식별자로 복사하지 말고 생성된 capability 스키마에서 정확한 이름을 고른다.

체크 습관: 프론트엔드가 파일·네트워크·셀 플러그인을 호출하도록 AI가 코드를 만든 직후에는 `src-tauri/capabilities/*.json`을 열어 권한과 scope를 확인한다. 사용자 정의 Rust 명령이 같은 작업을 직접 수행한다면 capability에 기대지 말고 명령 안에서 경로 검증·허용 목록·인증을 구현한다.

5.7 프론트엔드—SvelteKit 2 + Svelte 5 + Tauri

Tauri 앱의 프론트엔드는 WebView 안에서 동작하는 SvelteKit 애플리케이션이다. Tauri 브리지 패키지만 추가하면 시스템 API에 접근할 수 있다.

```
pnpm add @tauri-apps/api
```

핵심 임포트는 세 가지다.

```
import { invoke } from '@tauri-apps/api/core';
import { listen } from '@tauri-apps/api/event';
import { getCurrentWindow } from '@tauri-apps/api/window';
```

Svelte 5 rune과 Tauri 이벤트의 궁합

이 코드의 역할: 백엔드가 보내는 진행률을 `$state`에 받아 화면에 그대로 비춘다. `$derived`로 표시 텍스트를 자동으로 계산한다.

```
<script lang="ts">
import { invoke } from '@tauri-apps/api/core';
import { listen } from '@tauri-apps/api/event';

let progress = $state(0);
let isProcessing = $state(false);

async function startProcess() {
  isProcessing = true;
  try {
    await invoke('start_long_process');
```

```

    } catch (error) {
      isProcessing = false;
      console.error(error);
    }
  }

  let progressPercent = $derived(Math.round(progress * 100));

  $effect(() => {
    const subscriptions = [
      listen<number>('process_progress', (e) => { progress =
e.payload; }),
      listen('process_complete', () => { isProcessing = false; }),
    ];
    return () => {
      for (const pending of subscriptions) {
        void pending.then((unlisten) =>
unlisten()).catch(console.error);
      }
    };
  });
</script>

<div>{isProcessing ? `처리 중... ${progressPercent}%` : '준비'}</div>

```

여기서 잡아둘 핵심은 두 가지다. **\$state**로 만든 변수는 이벤트 콜백에서 바뀌도 자동으로 화면에 반영된다. 그리고 **listen()**이 돌려준 **unlisten()**을 컴포넌트 종료 시 반드시 호출한다. 전체 컴포넌트는 교육용 코드 묶음의 `tauri/ch05-local-notes/src/App.svelte`를 참고한다. Svelte 5 + Vite + Tauri 2로 만든 독립 실습이며 SvelteKit 전체 앱이나 Mandai 소스는 아니다. Rust 명령·앱 전용 파일·이벤트 해제·오류 처리를 확인하는 작은 예제다. 같은 폴더 README에 설치와 완료 기준을 적었다.

API 래퍼로 명령 호출 모으기

`invoke()`를 컴포넌트 안에서 직접 부르면 명령 이름 오타가 곳곳에 퍼진다. 모든 명령 호출을 `src/lib/api/` 디렉터리의 함수로 감싸면 변경에 강해진다.

```
// src/lib/api/files.ts
import { invoke } from '@tauri-apps/api/core';

export const FilesAPI = {
  readDirectory: (path: string) =>
    invoke<FileItem[]>('read_directory', { path }),
  readFile: (path: string) => invoke<string>('read_file', { path }),
  writeFile: (path: string, contents: string) =>
    invoke<void>('write_file', { path, contents }),
};
```

컴포넌트는 `FilesAPI.readFile(...)` 만 부르면 된다. 명령 이름이 바뀌어도 한 곳만 고치면 끝난다.

5.8 프로젝트 구조—참조 레이아웃

이 책의 6개 프로젝트 모두에서 같은 구조를 쓴다.

```
my-tauri-app/
├── src-tauri/                                # Rust 백엔드
│   ├── src/
│   │   ├── lib.rs                          # 진입점
│   │   ├── commands/                      # files.rs, ai.rs, database.rs ...
│   │   ├── state/                         # app_state.rs
│   │   └── utils/
│   ├── capabilities/main.json
│   ├── Cargo.toml
│   └── tauri.conf.json
├── src/                                      # SvelteKit 프론트엔드
│   ├── lib/
│   │   ├── api/                          # files.ts, ai.ts ...
│   │   ├── stores/                       # app.svelte.ts, files.svelte.ts
│   │   └── components/
│   ├── routes/
│   └── app.html
├── package.json
└── vite.config.ts
```

Claude Code에 작업을 요청할 때는 디렉터리를 함께 지정한다.

“사용자 선호도를 저장하는 명령을 `src-tauri/src/commands/database.rs`에 추가하고, `src/lib/api/index.ts`에 래퍼 함수도 같이 만들어줘.”

그러면 AI가 코드를 어디에 넣을지 헤매지 않는다.

5.9 AI 개발을 위한 Tauri 2 멘털 모델

Tauri 2는 Rust 백엔드와 WebView 프론트엔드의 역할 경계를 드러낸다. 그래서 AI에게 파일 위치와 책임을 지시하기 쉽다.

- 프론트엔드는 브라우저 API만으로 로컬 파일 시스템에 접근할 수 없다 -> 제한된 Tauri 플러그인이나 검증된 사용자 정의 명령을 쓴다
- 백엔드는 DOM을 직접 건드릴 수 없다 -> 이벤트로 알려야 한다
- 데이터는 경계를 넘을 때 반드시 변환된다
- 코어·플러그인 API 권한은 capability 설정에 미리 선언한다

이 경계가 많은 실수를 일찍 드러내지만 자동 안전 보장은 아니다. 등록된 사용자 정의 명령의 입력 검증과 Rust 내부 권한 검사는 앱 코드가 직접 책임진다.

프롬프팅 전략

같은 기능이라도 프롬프트를 어떻게 쓰느냐에 따라 결과가 다르다.

- ☒ "파일을 저장하는 기능 추가해줘"
- ☑ "src-tauri/src/commands/files.rs에 텍스트 내용을 파일 경로에 저장하는 Rust 명령을 추가해줘. 잘못된 경로와 권한 문제에 대한 예러 처리를 포함해. 그다음 src/lib/api/files.ts에 이 명령을 호출하는 TypeScript 래퍼를 만들어줘."

어느 쪽에서 어떤 일을 할지 명시할수록, 구조에 맞는 코드가 나온다.

5.10 AI가 Tauri 2에서 자주 저지르는 실수들

자주 만나는 실수 세 가지부터 본다. 전체 목록은 다음 절 빠른 참조에 있다.

1. 권한 누락: 프론트엔드 플러그인 호출은 만들었는데 `capabilities/main.json`의 권한이나 `scope` 업데이트를 빠뜨린다. `not allowed` 같은 런타임 에러가 나오면 플러그인 등록과 `capability`를 함께 확인한다.

2. 잠금 범위 오류: 공유 상태의 락을 잡은 채 네트워크·파일 작업을 `.await`한다. 어떤 잠금 타입이든 임계 구역을 짧게 만들고, 느린 작업 전에 락을 놓는다(5.3 참고).

3. Tauri 1 API: 학습 데이터에 Tauri 1 코드가 섞여 있어서, `window.__TAURI__.invoke()`나 `appWindow` 같은 옛 모양이 나올 수 있다. Tauri 2의 모듈 방식은 `@tauri-apps/api/core`의 `invoke`, `@tauri-apps/api/event`의 `listen`, `getCurrentWindow()`를 쓴다. 전역 API를 의도적으로 켜 경우에는 `window.__TAURI__.core.invoke()`처럼 namespace가 하나 더 있으므로 두 형식을 섞지 않는다.

마무리 박스: 이 장에서 반드시 기억할 것

이 장에서 반드시 기억할 것을 다시 정리한다.

- **두 영역의 경계:** Rust 백엔드와 WebView 프론트엔드는 메모리를 공유하지 않는다. 화면 일은 프론트가 맡고, OS 자원·DB·비밀 키는 최소 권한 플러그인이나 Rust 명령 뒤에 둔다.
- **명령 만들기:** `#[tauri::command]`를 붙이고, 핸들러에 등록하고, 프론트엔드에서 `invoke()`로 부른다. 이름 세 곳이 모두 같아야 한다.
- **상태 공유:** `tauri::State<T>`안의 잠금은 작업 성격에 맞게 고르고, 락을 잡은 채 느린 작업을 기다리지 않는다.
- **이벤트:** 백엔드가 작은 알림을 먼저 보낼 때는 `app.emit()`과 `listen()`을 쓴다. 고속 스트림은 묶어서 보내거나 `Channel`을 검토한다.
- **권한:** 프론트엔드의 코어·플러그인 호출에는 `capabilities/*.json` 권한과 `scope`가 필요하다. 사용자 정의 Rust 명령의 검증은 명령 구현이 직접 책임진다.
- **AI 협업:** 어느 쪽에서 어떤 일을 할지 분명히 말할수록 결과가 좋다.

빠른 참조—AI가 Tauri 2에서 하는 실수 10가지

실수	AI가 만드는 것	문제	올바른 버전
Tauri 1 API	<code>appWindow.listen(...)</code>	v1 전역 객체	<code>getCurrentWindow().listen(...)</code> 또는 <code>event listen(...)</code>

실수	AI가 만드는 것	문제	올바른 버전
권한 누락	capability 없이 FS 플러그인 호출	런타임 권한 에러	권한과 경로 scope 추가
잠금 범위 오류	락을 잡고 <code>.await</code>	장시간 대기·교착 위험	느린 작업 전에 락 해제
서비스 키 노출	프론트에서 비밀 키로 <code>fetch()</code>	번들·런타임에 서 키 노출	사용자 키는 보안 저장소, 서비스 키는 서버 프록시
무거운 동기 명령	메인 스레드에서 긴 계산·I/O	UI 멈춤	<code>async</code> 또는 <code>spawn_blocking</code>
문자열 에러	모든 곳 <code>Result<T, String></code>	처리 어려움	커스텀 에러 타입
전역 상태	<code>static mut GLOBAL_VAR</code>	경쟁 조건	<code>tauri::State<T></code>
무제한 프로세스 생성	프론트 입력을 그대로 Command에 전달	임의 명령 실행 위험	고정 명령·인자 검증 또는 제한된 shell scope
컨텍스트 혼합	WebView에 네이티브 DB 드라이버 사용	브라우저 환경에서 실행 불가	Rust 명령으로 DB 접근
이벤트 명명	"update"같은 일반명	이름 충돌	"ai_progress"같은 구체명

검증 체크리스트

AI가 새 명령을 만든 직후 매번 확인할 세 가지다.

1. **권한**: 프론트엔드가 파일·네트워크·셸 플러그인을 호출한다면 `src-tauri/capabilities/`에 최소 권한과 scope가 있는가?

```
grep -r "fs:\|shell:\|http:" src-tauri/capabilities/
```

2. **이름 일치**: Rust 함수명 = `generate_handler![]` 등록명 = 프론트엔드 `invoke()` 이름
3. **실행 테스트**: `npm run tauri dev`로 띄워 정상 동작과 실패 경로(잘못된 파일, 네트워크 실패)를 모두 확인

이 단계를 건너뛰면 나중에 더 어려운 버그를 만난다.

이 장에서 다룬 명령, 상태, 이벤트 패턴은 6장에서 다시 등장한다. 타이머, AI 스트리밍, DuckDB 레이어를 직접 만들면서 각 패턴이 실제 코드에서 어떻게 쓰이는지 확인한다.

6장. Mandai 설계—AI에게 얼마나 큰 단위로 맡길 수 있 나

Part II —Mandai (6~9장) 이제 실제 앱을 만든다. 1~5장이 원칙과 도구였다면, Part II는 그것들이 한 프로젝트 전체에서 어떻게 작동하는지 보여준다. 설계(6장) -> DB(7장) -> AI API(8장) -> 실수와 교훈(9장) 순서로, 루프가 실제로 어떻게 돌아가는지 따라간다. 첫 회독에서는 Mandai의 모든 기능을 외우지 않아도 된다. “AI에게 앱 전체를 한 번에 맡기지 않고, 화면·데이터·명령으로 나눠 맡긴다”는 구조만 잡으면 된다.

프로젝트를 시작할 때 먼저 답해야 할 질문은 “어떤 프롬프트를 써야 하나”가 아니다. 그보다 앞에 있는 질문이 있다. **내가 실제로 무엇을 만드는가.**

이 질문에 답하기 전에 AI에게 요청하면 어떻게 될까. AI는 방향 없이 빠르게 만들어낸다. 파일명은 그럴듯하고 폴더 구조도 있어 보이지만, 각 부분이 서로 맞지 않는다.

반대로 너무 크게 맡기면 어떨까. 앱 전체를 한 번에 넘기면 AI가 한 번에 소화하기 어렵다. 그렇다고 너무 잘게 나누면 이번엔 맥락이 끊긴다. AI는 앞에서 무엇을 만들었는지 모른 채 다음 부분을 작성한다.

Mandai는 이 균형을 찾기에 좋은 프로젝트였다. Mandal-Art, GTD, Pomodoro, AI 채팅은 각각 독립된 기능처럼 보인다. 하지만 실제 제품 안에서는 하나의 목표 관리 흐름으로 이어져야 했다. 이 장은 Mandai를 설계하면서 AI가 한 번에 받아낼 수 있는 크기를 찾아간 과정이다.

이 장에는 코드가 거의 없다. 대신 Mandai의 뼈대가 잡혀가는 순서를 그대로 따라간다. **내가 보낸 것, AI가 응답한 것, 그리고 그 응답이 설계를 어떻게 바꿨는지**를 함께 본다. 여기서 언어야 할 것은 완성된 폴더 구조가 아니다. AI와 설계를 주고받을 때 어떤 질문이 필요한지, 그 감각이다.

요리로 비유하면, AI에게 “저녁 한 상 차려줘”라고 하면 메뉴, 재료, 조리 순서를 AI가 마음대로 정한다. 반대로 “양파를 1cm로 썰어줘”만 반복하면 전체 요리가 보이지 않는다. 좋은 단위는 “카레를 만들 건데, 먼저 재료 목록과 조리 순서를 제안해줘” 정도다. 전체 방향은 있고, 한 번에 처리할 범위는 작다. Mandai 설계도 같은 방식으로 진행했다.

이 장의 흐름

Mandai라는 앱의 형태를 먼저 잡고, AI에게 맡길 수 있는 작업 단위를 점점 좁혀간다.

- Mandai는 처음에 어떤 앱이었나
- 첫 번째 실수—“전체를 한 번에 만들어줘”
- 크기 조절—AI가 한 번에 처리할 수 있는 단위
- AI의 반문을 설계 도구로 활용하기
- 세 계층으로 구성하기: 화면, 데이터, 명령
- AI 청크 크기 법칙
- 빌드 중간에 계획이 바뀔 때

학습 목표

- AI에게 얼마나 큰 작업을 한 번에 맡길 수 있는지, 그 한계가 어디인지 파악한다.
- AI가 되묻는 질문을 설계 결정의 도구로 활용하는 방법을 익힌다.
- 앱을 화면, 데이터, 명령 세 계층으로 나누는 전략을 배운다.
- Mandai 초기 설계 과정에서 AI와 함께 구조를 잡는 패턴을 직접 따라간다.

6.1 Mandai는 처음에 어떤 앱이었나

Mandai는 네 가지 기능을 하나로 묶은 생산성 도구다. 나열하면 흔한 앱처럼 들릴 수 있다. 하지만 핵심은 이 네 가지를 따로 쓰는 게 아니라 하나의 흐름으로 연결하는 것이다. 목표, 오늘 할 일, 집중 시간, AI 대화가 각각 분리되지 않고 서로 이어진다.

- **Mandal-Art (9×9 목표 그리드):** 중앙 목표 하나와 서브 목표 여덟 개. 각 서브 목표는 다시 9칸 그리드로 확장된다. $9 \times 9 =$ 총 81칸.
- **GTD:** 오늘의 할일 목록. 각 태스크는 81칸 중 하나에 바인딩된다.
- **Pomodoro:** 25분 집중, 5분 휴식. 완료한 세션을 DuckDB에 기록한다.
- **AI API 통합:** 채팅 창 하나에서 OpenAI, Claude, Gemini를 바꿔 쓴다. 스트리밍을 지원한다.

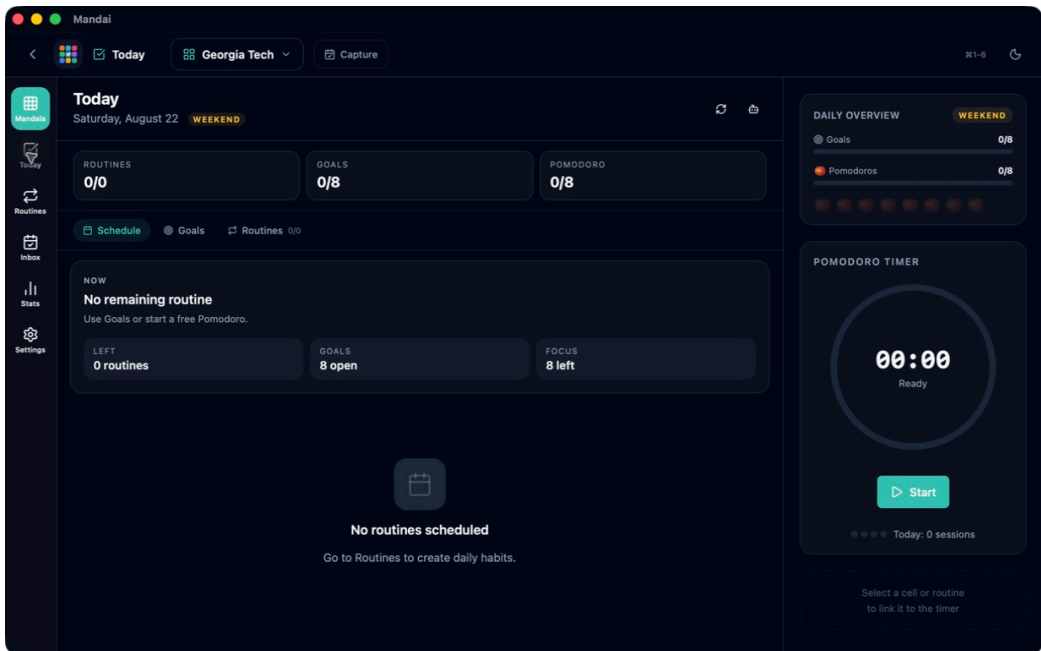


그림 6-1. 실제로 실행한 Mandai의 Today 화면. 일정, 목표, 루틴, Pomodoro 상태가 한 화면에 모여 있다.

네 가지를 하나로 합친 이유는 단순하다. **Pomodoro 세션은 내 81칸 중 하나에 연결될 때만 의미가 있다.** 집중 시간을 기록하는 것만으로는 부족하다. 어떤 목표의 어떤 영역에 시간을 썼는지도 남아야 한다. 그래야 나중에 내가 실제로 어디에 에너지를 쏟고 있는지 볼 수 있다.

기존 앱에는 그 연결이 없었다. Todoist에는 목표 트리가 없고, Forest에는 태스크가 없고, Notion에는 Pomodoro가 기본 흐름으로 들어 있지 않다.

그래서 직접 만들었다. 이 결정이 이후 모든 설계의 기준이 된다. 단순한 타이머 앱이 아니라 목표 구조와 시간 기록을 묶는 앱이라는 점을, AI에게 계속 알려줘야 했다.

6.2 첫 번째 실수—“전체를 한 번에 만들어줘”

첫 번째 프롬프트는 이랬다.

“Tauri 2 + SvelteKit 2 + Svelte 5로 Mandal-Art, GTD, Pomodoro, AI 채팅을 결합한 앱을 만들어줘. DuckDB 사용하고, AI 프로바이더 4개 연결하고, 스트리밍.”

Claude의 응답은 예상대로였다. 30줄짜리 폴더 구조가 나왔고, 300줄짜리 코드 블록이 `lib.rs`에 통째로 들어왔다. 절반은 컴파일일이 안 됐고, 나머지 절반은 내가 원하던 것이 아니었다. Mandal-Art 셀 ID는 임의로 정해졌고, Pomodoro 테이블은 태스크 테이블과 맞지 않았다.

그 결과물을 그대로 쓸 수는 없었다. 하지만 완전히 쓸모없지도 않았다. 내 설계에 어떤 허점이 있는지를 드러내 줬기 때문이다.

AI가 만들어낸 결과물에는 핵심 문제가 세 가지 있었다. 코드 자체보다 **어떤 결정이 잘못 내려졌는지를** 보는 것이 중요하다.

문제 1. 셀 ID로 UUID를 썼다

AI는 셀마다 무작위로 생긴 긴 식별자(UUID)를 붙였다. 그리고 “이 셀은 어떤 목표에 속하는가”를 적는 칸을 비워둘 수 있게 만들었다. 두 가지가 다 잘못이었다.

- **UUID는 위치 정보를 잃는다.** “중앙의 세 번째 영역, 다섯 번째 칸”같은 의미가 사라진다.
- **부모 목표가 비어 있어도 되니, 어디에도 속하지 않은 셀이 생길 수 있다.** $9 \times 9 = 81$ 칸이라는 구조 자체가 데이터로 강제되지 않는다.

내가 원하는 것은 사람이 읽을 수 있는 ID와, 반드시 어떤 목표에 속하는 셀이었다.

문제 2. Pomodoro 세션이 태스크와 연결되지 않았다

AI가 만든 세션 테이블에는 “어느 태스크의 세션인가”를 가리키는 연결이 빠져 있었다. 대신 자유롭게 적는 메모 문자열만 있었다.

이 한 가지 누락이 앱 전체 의도를 깨뜨린다. “Pomodoro 세션은 81칸 중 하나에 연결될 때만 의미가 있다”는 핵심 결정이 사라진 것이다. 세션 기록은 남지만, 어떤 목표의 어떤 칸에 쓴 시간인지 다시 묶을 수 없다.

문제 3. 폴더 구조는 그럴듯하지만 내용이 비어 있었다

`models.rs`, `migrations.rs`, `connection.rs` 같은 파일이 자동으로 만들어졌다. 이름은 그럴듯했지만 마이그레이션 파일은 “나중에 채우기” 주석으로 가득했고, DB 경로는 하드코딩이었다. 보기 좋은 골격 안에 알맹이가 없었다.

요청이 허술했고, AI는 그 빈틈을 스스로 채웠다. 문제는 AI가 채운 값들이 제품의 핵심 의도와 맞지 않았다는 점이다. 셀 ID 체계와 세션-태스크 연결은 앱의 중심 구조다. 초기에 잘못 잡으면 화면을 다 만든 뒤에도 계속 처음으로 돌아와야 한다.

주의: 요청이 모호하면 AI는 빈틈을 자기 판단으로 채워 그럴듯한 답을 만든다. AI는 무엇이 중요하지 모른다. 첫 번째 응답에는 잘못된 결정이 최소 세 개는 있다고 보면 된다.

교훈은 단순하다. **AI에게 “전체 앱”을 한 번에 맡기지 마라.** AI에게 넘기기 전에, 먼저 스스로 작업을 잘게 나눠야 한다. 나누지 않은 요청은 AI에게 자유를 주는 게 아니다. 제품의 핵심 결정을 AI가 추측하게 만드는 것이다.

6.3 크기 조절—AI가 한 번에 처리할 수 있는 단위

두 번째 세션은 이렇게 시작했다.

“앱 계획이 끝났다. 지금부터 정확히 한 가지씩 물어볼 것이다. 먼저 전체 구조를 들어보고, 묻고 싶은 것 최대 5가지를 던져봐. 코드는 쓰지 마.”

이 한 줄로 Claude의 응답 방식이 바뀌었다. Claude는 코드 대신 질문을 내놨다. 질문은 다섯 가지였다.

1. Mandal-Art 셀 ID의 체계는? (예: 1-3-5 vs `center.sub3.cell5` vs UUID)
2. Pomodoro 세션이 끝나면, 어떤 태스크에 붙이나—시작할 때 고정, 아니면 끝날 때 선택?
3. AI 채팅 히스토리는 태스크별, 셀별, 아니면 앱 전체?
4. 오프라인을 지원하나? 그렇다면 AI 호출은 어떻게 동작하나?
5. 설정(API 키 등)은 어디에 저장하나—keyring, 일반 파일, 아니면 DuckDB?

다섯 가지 중 세 가지는 아직 결정하지 않은 것들이었다. AI가 내 빈틈을 먼저 찾아낸 것이다. 첫 번째 세션에서 AI가 혼자 채워버린 부분이 바로 이것들이었다.

Claude가 먼저 질문하지 않을 때는 직접 유도한다. “코드는 쓰지 마. 이 설계에서 내가 아직 결정하지 않은 것 3가지를 찾아줘.”이 한 문장으로 Claude가 코드를 쓰는 대신 설계의 빈틈을 찾기 시작한다.

셀 ID 체계—두 번의 반문으로 확정

첫 번째로 좁힌 것은 셀 ID 체계였다. 나는 “셀 ID는 {center_idx}-{sub_idx}-{cell_idx}를 따른다. 문자열. 9×9 = 81개.”라고 답했다.

Claude가 다시 물었다: “중앙 자체(center)는 어떻게? 0-0-0처럼 가상 셀인가, 아니면 별도 테이블인가?”

내가 놓친 지점이었다. 중앙 목표는 셀이 아니지만 무의식적으로 셀처럼 다루고 있었다. “중앙은 별도 goal 테이블이 된다. 셀은 goal_id FK로 참조한다.”

Claude가 또 물었다: “목표가 별도라면, 활성 목표가 여러 개일 때는? 셀이 정확히 하나의 목표에 속하나, 아니면 여러 목표에 걸쳐 사용될 수 있나?”

한 번 답했더니 새 질문이 또 나왔다. 이것이 반복되는 패턴이다. 내가 답할 때마다 새 질문이 2~3개 나온다. AI가 계속해서 설계의 빈틈을 찾는다.

내 답: “한 번에 하나의 목표만 활성화. 셀은 goal_id FK로 정확히 하나의 목표에 속한다.”

두 번 주고받으니 셀 스키마가 확정됐다. 첫 번째 세션에서 Claude가 혼자 쓴 300줄보다 실질적인 결과였다.

Pomodoro 세션—분석 시나리오까지 따라간다

다음으로 Pomodoro 세션이 어느 태스크에 붙는지를 정했다. 나는 “Pomodoro는 시작할 때 태스크를 선택한다.”고 답했다.

Claude가 다시 물었다: “태스크 없이 25분 세션을 실행하면—예를 들어 이메일 처리?”

타당한 지적이었다. “그러면 태스크는 선택적. task_id는 nullable이고, note 컬럼이 대신한다.”

Claude가 또 잡아냈다:

“task_id가 nullable이면 분석은 어떻게 처리하나? '이번 주에 이메일에 3시간'과 '이번 주에 추적되지 않은 작업에 3시간'을 보고 싶은가?”

분석 사용 케이스는 생각 못 했다.

내 답: “네, notes는 검색 가능해야 한다. category enum 추가: ‘task’, ‘email’, ‘meeting’, ‘other’.”

그 결정 하나가 나중에 세션 히스토리 화면 전체 구조를 결정했다. AI가 되물었기 때문에 나중에 해야 할 결정을 지금 미리 할 수 있었다.

AI 채팅—선택지를 제시받기

세 번째 쟁점은 AI 채팅의 범위였다. 처음에 “AI 히스토리는 앱 전체로 평평하게”라고 답했다.

Claude는 그대로 받아들이지 않고 시나리오를 던졌다:

“셀‘2-3-4’에서’이 문제를 어떻게 해결하지?’라고 물었다고 하자. 나중에 셀‘5-1-2’에 있으면서 히스토리에서 같은 질문을 본다. 이제 컨텍스트가 틀렸다.

옵션: A) 셀별 히스토리 (셀당 1개 채팅 스레드 = 81개 스레드) B) 목표별 히스토리 (목표당 1개 채팅 스레드) C) 컨텍스트 태그가 있는 앱 전체 히스토리 (#cell:2-3-4, #task:write-tests) D) 하이브리드: 앱 전체지만 컨텍스트 필터 포함”

Claude가 내가 생각하지 못한 선택지를 제시했다. 옵션 C가 답이었다. 평평한 저장, 풍부한 태깅, 필터 가능한 뷰.

이 루프 하나로 채팅 데이터베이스 스키마 전체가 정의됐다. 세 개의 테이블: messages, contexts, 그리고 접합 테이블 message_contexts.

주의: 처음 떠오른 데이터 구조는 대부분 틀린다. AI는 빠뜨린 사용 시나리오를 찾아낸다. “그러면 어떻게?”를 최소 3번 파고든 다음에 스키마를 확정해라.

6.4 AI의 반문을 설계 도구로 활용하기

앞서 살펴본 세 루프에는 공통점이 있다. 내가 답하면 Claude가 다시 묻고, 그 질문이 설계의 빈 곳을 드러낸다. 이 흐름을 의도적으로 만들 수 있다.

내가 쓰는 템플릿은 이렇다.

“이 설계의 구멍을 감사해줘. 규칙을 따라: 1. 코드를 쓰지 마. 2. 내가 결정을 회피하고 있는 3곳을 찾아. 3. 각각에 대해 2~3개의 대안을 제시해.”

이 템플릿을 쓰면 Claude가 대안 생성기 역할을 한다. 내가 미처 몰랐던 선택지를 나열해 주면, 나는 하나를 고르고 왜 골랐는지 노트에 남긴다. 그 노트가 아키텍처 문서가 된다.

실제로 어떻게 작동하는지 보자. Claude에게 이 설계를 넘겼다.

“Mandai는 5개의 화면을 가진다: Today, Archive, Goals, Settings, Chat. Today 화면은 활성 목표의 81칸을 9×9 그리드로 보여준다. 사용자가 칸을 클릭하면 그 태스크들을 볼 수 있고, 태스크를 클릭하면 Pomodoro를 시작한다.”

감사 결과, 결정하지 않은 부분 3곳이 나왔다.

공백 1: 내비게이션 상태 - 옵션 A: 브라우저 스타일 라우팅 (/today, /archive, 등) - 옵션 B: 탭 기반 내비게이션 (SvelteKit stores) - 옵션 C: 모달 오버레이 (메인 화면 하나, 나머지는 팝업)

공백 2: 셀 상호작용 모델 - 옵션 A: 단일 클릭으로 열기, 더블 클릭으로 편집 - 옵션 B: 모든 액션에 오른쪽 클릭 컨텍스트 메뉴 - 옵션 C: 호버 시 액션 버튼 표시

공백 3: 타이머 상태 영속성 - 옵션 A: 앱이 닫히면 타이머 초기화 (세션 전용) - 옵션 B: 재시작해도 유지되는 타이머 (DB에 저장) - 옵션 C: 앱 닫기 전 타이머가 경고 표시 (사용자 선택)

세 가지 모두 결정하지 않은 상태였다. 설계처럼 보였던 것이 사실은 암묵적 가정들로 가득 찬 초안이었다.

내 결정: 1. **내비게이션:** 브라우저 스타일 라우팅. 뒤로 가기 버튼이 작동하고, 딥링크가 되고, 앱다운 느낌이 난다. 2. **셀 상호작용:** 단일 클릭으로 열기, 오른쪽 클릭으로 컨텍스트 메뉴. 처음 쓰는 사람도 바로 찾고, 파워 유저에겐 효율적이다. 3. **타이머 복원:** 앱을 다시 열면 진행 중이던 세션을 이어갈 수 있게 한다. 단, 세션 중에 창을 닫으려 하면 먼저 경고한다. 실수로 앱을 닫았다는 이유만으로 25분 집중 기록이 사라지면 안 된다.

이 감사를 Mandai에 열 번쯤 적용했다. 그 결과로 나온 결정 목록은 대략 이렇다.

- 셀 ID: {center_idx}-{sub_idx}-{cell_idx} 문자열
- goal 테이블은 cell 테이블과 별도
- Pomodoro task_id는 nullable, category enum 포함
- AI 히스토리는 컨텍스트 태깅이 있는 앱 전체 (#cell, #task, #goal)
- 내비게이션: SvelteKit 라우팅 (/today, /goals, 등)
- 셀 클릭: 단일로 열기, 오른쪽 클릭 컨텍스트 메뉴
- 타이머 복원: DB 기반, 앱을 다시 열면 진행 중이던 세션을 이어갈 수 있음
- 오프라인: AI 호출 실패 시 로컬 초안에 저장, 재연결 시 재시도 제공
- API 키: OS 키링 (tauri-plugin-store 아님)
- 마이그레이션: DuckDB의 _migrations 테이블, 앱 시작 시 확인

이 목록을 Cursor용 `.cursor/rules/mandai.mdc`와 Claude Code용 `CLAUDE.md`에 함께 넣었다. 한 도구의 규칙 파일을 다른 도구가 자동으로 읽는 것은 아니다. 이후 위반이 크게 줄었지만, 규칙 파일은 강제 장치가 아니므로 결과 검증은 계속했다.

감사 한 번에 주요 설계 영역당 약 10분이 걸린다. 대신 나중에 몇 시간을 아껴준다. 세션 2에서 발견한 설계 구멍이 세션 20에서 버그가 되지 않는다.

6.5 세 계층으로 구성하기: 화면, 데이터, 명령

AI에게 기능을 요청할 때는 세 계층 템플릿을 쓴다.

1. **화면** —사용자가 보고 클릭하는 것.
2. **데이터** —DB 테이블과 TypeScript 타입.
3. **명령** —화면에서 데이터로 가는 Rust 함수 (`#[tauri::command]`).

세 가지를 한꺼번에 요청하면 AI가 중심을 못 잡는다. 그래서 순서가 중요하다. **먼저 화면을 확정하고, 그 화면에서 필요한 데이터를 뽑아내고, 마지막으로 그 데이터를 다루는 명령을 열거한다.**

Today 화면에서 이 패턴이 어떻게 적용됐는지 보겠다.

계층 1: 화면 정의

레이아웃은 코드 대신 ASCII 목업으로 그렸다. 그림 도구가 없어도 글로 그릴 수 있고, AI가 헛갈리지 않는다. 아래는 Claude에게 그대로 보낸 목업이다.

```
+-----+
| MANDAI                                     [타이머] 18:42 |
+-----+
| 9x9 그리드 | 선택된 셀의 태스크 |
|             |                 |
| [완료] [대기] [진행] | [ ] 태스크 1    0/3 |
| [대기] [완료] [대기] | [ ] 태스크 2    2/3 |
|             | [완료] 태스크 3    3/3 |
|             | [+ 새 태스크] |
+-----+
```

상호작용 규칙은 텍스트로 같이 보낸다.

- 칸 클릭 -> 아래에 태스크 표시

- 태스크 클릭 -> Pomodoro 시작
- 세션 중 오른쪽 상단에 타이머 표시
- 진행 바로 각 칸의 태스크 완료 상태 표시

이 목록을 본 Claude가 두 가지를 확인했다.

“1. 선택된 칸이 없을 때 무엇을 보여주나—81개 전체 태스크, 아니면 빈 화면? 2. 태스크 진행 바는 어떻게 업데이트되나—세션 중 실시간, 아니면 세션 종료 후?”

내가 결정하지 않은 부분이였다. 답했다: “선택된 칸 없음 = 빈 태스크 목록, '칸을 클릭해서 태스크를 보세요'라는 힌트 텍스트. 진행 바는 세션 완료 후에만 업데이트.”

계층 2: 데이터 요구사항

화면이 확정되자 Claude가 필요한 데이터를 뽑아냈다:

“이 화면에 필요한 것: - 활성 목표: `goal_id`, `title` - 그 목표의 81개 셀 전부: `cell_id`, `title`, `task_progress` (완료/전체) - 선택된 셀의 태스크: `task_id`, `title`, `completed`, `cell_id` - 활성 세션: `session_id`, `task_id`, `started_at`, `target_minutes` - 세션 카운트: 오늘 카운트, 이번 주 카운트”

한 가지를 바꿨다: “`task_progress`를 미리 계산하지 마. DB는 원시 카운트만 반환: `completed_tasks`, `total_tasks`. 비율은 프론트엔드가 계산한다.” DB가 표시 형식까지 결정하면 화면이 바뀔 때마다 쿼리를 다시 써야 하기 때문이다.

Claude가 후속 질문을 던졌다: “태스크 카운트에 완료된 태스크도 포함하나, 아니면 활성인 것만? '이번 주'의 정의는 뭔가—월요일 시작 아니면 롤링 7일?”

내가 빠뜨린 엣지 케이스였다. “모든 태스크가 진행에 포함, 완료됐든 아니든. '이번 주'는 사용자 시간대의 월요일 00:00부터 일요일 23:59까지.”

계층 3: 명령 열거

화면과 데이터가 정해지자 Tauri 명령 목록도 자연스럽게 정리됐다. 화면에 필요한 데이터 종류만큼 명령이 만들어진다.

Claude가 제안한 명령은 여섯 개였다. 이름만 봐도 무엇을 가져오는 함수인지 한 줄에 드러난다.

명령	반환	용도
<code>get_active_goal</code>	<code>Goal</code>	현재 활성 목표 하나
<code>get_goal_cells(goal_id)</code>	<code>Vec<CellWithProgress></code>	81칸의 텍스트와 진행 카운트
<code>get_cell_tasks(cell_id)</code>	<code>Vec<Task></code>	선택한 셀의 태스크
<code>get_active_session</code>	<code>Option<PomodoroSession></code>	진행 중인 세션이 있다면
<code>get_session_counts</code>	<code>SessionCounts</code>	오늘/이번 주 세션 수
<code>start_pomodoro(task_id, minutes)</code>	<code>String</code>	새 세션 시작

이 중 `get_session_counts`만 시그니처를 조정했다. 오늘/이번 주처럼 고정된 시간대만 받으면 나중에 “지난 30일”같은 범위를 추가하기 어렵기 때문이다. 시간 범위를 `enum`(여러 선택지를 하나로 묶은 타입)으로 받도록 바꿨다.

핵심 아이디어는 단순하다. 호출하는 쪽이 어떤 기간을 원하는지 값으로 알려주면, 명령은 그 기간의 세션 수를 세어 돌려준다. `enum` 안에 `Today`, `ThisWeek`, `Custom`(시작·끝 날짜를 직접 지정) 세 가지 선택지를 둔다. **enum에 새 선택지가 추가돼도 명령 시그니처는 그대로 둘 수 있다.** 나중에 “지난 30일”같은 화면이 늘어나도 명령은 손대지 않고, `enum`에 새 항목만 더하면 된다.

명령의 타입은 입력·출력·오류를 한 세트로 정한다. 예를 들어 `create_goal`은 `title` 문자열을 받고 `id·title·created_at`을 반환하며, 빈 제목과 DB 실패를 서로 다른 오류로 구분한다. `enum` 상태를 JSON으로 넘길 때는 “`running`”, “`paused`”, “`completed`”처럼 고정 문자열을 쓰고 Rust와 TypeScript 양쪽에서 같은 목록을 유지한다. 화면 표시 문구를 데이터 상태값으로 쓰면 번역 때 계약이 깨진다.

이게 패턴이다: **화면 -> 데이터 -> 명령**. 각 계층은 이전 계층에서 나온다. AI는 한 번에 한 계층씩 처리할 때 잘 작동한다. 세 계층을 한꺼번에 주면 결과가 뒤섞인다.

명령 6개는 곧 **Request -> Build -> Verify 루프 6번**이다. 명령당 최대 30분. 화면 하나 = 반나절.

주의: 계층 사이를 오가면 AI가 헛갈린다. 하나의 프롬프트에서 화면 레이아웃과 DB 스키마를 물으면, 받은 UI 프레임워크, 받은 SQL인 응답이 나온다. 대화당 한 계층만.

6.6 AI 청크 크기 법칙

이 과정을 정리하면서 세 가지 법칙을 세웠다.

법칙 1. 한 번에 화면 하나, 또는 명령 하나. 전체 앱 ≫ 기능 하나 ≫ 화면 하나 ≫ 명령 하나. 오른쪽으로 갈수록 AI에 넘기기 좋다.

법칙 2. 코드를 요청하기 전에 질문을 유도하라. 처음 5~10분은 Claude가 나에게 질문하게 두는 데 쓴다. 그러면 설계에서 빠진 부분이 드러난다. 그 시간이 나중에 디버깅 시간으로 두 배 돌아오는 것을 막아준다.

법칙 3. 내가 결정하고, 이유는 도구별 프로젝트 규칙 파일에 남긴다. AI는 대안을 잘 만들지만, 무엇을 쓸지는 결정하지 못한다. 내가 직접 고르고, 왜 골랐는지 문서에 써둔다. 그러면 다음 세션에서 Claude가 같은 문제로 흔들리지 않는다.

체크가 커질수록 누락과 엇갈린 가정이 늘어났다. 아래 수치는 여섯 프로젝트에서 사용한 모델과 당시 코드베이스를 기준으로 한 작업 기록이지, 모든 AI 도구의 성능표가 아니다.

- **전체 앱(약 1000줄, 3~4시간):** 이 프로젝트들에서는 누락이 많아 검토 비용이 컸다. 초안 탐색 외에는 쓰지 않았다.
- **기능 단위(예: “Pomodoro 전체”, 1~2시간):** 초기 탐색 정도에만 쓴다.
- **화면 하나(30~45분):** 표준 작업 단위. 가장 자주 쓴다.
- **명령 하나(5~15분):** 화면 수준에서 막힐 때 한 단계 더 쪼갬다.
- **함수 하나(2~5분):** 특정 로직을 디버깅할 때만.

실제 작업에서는 다음 기준으로 자른다.

체크	포함해도 되는 것	포함하면 큰 것
화면 하나	컴포넌트 배치, 상태 표시, 기존 명령 호출	새 DB 스키마와 새 API 설계까지 함께 요청
명령 하나	입력 검증, DB 쿼리 하나, 에러 타입	여러 테이블 마이그레이션과 UI 연결
함수 하나	계산 로직, 포맷 변환, 엡지 케이스 테스트	파일 구조 재배치나 공통 상태 변경

판단이 어려울 때는 이렇게 자문한다. **이 체크를 한 문장으로 설명할 수 있는가. 독립적으로 테스트할 수 있는가. 커피 한 잔 후에도 요구사항을 기억할 수 있는가. 결정 하나만 내리는가.** 어느 하나라도 답이 “아니오”라면 체크가 너무 크다.

반대로 AI가 같은 질문을 체크마다 반복하거나, 두 체크가 로직의 대부분을 공유한다면 체크가 너무 작은 것이다. 합쳐서 한 번에 다룬다.

최적의 청크 크기는 **한 번에 동작하는 코드가 나오는 가장 큰 단위**다. 도메인과 모델에 따라 달라지지만, 대부분 생각보다 작다.

6.7 빌드 중간에 계획이 바뀔 때

6.2~6.6절의 계획 과정은 설계가 처음부터 끝까지 안정적이라고 가정한다. 하지만 현실에서는 실제로 만들다가 새로운 요구사항을 발견하는 경우가 많다. AI도 예상하지 못한 엣지 케이스가 나온다. 처음 계획이 틀어지는 건 피할 수 없다.

Mandai를 개발하는 동안 이런 일이 두 번 있었다. 개발 흐름을 유지하면서 설계를 어떻게 바꿨는지 순서대로 보여주겠다.

피벗 1: 셀 ID 충돌 문제

빌드 3주째에 셀 ID 체계에서 문제가 생겼다. 원래 계획에서 셀 ID는 {center_idx}-{sub_idx}-{cell_idx} 형식이고, 각 인덱스 값은 1~9였다. (최종 구현에서는 0~8 기반으로 바뀐다—7장 스키마 참고)

문제: 목표 전환 기능을 구현하다 보니, 사용자가 1-3-5 같은 셀이 있는 목표를 여러 개 만들 수 있다는 걸 알게 됐다. 셀 ID 값이 목표마다 달라지지 않으니, “목표 A의 셀 1-3-5”에 있는 태스크와 “목표 B의 셀 1-3-5”에 있는 태스크가 섞이는 문제가 생겼다.

스키마 전체를 다시 쓸 수도 있었지만, 그러면 DB 명령 15개를 처음부터 다시 해야 했다. 대신 AI에게 선택지를 물어봤다.

“설계 문제가 있다. ‘1-3-5’같은 셀 ID들이 목표 간에 충돌한다. 기존 태스크와 세션 테이블을 변경하지 않고 고유한 셀 참조가 필요하다. 선택지는 뭔가?”

Claude가 제시한 세 가지:

- A) **goal_id로 접두사:** goal_uuid_1-3-5 (고유하지만 보기 안 좋음)
- B) **복합 키:** cell_id를 '1-3-5'로 유지하되, 항상 (goal_id, cell_id) 쌍으로 쿼리
- C) **계층형 ID:** {goal_idx}-{center_idx}-{sub_idx}-{cell_idx}

옵션 B는 테이블 구조를 건드리지 않고 쿼리 조건만 추가하면 됐다. 이 결정을 다음 작업이 어기지 않도록 `.cursor/rules/mandai.mdc`에 네 줄을 추가했다.

- 셀 ID는 문자열 형식 '1-3-5'를 그대로 유지한다.

- 셀 ID는 한 목표 안에서만 고유하다.
- 모든 셀 쿼리는 `goal_id` 조건을 함께 써야 한다.
- 인덱스는 (`goal_id`, `cell_id`) 복합으로 잡는다.

이 네 줄이 앞으로의 모든 셀 쿼리를 묶는다. 새 화면을 만들 때 Claude가 `WHERE cell_id = ?`만 쓰면 규칙을 어기는 코드가 되고, 룰 파일이 이를 막아준다.

마이그레이션이 이를 대신 2시간 만에 끝냈다. 변경 범위를 최소화한 덕분이다.

피벗 2: 실시간 타이머 문제

원래 계획에서 Pomodoro 타이머는 브라우저 안에서만 실행된다. 앱을 닫으면 세션도 함께 사라진다.

문제: 사용자들이 세션 도중에 실수로 앱을 자주 닫았다. 25분 세션이 `Cmd+W` 한 번에 날아갔다. 생산성 앱에서는 받아들이기 어려운 동작이다.

보통은 앱이 종료돼도 남아 있는 백그라운드 서비스로 해결한다. 하지만 Tauri 2는 모든 플랫폼에서 백그라운드 서비스를 동일하게 지원하지 않는다.

여기서 요구사항을 “앱을 닫아도 타이머가 유지돼야 한다”라고만 쓰면 설계 방향이 흐릿하다. 더 정확한 표현은 “앱을 다시 열었을 때 진행 중이던 세션을 복원한다”에 가깝다. 핵심은 타이머가 독립적으로 계속 흐르는 게 아니라, 세션의 시작 시각을 저장해 두었다가 다음 실행 때 남은 시간을 계산하는 것이다.

Claude는 DB를 단순 저장소가 아니라 **상태 복원 수단**으로 쓰는 패턴을 제안했다. 핵심 흐름은 다음과 같다.

1. 세션 시작 시 `started_at` 타임스탬프와 함께 세션 레코드를 만든다.
2. 30초마다 `last_heartbeat`를 업데이트한다.
3. 앱 재시작 시 `ended_at` IS NULL인 세션이 있는지 확인한다.
4. 있다면 다이얼로그: “진행 중인 세션이 있었습니다. 계속 (X분 남음) / 완료로 표시 / 삭제”
5. “계속”을 고르면 `now - started_at`으로 남은 시간을 계산해 타이머를 재개한다.

이 패턴이 동작하는 이유는 단순하다. 매초 남은 시간을 저장하는 대신 **시작 시각 한 번만** 저장하기 때문이다. 앱이 꺼져 있던 시간도 `now - started_at` 한 번의 계산으로 메워진다. 백그라운드 서비스 없이도 세션의 연속성이 보존된다.

구현에는 하루 저녁 정도가 걸렸다. “진행 중이던 세션을 이어갈까요?”라고 묻는 복원 다이얼로그는 사용자 반응이 좋았다. 기능이 하나 늘어난 것 이상으로, 실수로 창을 닫아도 작업 흐름이 끊기지 않는다는 신뢰를 줬다.

피벗이 너무 잦아질 때

한 프로젝트에서 피벗 두 번은 정상이다. 다섯 번이면 원래 설계가 근본적으로 잘못됐다는 신호다. 둘을 구분하는 기준은 다음과 같다.

정상적인 피벗 (계속 빌드): - 문제가 하나의 제약에서 비롯된다 (ID 충돌, 영속성 요구사항 등) - 영향 범위가 좁고 수정 방향이 명확하다 - AI가 구체적인 해결책을 2~3개 제안한다 - 수정이 전체 설계를 오히려 개선한다

문제가 있는 피벗 (계획 재시작): - 여러 문제가 동시에 얽혀 나온다 - AI 응답에 해결책보다 되묻는 질문이 많다 - “간단한” 수정이 DB 테이블 5개 이상을 건드린다 - 같은 핵심 개념을 반복해서 설명하게 된다

문제가 있는 피벗 신호가 보이면 **6.4절 (설계 감사)로 돌아가라**. 현재 상태에서 구멍 찾기 템플릿을 다시 실행한다. 피벗이 반복된다는 건 보통 더 깊은 아키텍처 문제가 있다는 뜻이다.

마무리 박스: 이 장에서 반드시 기억할 것

이 장에서 반드시 기억할 것은 단순하다. **AI에게 한 번에 너무 큰 일을 맡기지 마라. 코드를 요청하기 전에 질문부터 받아라. 결정은 내가 하고, 그 이유는 문서에 남겨라.**

핵심: **AI를 쓰면 피벗 비용이 줄어든다. 없어지진 않는다.** 일찍, 자주 피벗해라—단, 설계 자체가 잘못됐을 때는 반드시 알아차려야 한다.

그 민첩함은 적절한 청크 크기에서 나온다. AI는 어떤 크기의 청크든 빌드할 수 있지만, 청크 크기가 결과물의 품질을 결정한다. 너무 크면 없는 제약을 만들어낸다. 너무 작으면 맥락이 사라진다.

배운 것:

- **AI가 잘못 추측할 때 설계의 빈틈이 보인다.** “전부 만들어”프롬프트에서 Claude가 임의로 채운 부분이 정확히 내가 명시하지 않은 요구사항을 가리킨다.
- **초기 단계에서는 코드보다 질문이 더 가치 있다.** “나한테 5가지 질문하고, 코드는 쓰지 마”프롬프트 하나로 Claude를 코드 생성기에서 설계 완성도 검사기로 바꿀 수 있다.
- **AI의 반문이 옛지 케이스를 드러낸다.** AI가 응답할 때마다 새 질문이 2~3개 생긴다. 이건 버그가 아니라 내가 미처 명시하지 않은 시나리오를 짚어주는 신호다.
- **아키텍처 감사 템플릿이 미뤄둔 결정을 찾아낸다.** “내가 결정을 미뤄둔 곳 3곳을 찾아”프롬프트를 쓰면 AI가 해당 지점을 짚고 선택지를 늘어놓는다. 고르는 건 내 몫이다.

- **계층을 나누면 AI의 응답이 산만해지지 않는다.** 화면 -> 데이터 -> 명령을 별도 대화로 분리해서 AI가 한 번에 한 가지에 집중하게 한다.
- **체크 크기 기준: 화면 하나 = 명령 4~6개 = 반나절.** 모멘텀을 유지하면서 방향도 바꿀 수 있는 단위다.
- **작은 피벗은 정상이다.** AI 지원 개발에서 방향 수정은 실패의 신호가 아니라 예상된 흐름이다. 단, 같은 지점에서 작은 피벗이 반복된다면 표면 아래에 구조 문제가 있다는 신호다.
- **결정을 문서화하면 흔들림을 줄일 수 있다.** 설계 선택과 그 이유를 `.cursor/rules/mandai.mdc`에 남긴다. 새 세션에서도 같은 맥락을 다시 제공할 수 있지만, AI가 실제로 따랐는지는 검증해야 한다.

7장에서는 이 기준을 실제 DuckDB 스키마 설계에 적용한다. 큰 기능을 AI가 소화할 수 있는 청크로 나누고, AI의 반문을 설계 도구로 활용하고, 모멘텀을 잃지 않고 계획을 바꾸는 방법을 첫 번째 완전한 Request -> Build -> Verify 루프에서 확인한다.

7장. DB 스키마—첫 번째 Request -> Build -> Ver- ify 루프

데이터베이스 설계는 AI와 함께 개발할 때 가장 먼저 막히는 지점이다. AI가 SQL을 못 써서가 아니다. 간단한 CREATE TABLE은 AI가 첫 답변에서 바로 만들어 준다. 진짜 어려운 부분은 따로 있다. 첫 루프에서 정한 스키마(테이블 구조)가 이후 모든 기능의 기반이 된다는 점이다. 이 단계에서 테이블 간 관계를 잘못 잡으면, 나중에 화면, 명령 처리 코드, 마이그레이션(기존 데이터를 새 구조로 옮기는 작업), 분석 쿼리까지 한꺼번에 고쳐야 한다.

Mandai의 스키마는 세 번 수정한 끝에 안정됐다. 처음 버전이 명백히 틀린 것은 아니었다. 컴파일도 됐고, 실행도 됐다. 하지만 기능을 세 개 더 추가한 뒤에야 어디가 불편한지 드러났다. 이 장에서 중요한 것은 최종 스키마뿐만이 아니라, 어떤 요청을 AI에게 보냈고 검토하면서 무엇을 발견했는지의 과정이다. 독자가 자신의 프로젝트에서 같은 루프를 직접 적용할 수 있게 하기 위해서다.

이 장이 왜 “루프 이야기”인지 먼저 보자. 세 번의 루프가 어디서 끝나고 어디서 더 필요했는지부터 한눈에 보고 시작하면, 본문이 추상적으로 느껴질 때 이 표로 다시 돌아올 수 있다.

라운드	내 요청	Claude의 결과물	검증에서 발견한 문제
1	7개 테이블 스키마 초안	기본 CREATE 문	세션 종료 시간, 셀 ID 검증, 소프트 삭제 전략이 빠짐
2	세 가지 공백 채우기 + 인덱스	제약과 복합 인덱스 추가	_migrations에 유일성 보장 없음
3	Rust 마이그레이션 러너	한 번에 동작	없음

스키마 설계는 2~3번 루프, 정형 패턴(마이그레이션 러너, 쿼리 헬퍼, 타입 레이어)은 1번 루프에 끝난다—이 구분이 이 장의 핵심 메시지다.

첫 회독에서는 스키마 결정과 실패 처리 순서를 따라간다. migration은 버전 확인 -> 아직 적용하지 않은 SQL 실행 -> 성공한 버전 기록으로 읽으면 된다. JSON 값을 DB에 전달할 때는 null·숫자·문자열·boolean의 타입을 구분하고 허용하지 않는 값은 거부한다. 아래 설명과 예시는 이 계약을 이해하기 위한 것이며 상용 앱 전체를 복사해 실행하는 절차는 아니다.

이 장의 흐름

1. **왜 DuckDB인가** —분석 쿼리에 강한 데이터베이스를 고른 이유
2. **첫 번째 루프** —스키마 초안을 받고, 빠진 결정을 찾아낸다
3. **두 번째 루프** —세션 종료, 셀 ID 검증, 소프트 삭제(데이터를 지우지 않고 지운 표시만 남기는 방식)를 채워 넣는다
4. **세 번째 루프** —마이그레이션 시스템을 한 번에 완성한다
5. **검증** —AI가 만든 테스트로 제약이 실제로 작동하는지 확인한다
6. **연결** —Rust 명령과 TypeScript 타입 레이어로 프론트와 잇는다
7. **루프 패턴** —어떤 작업이 1번에 끝나고 어떤 작업이 2~3번 필요한지 분류한다

학습 목표

이 장을 끝내면 다음을 할 수 있다.

- DuckDB가 SQLite와 무엇이 다른지, AI 프롬프트에 왜 그 차이를 명시해야 하는지 설명한다.
- 스키마 설계를 한 번에 끝내려 하지 않고, 빠진 결정을 단계적으로 채우는 루프를 적용한다.
- CHECK 제약(잘못된 값이 들어가지 못하게 막는 규칙)과 외래 키, 인덱스를 쿼리 패턴에 맞게 설계한다.
- Tauri 명령과 TypeScript 타입 레이어를 연결해 타입 안전한 DB 접근을 만든다.

7.1 DuckDB를 선택한 이유

먼저 봐야 할 것—7개 테이블의 관계

컬럼 이름보다 관계를 먼저 보자. Mandai의 핵심 흐름은 다음과 같다.

```
goals(목표)
├── 1:N - cells(81칸)
│   ├── 1:N - tasks(태스크)
│   │   └── 1:N - sessions(Pomodoro)
```

```
chats - goal/cell 컨텍스트
settings - 앱 전체 설정
_migrations - 버전 추적
```

목표(goals) 하나에는 $9 \times 9 = 81$ 개의 셀(cells)이 달리고, 각 셀에는 오늘의 태스크(tasks)가 붙고, 태스크마다 Pomodoro 세션(sessions)이 쌓인다. AI 채팅(chats)은 여기에 컨텍스트로 붙는다. 이 관계도를 머릿속에 두면, 왜 외래 키와 인덱스가 필요한지, 왜 한 테이블만 보고 결정할 수 없는지 자연스럽게 이해된다.

왜 DuckDB인가

처음에는 SQLite로도 충분해 보인다. 단순한 데이터 저장만 한다면 SQLite가 더 친숙하고 자료도 많다. 하지만 Mandai는 시간이 지날수록 분석 질문을 던지게 된다. “지난 3개월 동안 가장 많이 집중한 셀은?” “요일별 집중 시간 중앙값은?” 같은 질문이다. 데이터를 저장하는 게 아니라 분석하는 일이다. DuckDB는 이 분석 쪽에 특화되어 있다.

DuckDB는 컬럼형 엔진이다. 일반 DB가 데이터를 한 행씩 저장한다면, DuckDB는 한 컬럼씩 저장한다. 덕분에 합계나 평균처럼 한 컬럼 전체를 훑는 집계 쿼리가 빠르다. SQL 문법은 SQLite와 크게 다르지 않아 새로 배울 것도 적다.

데이터베이스 장이 어렵게 느껴지는 이유는 따로 있다. 테이블을 어떻게 설계하느냐가 곧 앱의 정책이 되기 때문이다. 예를 들어 `task_id`를 비어 있어도 되는 값으로 둘지, 데이터를 지울 때 실제로 삭제할지 아니면 표시만 남길지—이런 선택이 화면, 분석 쿼리, 마이그레이션까지 영향을 준다. 단순한 취향 문제가 아니다.

DuckDB vs SQLite —AI가 자주 틀리는 부분

주의: 대부분의 AI 모델은 SQLite 문법을 기본값으로 쓴다. DuckDB 고유 기능을 명시하지 않으면 DuckDB의 성능을 살리지 못하는 평범한 SQL이 나온다.

Claude에게 “데이터베이스 스키마를 만들어 달라”고 처음 요청했을 때 SQLite에 가까운 답변이 나왔다. 분석에 유리한 타입이나 함수가 빠져 있었다.

영역	DuckDB	SQLite
타임스탬프	TIMESTAMP	DATETIME
큰 정수	BIGINT	INTEGER
분석 함수	percentile_cont(), stddev() 내장	대부분 직접 구현
배열	네이티브 지원	없음

프롬프트에 “DuckDB 문법을 사용하고, SQLite 패턴으로 작성하지 마라”라고 한 줄 추가하는 것만으로도 AI가 고르는 타입과 예제 쿼리가 달라진다. 이 차이를 체감하는 것이 이 장에서 가장 실용적인 수확 중 하나다.

7.2 첫 번째 루프—요청 초안 작성

스키마를 Claude에게 요청하기 전에 Tauri에서 DuckDB 연결부터 정해야 했다. 명령마다 새 연결을 열지, 아니면 앱 전체에서 하나의 연결을 공유할지에 따라 여러 처리가 달라지기 때문이다.

src-tauri/Cargo.toml에 의존성을 추가한다. bundled 피처를 사용하면 DuckDB가 앱에 함께 컴파일되어 사용자가 따로 설치할 필요가 없다.

```
[dependencies]
duckdb = { version = "1.5", features = ["bundled"] }
```

연결은 한 번 열고 공유한다

연결은 앱 전체에서 하나만 만들어 공유한다. 다음 코드는 앱이 시작될 때 DuckDB 연결을 단 한 번 만 들고, 전역 상태로 보관한 뒤 마이그레이션을 실행한다.

```
static DB: OnceLock<Mutex<Connection>> = OnceLock::new();

pub fn init_db() -> Result<(), String> {
    let mut conn = Connection::open_in_memory()
        .map_err(|e| format!("DB 연결 실패: {}", e))?;
    run_migrations(&mut conn)
        .map_err(|e| format!("마이그레이션 실패: {}", e))?;
    DB.set(Mutex::new(conn))
        .map_err(|_| "DB already initialized".to_string())?;
    Ok(())
}
```

여기서 OnceLock은 “앱 전체에서 딱 한 번만 초기화되는 값”을 만드는 Rust 타입이고, Mutex는 “한 번에 하나만 접근하게 한다”는 잠금이다. 이 예제는 단순성을 위해 단일 연결의 쿼리를 직렬화한다. DuckDB가 여러 연결을 절대 금지하는 것은 아니지만, 한 연결에서 동시에 여러 쿼리를 실행하는 구조와 파일 쓰기 동시성을 무심코 섞으면 관리가 어려워진다.

5장과 의 연결: 읽기·쓰기가 짧은 일반 데이터 구조에는 RwLock이 편리하고, 단일 연결처럼 한 번에 한 작업만 사용하게 할 자원에는 Mutex가 자연스럽다. 다만 DuckDB 쿼리는 동기 I/O이므로 비동기 Tauri 명령에서 오래 실행한다면 락을 잡은 채 실행기를 막지 말고 spawn_blocking이나 전용 DB 작업 스레드로 분리한다. 읽기 동시성이 실제로 필요해질 때만 연결 풀이나 읽기 전용 연결을 검토한다.

주의: 실제 배포 앱에서는 Connection::open_in_memory()를 쓰면 안 된다. 앱을 닫으면 데이터가 모두 사라진다. 실제 파일 경로를 지정하고, 첫 실행에서 파일이 없는 경우도 처리해야 한다.

스키마 요청 보내기

연결 방식을 정한 뒤 Claude에게 스키마를 요청했다. 이 프롬프트에는 테이블 목록만 나열하지 않고 **앱이 데이터를 어떻게 사용하는지**까지 함께 적었다. 테이블 이름만 던지면 AI는 그 테이블이 단순 저장 용인지 분석용인지 알 수 없어 “표준 CRUD”용 컬럼만 골라 준다. 사용 흐름을 적어 줘야 외래 키, 타임 스탬프, 인덱스 후보 같은 것을 미리 고려해 준다.

DuckDB 스키마를 작성해줘. Mandai 앱을 위한 것. 테이블 일곱 개.

- goals: 중앙 목표
- cells: 9×9 = 81칸, goal에 FK
- tasks: 오늘의 태스크, cell에 FK (선택적)
- sessions: Pomodoro 세션, task에 FK (선택적)
- chats: AI 채팅 메시지
- settings: 키-값
- _migrations: 마이그레이션 추적

명시적 외래 키. DuckDB 문법. CREATE TABLE만, INSERT 예제는 없음.

컨텍스트: 이것은 생산성 앱이다. 목표는 9×9 셀 그리드를 가진다 (Mandala 차트처럼).

사용자는 셀에 태스크를 추가하고, 태스크에 Pomodoro 세션을 실행한다.

진행 상황에 대해 AI와 채팅한다.

분석 쿼리는 시간 범위별로 세션을 집계한다.

집중 시간 비율도 계산해야 한다.

초안 검토—무엇이 빠졌나

Claude의 초안에서 goals와 cells 두 테이블은 외래 키, NOT NULL, 기본값까지 자연스럽게 들어 가 있었다. 잘된 부분이다. 하지만 검토하면서 결정이 빠진 곳 세 가지가 보였다.

1. **sessions에 start_at은 있지만 end_at이 없다.** 총 집중 시간을 계산할 방법이 없고, 타이머 가 도중에 종료되면 세션이 영원히 열린 상태로 남는다.
2. **cells.id에 생성 규칙이 없다.** 6장에서 {center_idx}-{sub_idx}-{cell_idx} 형식 (예: 0-1-4)으로 정했는데, 그 규칙이 스키마에 반영되지 않았다.
3. **소프트 삭제 전략이 일관되지 않다.** goals에는 deleted_at이 있지만 tasks에는 없다. 태스크를 삭제하면 연결된 세션은 어떻게 되는가?

세 가지 모두 6장에서 결정하지 않고 넘어간 것들이다. 내가 잊고 있었고 Claude도 지나쳤다. **검토하는 과정에서 놓쳤던 결정들이 드러난다. 첫 번째 루프의 역할이 바로 그것이다.**

7.3 두 번째 루프—구멍 메우기

6장에서 정한 규칙 다시 보기: 셀 ID는 {center}-{sub}-{cell} 형식이고 각 인덱스는 0~8 범위다. 예: 0-1-4는 중앙 영역 0, 그 안의 서브 영역 1, 그 안의 셀 4를 가리킨다. 이 규칙을 DB 제약으로 옮기는 것이 이 절의 핵심이다.

두 번째 루프에서는 초안을 처음부터 다시 만들지 않았다. 잘된 부분은 두고, 빠진 결정 세 가지만 구체적으로 지목해 피드백했다.

세 가지 수정이 필요해:

1. `sessions`에 `end\_at` TIMESTAMP NULL`과 `ended_reason` TEXT` 추가.
값: completed / skipped / interrupted.
`end\_at` NULL은 세션이 아직 실행 중임을 의미.
2. `cells.id`에 CHECK 제약 추가: 형식은 "0-1-4"처럼 `{center}-{sub}-{cell}`.
3. 소프트 삭제는 `deleted\_at` TIMESTAMP NULL`로 표준화. `tasks`와 `goals`에만.
세션은 이력 데이터이므로 남기고, 셀은 절대 삭제되지 않음 (구조적이므로).

세 테이블만 다시 써. 쿼리 패턴에 맞는 인덱스도 제안해.

“소프트 삭제 추가”라고만 하면 Claude가 스스로 방식을 골랐을 것이다. 정확한 접근과 이유를 함께 적어 줘야 그 지침대로 따른다.

수정된 핵심 테이블

수정본의 핵심은 sessions와 cells에 들어간 CHECK 제약이다. 잘못된 데이터가 애초에 들어가지 못하도록 DB가 직접 막는다.

```
CREATE TABLE cells (
  id TEXT PRIMARY KEY,
  goal_id INTEGER NOT NULL,
  center_idx INTEGER NOT NULL,
  sub_idx INTEGER NOT NULL,
  cell_idx INTEGER NOT NULL,
  FOREIGN KEY (goal_id) REFERENCES goals(id),
  CHECK (id = printf('%d-%d-%d', center_idx, sub_idx, cell_idx)),
```

```

CHECK (center_idx BETWEEN 0 AND 8),
CHECK (sub_idx BETWEEN 0 AND 8),
CHECK (cell_idx BETWEEN 0 AND 8)
);

CREATE TABLE sessions (
  id INTEGER PRIMARY KEY,
  task_id INTEGER NULL,
  start_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  end_at TIMESTAMP NULL,
  ended_reason TEXT NULL,
  target_minutes INTEGER NOT NULL DEFAULT 25,
  FOREIGN KEY (task_id) REFERENCES tasks(id),
  CHECK (
    ended_reason IS NULL OR
    ended_reason IN ('completed', 'skipped', 'interrupted')
  ),
  CHECK (end_at IS NULL OR end_at >= start_at),
  CHECK (target_minutes > 0)
);

```

중요한 줄: 마지막의 CHECK 제약들이다. 셀 인덱스가 0-8 범위를 벗어나지 못하게 하고, 세션 종료 시간이 시작 시간보다 빠를 수 없게 하고, 목표 시간은 항상 양수로 강제한다. 내가 요청하지 않은 안전장치를 Claude가 추가해 줬다.

인덱스—자주 실행할 쿼리부터 정한다

인덱스는 자주 실행할 쿼리를 먼저 정해두고 그에 맞춰 만든다. 다음은 Mandai에서 가장 자주 쓸 세 가지 쿼리에 맞춘 인덱스다.

```

CREATE INDEX idx_cells_goal ON cells(goal_id);
CREATE INDEX idx_sessions_task_start ON sessions(task_id, start_at
DESC);
CREATE INDEX idx_sessions_completed ON sessions(ended_reason,
start_at);

```

혼자 작업했다면 컬럼마다 따로 인덱스를 만들었을 것이다. Claude는 실제 쿼리 패턴을 먼저 생각한 뒤 복합 인덱스를 만들었다.

주의: 처음부터 인덱스를 많이 만들 필요는 없다. 인덱스가 늘어날수록 쓰기 속도가 느려지고 디스크 공간도 더 쓴다. DuckDB는 일부 인덱스 옵션(특정 조건만 대상으로 하는 partial index 등)을 지원하지 않으므로, 이런 필터링은 쿼리 쪽 WHERE 절로 처리한다.

두 번째 검토에서 한 가지 문제가 더 보였다. `_migrations` 테이블에 실행 순서 보장이 없다. 마이그레이션 러너가 두 번 실행될 때 중복 삽입을 막을 수 없다.

7.4 세 번째 루프—마이그레이션 시스템

세 번째 루프에서는 스키마 자체보다 그것을 실행하는 시스템을 만든다. **마이그레이션 러너란?** “어떤 SQL을 어떤 순서로, 그리고 한 번만 실행할지”관리하는 코드다. 앱이 업데이트되면서 스키마가 바뀔 때 기존 사용자의 DB를 자동으로 새 구조로 맞춰 준다.

요청은 단순했다. “DuckDB Connection을 받아 아직 적용되지 않은 마이그레이션만 순서대로 실행하고, 적용한 기록을 `_migrations`에 남기는 함수. 핵심 로직 10줄 이하.”

정상 흐름은 첫 시도에서 동작했다. 하지만 출간 전 실패 복구를 다시 검토하니 한 가지 구멍이 있었다. SQL 실행 뒤 이력 기록에 실패하면 다음 실행에서 같은 마이그레이션을 다시 시도할 수 있다. 그래서 **마이그레이션 SQL과 적용 이력 기록을 같은 트랜잭션으로 묶었다.**

```
pub fn run_migrations(conn: &mut Connection) -> duckdb::Result<()> {
    conn.execute_batch(include_str!("../migrations/000_meta.sql"))?;

    const MIGRATIONS: &[(&str, &str)] = &[
        ("001", include_str!("../migrations/001_init.sql")),
        ("002", include_str!("../migrations/002_add_indexes.sql")),
    ];

    for &(version, sql) in MIGRATIONS {
        let applied: i64 = conn.query_row(
            "SELECT COUNT(*) FROM _migrations WHERE version = ?",
            [version], |row| row.get(0))?;
        if applied == 0 {
            let tx = conn.transaction()?;
            tx.execute_batch(sql)?;
            tx.execute(
                "INSERT INTO _migrations(version, applied_at) \
                 VALUES (?, CURRENT_TIMESTAMP)",
```

```

        [version],
    )?;
    tx.commit()?;
}
}
Ok(())
}

```

중요한 두 줄: - `include_str!("...")`: SQL 파일을 컴파일 시점에 바이너리에 직접 포함시키는 매크로다. 실행 중에 파일을 따로 읽지 않아도 된다. - `if applied == 0`: 이미 적용된 버전은 건너뛴다. `_migrations.version`의 유일성 제약과 트랜잭션까지 함께 있어야 중복 적용과 반쪽 적용을 막을 수 있다.

이런 종류의 코드는 패턴이 정해져 있어서 Claude가 한 번에 잘 만든다. **AI에게 한 번에 맡기기 좋은 영역이다.** 첫 회독에서는 코드를 한 줄씩 분석하기보다, 이 함수가 “버전 확인 -> 미적용 SQL 실행 -> 기록 남기기” 세 단계라는 형식만 기억해도 충분하다.

주의: 이 방식은 마이그레이션을 추가할 때마다 재컴파일이 필요하다. 데스크톱 앱에서는 문제없지만, 서버 환경이라면 파일을 동적으로 불러오는 방식이 낫다.

`migrations/000_meta.sql`에는 `_migrations` 테이블 정의만 들어가고, `migrations/001_init.sql`에는 앞서 만든 모든 테이블이 들어간다.

지금부터 7.5~7.8은 **스키마 다음 단계**다. 테스트로 제약을 검증하고(7.5), Rust 명령으로 노출하고(7.6), 자주 쓰는 쿼리 패턴을 정리하고(7.7), 마지막으로 TypeScript 쪽에서 타입 안전하게 받는 얇은 헬퍼(7.8)를 만든다. 각각은 짧다.

7.5 스키마 테스트—AI로 품질 검증하기

쿼리 레이어로 넘어가기 전에 스키마가 제대로 작동하는지 확인했다. AI에게 “엡지 케이스와 제약 위반에 집중한 테스트 INSERT를 만들어 달라”고 요청했다. 먼저 외래 키 검사가 다른 오류를 가리지 않도록 테스트 fixture로 `goal_id = 1, task_id = 1`인 부모 행을 만든다. 핵심은 **오류가 나야 정상**인 다음 같은 케이스들이다.

```

-- 실패해야 함: 잘못된 셀 ID 형식
INSERT INTO cells (id, goal_id, center_idx, sub_idx, cell_idx, text)
VALUES ('invalid', 1, 0, 0, 0, '나쁜 ID');

-- 실패해야 함: 셀 인덱스 범위 초과
INSERT INTO cells (id, goal_id, center_idx, sub_idx, cell_idx, text)
VALUES ('9-0-0', 1, 9, 0, 0, '범위 초과');

-- 실패해야 함: 종료 시간이 시작 시간보다 빠름
INSERT INTO sessions (task_id, start_at, end_at) VALUES
(1, '2026-04-17 10:00:00', '2026-04-17 09:30:00');

```

세 쿼리 모두 CHECK constraint failed류의 에러가 터야 한다. 만약 그중 하나라도 성공한다면 해당 CHECK 제약이 제대로 걸리지 않은 것이다.

테스트에는 정상 생성·중복 생성·잘못된 참조·migration 재실행·중간 실패를 넣는다. 목표 삭제 정책을 설명용으로 정하면, 하위 셀이 있는 목표는 먼저 거부하고 사용자가 명시적으로 하위 항목까지 삭제할 때 한 트랜잭션으로 처리할 수 있다. 앱이 충돌한 세션은 시작 시 running 상태를 찾아“복구 필요”로 표시하고 완료 시간으로 임의 변환하지 않는다. 실제 정책은 요구사항으로 확정해야 하며 이 두 예는 판단 순서를 보여준다.

7.6 한 번에 완성된 명령—db_query_json

스키마가 확정되자마자 Rust 명령을 연결했다. 이 명령은 프론트엔드에서 보낸 SELECT 쿼리를 실행하고, 결과를 JSON 배열 문자열로 돌려준다. 시그니처는 다음과 같다.

```

#[tauri::command]
pub fn db_query_json(
    sql: String,
    params: Vec<serde_json::Value>,
) -> Result<String, String>

```

핵심 아이디어는 사용자가 보낸 SELECT를 서브쿼리로 감싼 뒤, DuckDB의 json_group_array(to_json(t))로 결과 전체를 JSON 배열 하나로 묶는 것이다. 한번의 호출로 모든 행이 직렬화되어 프론트엔드로 넘어간다. 결과가 없으면 집계값이 NULL일 수 있으므로 구현에서는 이를 []로 정규화한다.

JSON 파라미터 변환은 null->SQL NULL, boolean->BOOL, 정수->정수 타입, 유한한 실수->실수 타입, 문자열->TEXT 순서로 분기한다. 배열·객체는 해당 열이 JSON을 저장하도록 설계된 경우에만 직렬화하고 아니면 오류로 돌려준다. 큰 정수를 f64로 먼저 바꾸면 정밀도를 잃으므로 정수 분기를 먼저 검사한다. 값은 바인딩 파라미터로 넘기고 SQL 문자열에 이어 붙이지 않는다. 테이블·열 이름은 바인딩으로 보호되지 않으므로 허용 목록에서 선택한다.

쓰기 전용 명령 db_execute도 비슷한 시그니처를 가지며, query_row 대신 execute를 호출하고 영향받은 행 수를 반환한다.

주의: 범용 쿼리 명령은 강력한 만큼 위험하다. 읽기와 쓰기를 함수 두 개로 나누기만 해서는 보안 경계가 되지 않는다. 프론트엔드가 db_execute에 DELETE나 DROP을 보낼 수 있기 때문이다. 이 명령은 로컬 프로토타입과 내부 도구에만 한정하고, 프로덕션에서는 create_task, finish_session처럼 허용된 동작만 받는 도메인 명령과 파라미터 바인딩을 노출한다.

7.7 Mandai를 위한 DuckDB 쿼리 패턴

스키마와 명령을 갖춘 뒤 실제 쿼리를 만들어 갔다. 첫 회독에서는 DuckDB의 강점이 한 줄로 드러나는 다음 예제 하나만 봐도 충분하다.

```
SELECT
    date_trunc('month', start_at) as month,
    COUNT(*) as total_sessions,
    COUNT(*) FILTER (WHERE ended_reason = 'completed') as completed
FROM sessions
WHERE start_at >= '2026-01-01'
GROUP BY month
ORDER BY month;
```

중요한 줄: COUNT(*) FILTER (WHERE ...) 부분이다. 같은 결과를 SQLite로 내려면 CASE WHEN 문을 길게 써야 한다. DuckDB에서는 조건부 집계를 짧고 읽기 좋게 쓸 수 있다.

분석은 먼저 집계 단위를 정한다. 요일별 집중 시간 중앙값은 완료 세션에서 시간대가 맞는 요일과 duration을 구하고 요일로 묶어 percentile_cont(0.5)를 적용한다. 완료율은 완료 세션 수/시작 세션 수로 정의하되 분모가 0이면 계산 불가다. 날짜 경계를 UTC와 현지 시각 중 무엇으로 잡았는지 명시해야 다른 화면의 합계와 맞는다. 다음 장은 AI API 통합으로 넘어간다.

소프트 삭제와 조인의 주의점

여러 테이블을 조인할 때 한 가지만 기억하면 된다. 소프트 삭제를 쓰는 경우, 모든 분석 쿼리의 조인 조건이나 WHERE 절에 `deleted_at IS NULL`을 빠뜨리지 말아야 한다. 빠뜨리면 삭제된 데이터가 통계에 섞여 들어간다.

```
LEFT JOIN tasks t ON c.id = t.cell_id AND t.deleted_at IS NULL
```

7.8 TypeScript 타입 레이어—AI가 확실히 도움이 된 곳

DB 명령 위에 TypeScript 타입 레이어를 씌우는 작업에서 AI는 거의 손댈 것 없는 코드를 바로 내놓았다. 이 레이어는 프론트엔드에서 쿼리 결과를 타입 안전하게 다루도록 얇은 헬퍼 세 개를 제공한다.

```
import { invoke } from "@tauri-apps/api/core";

export async function query<T>({
  sql: string,
  params: unknown[] = [],
}): Promise<T[]> {
  const json = await invoke<string>("db_query_json", { sql, params });
  return JSON.parse(json) as T[];
}

export async function one<T>({
  sql: string,
  params: unknown[] = [],
}): Promise<T | null> {
  const rows = await query<T>(sql, params);
  return rows[0] ?? null;
}

export async function execute(
  sql: string,
  params: unknown[] = [],
): Promise<number> {
  return invoke<number>("db_execute", { sql, params });
}
```

세 함수의 역할은 단순하다. - `query<T>` —여러 행을 배열로 반환 - `one<T>` —한 행만 반환(없으면 null) - `execute` —쓰기 전용, 영향받은 행 수 반환

제네릭 `<T>`를 받기 때문에 `query<Goal>(...)`처럼 호출하면 자동으로 `Promise<Goal[]>`가 반환된다. 호출부 코드도 단순해진다.

```
interface Goal { id: number; title: string; created_at: string; }

const goals = await query<Goal>("SELECT * FROM goals WHERE deleted_at IS NULL");
const activeGoal = await one<Goal>("SELECT * FROM goals WHERE id = ?", [goalId]);
```

이 TypeScript 레이어는 루프가 한 번도 없었다. 요청 하나로 끝났다. Tauri의 `invoke`, 제네릭, JSON 파싱을 조합하는 정해진 패턴이기 때문이다. 판단이 필요 없는 작업에서 AI는 빠르고 정확하다. 반대로 7.2~7.4절처럼 무언가를 결정해야 하는 작업에서는 AI 혼자 답을 낼 수 없다. 그 결정은 개발자가 해야 한다.

주의: 이 레이어는 실행 중에 데이터를 검증하지 않는다. SQL이 예상과 다른 컬럼을 반환해도 오류가 나지 않고, 런타임에서 `undefined` 속성이 나타난다. 프로덕션에서는 Zod 같은 런타임 검증 라이브러리를 함께 쓰는 것이 안전하다.

7.9 이 장의 루프 요약

도입부에서 본 루프 표를 다시 보면, 어떤 작업이 한 번에 끝나고 어떤 작업이 여러 번 루프가 필요했는지 패턴이 분명해진다. 한 번에 완성된 것: `run_migrations()`, `db_query_json`, TypeScript 헬퍼, 테스트 케이스. 2~3번 루프가 필요했던 것: 스키마 설계, 제약 명세, 인덱스 전략.

Mandai 전체에서 이 패턴이 반복됐다. 코드에 정해진 형식이 있고 경계가 명확하면 AI가 한 번에 끝낸다. 설계 결정이 섞여 있으면 2~3번 루프가 필요하다. 이 구분을 알면 프롬프트를 보내기 전에 몇 번 대화가 필요할지 미리 짐작할 수 있다.

DB 작업 패턴 분류

- **스키마 설계** = 2~3번 루프 (비즈니스 결정이 많음)
- **쿼리 함수** = 1번 루프 (확립된 패턴)

- **타입 레이어** = 1번 루프 (정해진 형식)
- **테스트 생성** = 1번 루프 (AI가 엡지 케이스를 잘 잡아냄)
- **마이그레이션 시스템** = 1번 루프 (Rust 패턴이 잡혀 있음)

7.10 Rust 측 명령 레지스트리

마지막으로 `lib.rs`의 `.invoke_handler(tauri::generate_handler! [...])` 목록에 프론트엔드가 사용할 두 명령(`db_query_json`, `db_execute`)을 추가하고, `setup` 블록에서는 `init_db()`만 호출한다. 앞의 코드처럼 `init_db()`가 내부에서 마이그레이션을 실행하므로 `run_migrations()`를 다시 호출하면 안 된다. 마이그레이션은 앱 시작 책임이며 프론트엔드 명령으로 노출하지 않는다.

이 책의 작은 앱에서는 중앙 등록 목록 하나가 가장 단순하다. 명령이 늘어나면 모듈별 등록 함수를 두거나 별도 빌더 패턴으로 나눌 수도 있다. 어느 구조를 택하든 최종적으로 호출할 명령이 `handler`에 등록되어야 한다. 정확한 기본 골격은 5장의 등록 코드를 그대로 따른다.

주의: `init_db()`나 `run_migrations()`가 실패하면 앱 시작 시 패닉한다. 프로덕션에서는 데이터베이스 초기화 오류를 적절히 처리해야 한다.

개발 중 디버깅 팁: 패닉이 나면 메시지가 한 줄로 스쳐 지나가기 쉽다. 터미널을 띄운 채로 `pnpm tauri dev`를 실행하면 첫 실행 실패 메시지를 놓치지 않고 볼 수 있다. 메시지가 보이지 않으면 `init_db()`의 `map_err` 안에 `eprintln!`을 임시로 넣어 어느 단계에서 막혔는지 좁히면 된다.

다음 장에서는 OpenAI, Claude, Gemini 세 AI 프로바이더와 스트리밍 연동을 구현한다. 스키마보다 복잡하다. SSE 형식, 오류 형식, 타임아웃이 프로바이더마다 다르고, 일부 구현은 다섯 번 이상 루프를 돌았다.

마무리 박스: 이 장에서 반드시 기억할 것

이 장을 한 번 읽고 나서 다음 다섯 가지만 기억하면 된다.

1. **스키마 설계는 한 번에 끝나지 않는다.** 첫 번째 초안에서는 빠진 결정이 드러나는 게 정상이다. 그 빈틈을 두 번째, 세 번째 루프에서 채운다.

2. **AI는 SQLite를 기본값으로 쓴다.** DuckDB를 쓴다면 프롬프트에 명시하라. 그래야 분석 함수, 풍부한 타입, JSON 함수 같은 강점을 살린다.
3. **CHECK 제약은 잘못된 데이터가 들어가지 못하게 DB가 직접 막는 규칙이다.** 셀 인덱스 범위, 선택 종료 시간 같은 기본 규칙은 처음부터 걸어 뒤라.
4. **연결 정책은 하나로 통일한다.** 이 장의 작은 앱은 OnceLock<Mutex<Connection>>으로 단일 연결을 직렬화한다. 동시성이 필요해지기 전부터 연결을 늘리지 말고, 필요해지면 측정 후 작업 스레드·읽기 연결·연결 풀 중 하나를 선택한다.
5. **AI가 한 번에 끝내는 작업과 루프가 필요한 작업을 구분하라.** 정형 패턴(마이그레이션 러너, 쿼리 헬퍼, 타입 레이어)은 한 번에 끝난다. 설계 결정(스키마, 제약, 인덱스 전략)은 2~3번 루프가 필요하다.

배운 것: 데이터베이스 스키마 작업은 AI와 협업할 때 예측 가능한 패턴을 따른다. 설계 결정은 AI와 2~3번 주고받아야 하지만, 구현 세부는 한 번에 끝나는 경우가 많다. 다음 장의 스트리밍 AI API는 반복 횟수가 더 많고 해결책도 복잡하다.

8장. AI API 통합—AI를 사용해서 AI 코드 작성하기

세 가지 AI provider(OpenAI, Claude, Gemini)를 하나의 UI에 연결하는 작업은 AI 코딩 도구가 가장 유용하면서도 함정이 많은 영역이다. provider마다 인증 방식, 응답 형식, 오류 모양이 모두 다르다. 문서에 적힌 스펙은 AI가 잘 처리한다. 하지만 문서에 없는 부분, 즉 사용자가 취소 버튼을 눌렀을 때 HTTP 요청이 어떻게 끊기는지, 스트림이 멈추면 무엇을 보여줄지 같은 문제는 다르다.

이 장의 큰 그림은 한 문장이다. **차이는 Rust 한 곳에 모으고, 프론트엔드는 세 가지 이벤트만 받는다.**

[프론트엔드 (SvelteKit)]

```
ask_ai_stream(provider, model, messages)
```

|

v

[Tauri 명령] --> [Provider 레이어(Rust)]

- OpenAI

- Claude HTTP 스트림 -> 텍스트 조각 추출

- Gemini

|

v

[프론트엔드 listen]

```
ai_chunk(텍스트 조각) / ai_done(완료, 사용량) / ai_error(오류)
```

화면에서 사용자가 보는 것은 훨씬 단순하다. 채팅 입력창에 질문을 넣으면 답변 영역에 글자가 조금씩 붙는다. 예를 들어 provider가 "Hel", "lo", " world" 세 조각을 보내면 프론트엔드는 이를 이어 붙여 "Hello world"로 보여준다. 이 장의 코드는 바로 이“조각을 받아 이어붙이는 흐름”을 안정적으로 만드는 작업이다.

provider마다 응답·오류 형식이 달라도, 프론트엔드는 `ai_chunk`(텍스트 조각), `ai_done`(완료), `ai_error`(오류) 세 가지 이벤트만 알면 된다. 차이를 흡수하는 일은 Rust provider 레이어가 모두 맡는다.

이 책에서 처음 만나는 용어 하나를 짚어두자. **SSE(Server-Sent Events)**는 서버가 한 번 열어둔 HTTP 연결로 응답을 조금씩 흘러보내는 방식이다. AI provider는 토큰이 생성되는 즉시 한 조각씩 보내기 위해 SSE를 많이 쓴다. 프론트엔드는 전체 응답을 기다리지 않고 도착하는 조각을 곧바로 화면에 이어붙일 수 있어, “타이핑되는 듯한”경험을 만든다.

처음 구현할 때 가장 중요한 선택은“**차이를 어디에 숨길 것인가**”였다. 차이를 UI 곳곳에 흩뿌리면 버튼·채팅창·오류 토스트가 provider별 조건문으로 복잡해진다. 차이를 Rust 한 곳에 모으면 프론트엔드는 같은 이벤트만 받는다. 사용자에게 필요한 것은 응답이 오고 있는지, 끝났는지, 실패했는지뿐이다.

이 장의 흐름

- 요구사항을 먼저 글로 정리하기
- 다중 Provider 추상화—Rust trait 패턴
- 이벤트 구조와 스트림 취소 (핵심 줄기)
- provider별 이벤트를 공통 `delta·done·error`로 바꾸고 스트림 ID별로 취소·종료를 처리한다.
- 9장으로 이어지는 정리

학습 목표

- 여러 AI provider를 하나의 인터페이스로 묶는 Rust trait(인터페이스 같은 개념) 패턴을 이해한다.
- `ai_chunk/ai_done/ai_error` 세 이벤트로 스트리밍을 다루는 흐름을 익힌다.
- provider마다 다른 SSE·오류·인증 차이를 한 곳에서 흡수하는 이유를 안다.
- API 키 보관과 같은 기본 보안 원칙을 안다.

이 장은 Rust 비동기, SSE, provider별 응답 차이, 취소 처리까지 한꺼번에 나온다. 처음부터 모두 이해하려고 하면 어렵다. 첫 회독에서는 다음 네 가지만 잡으면 된다.

- UI는 provider별 차이를 몰라도 된다.
- Rust provider 레이어가 OpenAI, Claude, Gemini의 차이를 흡수한다.
- 프론트엔드는 `ai_chunk`, `ai_done`, `ai_error` 세 이벤트만 받는다.
- API 키와 비용 제한 같은 운영 세부는 중요하지만, 앱 흐름을 이해한 뒤 다시 보면 된다.

`tokio::select!`, provider별 SSE 종료 신호, rate limiting은 지금 완전히 외울 필요가 없다. 이 장에서는 **“차이는 한곳에 숨기고, 화면은 단순하게 유지한다”**는 원칙만 기억하면 된다.

8.1 요구사항을 먼저 글로 정리하기

AI에게 “세 가지 provider를 연결해줘”라고만 하면 결과가 들쭉날쭉하다. provider마다 다른 예제 코드가 뒤섞이고, 오류 처리도 제각각으로 나온다. 그래서 코드를 짜기 전에 요구사항을 먼저 정리했다.

- 프론트엔드는 provider를 몰라도 된다. 명령 하나만 쓴다: `ask_ai_stream(provider, model, messages)`.
- 스트리밍은 provider의 작은 delta를 잠깐 모은 뒤 Tauri emit으로 `ai_chunk` 이벤트를 발생시킨다. 프론트엔드는 `listen`으로 받는다. 처리량이 커지면 Tauri Channel로 바꾼다.
- 토큰 사용량은 스트림이 끝날 때 `ai_done` 이벤트로 한 번 보낸다.
- 모든 에러는 `ai_error` 이벤트 하나로 통합한다.
- 사용자 소유 API 키는 OS keyring에서 읽고 명령 파라미터로 넘기지 않는다. 개발사 소유 키는 서버에 둔다.
- `cancel_ai_stream(stream_id)` 명령으로 진행 중인 요청을 끊는다.

이 여섯 줄을 프롬프트 맨 위에 붙였다. 당시 작업 기록에서는 같은 범위를 설명 없이 요청했을 때보다 수정 라운드가 대략 절반으로 줄었다. 이는 일반적인 성능 수치가 아니라 이 프로젝트의 비교 기록이다. 스트리밍처럼 상태가 복잡한 기능일수록 시작 전에 이벤트 이름과 책임 경계를 못 박아두는 편이 결과를 비교하기 쉽다.

핵심 설계 원칙은 **“프론트엔드를 provider 변화로부터 보호한다”**는 것이다. 모델 이름이 바뀌거나 provider가 추가되어도 채팅 UI는 손대지 않아야 한다.

보안: 환경 변수는 자식 프로세스·로그·운영체제별 프로세스 검사 기능에 노출될 수 있으므로 데스크톱 앱의 장기 저장소로 쓰지 않는다. 사용자가 자기 API 키를 넣는 BYOK 방식이라면 macOS Keychain, Windows Credential Manager, Linux Secret Service를 Rust의 keyring 같은 프레임워크로 사용한다. 개발사가 소유한 공용 서비스 키는 데스크톱 앱 안에 숨길 수 없다. 이런 키

는 서버 프록시에 보관하고 사용자 인증과 사용량 제한도 서버에서 처리해야 한다. 공식 Tauri keyring 플러그인이 따로 있다고 가정해서는 안 된다.

8.2 다중 Provider 추상화—Rust Trait

왜 trait이 필요한가부터 짧게 짚자. provider별 코드를 명령 핸들러 안에 그냥 넣으면, 갈수록 다음과 같이 부풀어 오른다.

```
if provider == "openai" { ... 100줄 ... }
else if provider == "claude" { ... 100줄 ... }
else if provider == "gemini" { ... 100줄 ... }
```

provider가 늘 때마다 같은 함수가 더 길어지고, 한 곳을 고치면 다른 분기도 깨지기 쉽다. 공통 인터페이스로 묶어 두면, 새 provider는 같은 약속에 맞춰 별도 파일에 구현하면 된다.

Rust의 **trait**은 **TypeScript의 인터페이스와 비슷한 개념**으로, “이 목록에 있는 기능을 모두 구현해야 provider로 쓸 수 있다”는 계약서 역할을 한다. 이 프로젝트의 provider trait은 메서드 네 개만 두면 된다.

TypeScript로 비유하면 이런 모양에 가깝다.

```
interface ChatProvider {
  streamChat(input: ChatInput): Promise<void>;
  parseChunk(event: unknown): string;
  parseUsage(event: unknown): Usage;
  mapError(error: unknown): AiError;
}
```

Rust trait도 같은 약속을 강제한다. 차이는 문법뿐이다. 초보자는 “provider마다 반드시 같은 네 기능을 구현하게 만드는 계약”으로 이해하면 충분하다.

메서드	책임
stream_chat	HTTP 요청을 보내고 SSE 이벤트를 받아 emit한다
parse_chunk	한 이벤트에서 텍스트 조각(delta)을 꺼낸다
parse_usage	토큰 사용량 데이터를 꺼낸다
map_error	HTTP 상태와 응답을 공통 에러 형식으로 바꾼다

이 네 가지를 정해두면, 명령 핸들러는 provider 문자열을 보고 해당 인스턴스를 골라 stream_chat 만 호출하면 된다. 의사 코드로는 다음과 같은 한 토막이다.

```
// 명령 핸들러: 분기는 한 번, 이후엔 trait 메서드만 부른다
let p: Box<dyn ChatProvider> = match provider {
    "openai" => Box::new(OpenAi),
    "claude" => Box::new(Claude),
    "gemini" => Box::new(Gemini),
    _        => return Err(unknown_provider()),
};
p.stream_chat(model, messages, stream_id, app).await
```

새 provider를 추가할 때는 같은 메서드 네 개를 구현한 타입을 추가하고 match 한 줄만 늘리면 된다. 명령 핸들러 본체와 프론트엔드는 건드리지 않는다.

공통 provider 인터페이스의 계약은 요청과 취소 신호를 받고, 텍스트 delta·종료·오류 중 하나의 이벤트를 돌려주는 것이다. 예를 들어 내부 타입을 `Delta { stream_id, text }, Done { stream_id }, Error { stream_id, code, message }`로 정할 수 있다. provider의 원본 응답은 어댑터 안에서 이 타입으로 바꾸고 UI는 provider 이름에 따라 분기하지 않는다. 인증키·HTTP 응답 원문을 오류 메시지에 그대로 넣지 않는다.

8.3 이벤트 구조

trait이 “차이를 한곳에 모으는 그릇”이라면, 이벤트 구조는 “그 그릇이 바깥에 보여주는 일정한 모양”이다. 두 가지는 한 쌍이다. trait이 있어도 이벤트 이름이 매번 바뀌면 프론트엔드가 흔들리고, 이벤트가 통일돼 있어도 내부에 추상화가 없으면 핸들러가 부풀어 오른다.

첫 단계에서는 실제 HTTP 통신을 미루고, **Tauri 명령과 이벤트 발행 구조만 먼저** 잡았다. 핵심은 두 가지다.

- 백엔드는 명령이 호출되면 `tokio::spawn`(Rust의 비동기 런타임 tokio가 제공하는, 작업을 백그라운드로 띄우는 함수)으로 작업을 띄우고 즉시 반환한다. **JavaScript에서 Promise를 만들어 백그라운드로 흘려보내는 것과 비슷한 역할**이다. 그래서 프론트엔드는 응답을 기다리지 않는다.
- 프론트엔드는 `listen`으로 `ai_chunk`, `ai_done`, `ai_error` 이벤트를 구독한다.

프론트엔드 리스너의 핵심 흐름은 네 줄로 줄어든다.

```
const unChunk = await listen<{ delta: string }>(
  "ai_chunk",
  e => buf += e.payload.delta,
);
const unDone = await listen<{ usage: Usage }>(
  "ai_done",
  e => { usage = e.payload.usage; },
);
await invoke("ask_ai_stream", { provider: "claude", model, messages });
```

핵심은 **buf가 delta를 이어붙이는 방식**이다. ai_chunk 이벤트는 전체 메시지를 한 번에 보내지 않는다. 새로 도착한 조각만 전달하고, 프론트엔드가 순서대로 붙여 완전한 메시지를 만든다. 다만 provider가 토큰을 보낼 때마다 전역 이벤트를 하나씩 만들면 직렬화와 렌더링 비용이 커진다. 이 프로젝트는 짧은 시간 동안 도착한 delta를 묶어 보냈고, 더 높은 처리량이 필요하면 순서 보장에 맞는 Tauri Channel을 쓰도록 경계를 남겼다.

ai_chunk: "Hel"	buf = "Hel"
ai_chunk: "lo"	buf = "Hello"
ai_chunk: " world"	buf = "Hello world"
ai_done	렌더링 종료, usage 저장

또한, 같은 정규화의 일부로 **프론트엔드가 받는 페이로드 형식은 네 가지로 고정**한다. ai_chunk는 { delta }, ai_done은 { usage }, ai_error는 { provider, kind, message }, 그리고 취소 알림은 { stream_id }다. provider가 늘어도 이 모양은 바뀌지 않는다.

Claude가 한때 ai_done에 model.finish_reason까지 끼워 넣으려 했지만, “프론트엔드에서 안 쓰는 필드는 일단 빼고, 필요해질 때 추가하자”고 정리했다. **AI는 일단 필드를 더 넣으려 한다.** 그냥 두면 페이로드가 계속 불어난다.

리스너를 등록한 뒤에는 반드시 정리(cleanup)해야 한다. listen이 돌려주는 함수(unChunk, unDone)를 호출하지 않으면, 다음 메시지에서 같은 콜백이 두 번 세 번 쌓여 호출되고, 화면에는 같은 텍스트가 중복으로 찍힌다. 페이지를 떠나도 리스너가 살아남아 메모리를 잡아둔다.

정리 시점은 두 곳이다—스트림이 끝난 직후(ai_done 또는 ai_error 수신 시), 그리고 컴포넌트가 unmount될 때(Svelte라면 onDestroy, React라면 useEffect의 cleanup).

스트림 취소

여기까지의 구조에는 한 가지 문제가 있다. **한 번 시작된 스트림을 멈출 방법이 없다.** 사용자가 “정지”를 클릭하거나 페이지를 벗어나도, 백그라운드 작업이 계속 실행된다. API 크레딧을 소모하고, 이미 닫힌 프론트엔드에 이벤트를 계속 보낸다.

해결의 핵심 한 줄은 이렇다. **스트림 하나하나에 ID와 취소 채널을 따로 둔다.** 앱 전체에 채널을 하나만 두면 취소 신호가 모든 스트림을 깨워, 누구를 멈춰야 하는지 알 수 없다(이런 상황을 두 작업이 엉키는 **race condition**이라 부른다).

구체적으로는 스트림을 시작할 때 UUID로 `stream_id`를 만들고 ID별로 취소 채널을 등록한다. `cancel_ai_stream(stream_id)`이 호출되면 그 ID의 채널에만 신호를 보낸다.

작업 쪽에서는 `tokio::select!`라는 도구로 “취소 신호 도착”과 “HTTP 스트림 완료”중 먼저 발생한 쪽을 따라간다. `tokio::select!`는 **JavaScript의 Promise.race와 비슷하게**, 여러 비동기 작업 중 가장 먼저 끝나는 쪽을 처리한다. 취소가 먼저 도착하면 백엔드는 `ai_cancelled`로 알리는데, 프론트엔드는 이 페이로드로 `stream_id`를 받아 어느 스트림의 종료인지 식별한다.

등록 순서는 스트림 ID 생성 -> 이벤트 리스너 준비 -> 요청 시작이다. 리스너 준비 전에 응답이 오면 첫 토큰을 잃을 수 있다. 각 callback은 자기 `stream_id`만 처리하고, `done·error·취소·화면 종료`에서 모두 같은 `cleanup`을 호출한다. `cleanup`은 여러 번 호출해도 안전하게 만들며, 등록이 늦게 끝난 뒤 이미 종료된 화면이면 새 리스너를 즉시 해제한다. 5장 예제의 `disposed` 처리가 같은 원리다.

고려사항: 스트림이 자연스럽게 끝나는 순간과 취소 신호가 겹치면, `ai_done`과 `ai_cancelled` 이벤트가 동시에 발생할 수 있다. 프론트엔드는 첫 번째 완료 신호를 받은 뒤 오는 추가 이벤트를 무시해야 한다.

8.4 SSE 파싱이 실패한 곳

여기서부터 8.6까지는 “첫 회독 이후 실제 코드를 짜며 마주칠 디테일”이다. 첫 읽기에는 큰 그림 한 줄씩만 잡고 넘어가도 된다.

큰 그림은 단순하다. **provider마다 종료 신호·이벤트 종류·인증 위치가 다르다.** 이 프로젝트가 사용한 OpenAI Chat Completions 스트림은 마지막에 `data: [DONE]`을 보내므로 모든 `data:` 줄을 JSON으로 가정하면 깨진다. Responses API처럼 이벤트 형식이 다른 API로 바꾸면 파서도 공식 스펙에 맞게 함께 바뀌어야 한다.

Anthropic(Claude)은 SSE의 `event`: 줄로 분기해야 한다. `message_delta`의 `usage.output_tokens`는 누적 카운트이므로 이벤트마다 더하지 않고 마지막 값을 사용한다. Gemini의 `REST streamGenerateContent`를 SSE로 받으려면 `alt=sse`를 사용한다. API 키는 URL 쿼리에 넣기보다 공식 예제처럼 `x-goog-api-key` 헤더로 전달해 URL·접근 로그 노출을 줄인다.

provider 어댑터는 공통 요청을 provider별 JSON으로 바꾸고, 받은 이벤트를 공통 스트림 타입으로 되돌린다. SSE는 네트워크 청크 하나가 이벤트 하나라는 보장이 없으므로 버퍼에 모아 빈 줄로 구분된 이벤트를 파싱한다. UTF-8 문자가 청크 경계에서 나뉘는 경우도 처리한다. HTTP는 request, JSON은 `serde_json`으로 처리할 수 있으며 provider의 필드 이름·종료 표시는 버전별 API 계약에 맞춰야 한다.

주의 1: Anthropic 호출에는 `anthropic-version` 헤더가 반드시 있어야 한다. 빠지면 HTTP 400으로 실패한다. 버전 문자열은 코드에 하드코딩하지 말고 공식 문서에서 현재 값을 확인하자.

주의 2: Gemini API 키는 `x-goog-api-key` 헤더로 전달한다. 헤더도 디버그 로그에 남기지 말아야 하며, 개발사 소유 키를 배포 앱에 넣어야 하는 제품이라면 OAuth 또는 서버 프록시 구조를 검토한다.

주의 3: 토큰 수 정확도는 provider마다 다르다. 토큰 수에 직접 의존하는 결제 로직은 만들지 않는 게 좋다.

첫 회독에서는 위 세 주의사항을 외울 필요가 없다. 중요한 것은 **provider마다 스트림 형식이 다르므로, 차이를 UI가 아니라 provider 레이어에서 처리해야 한다**는 점이다.

8.5 에러 처리—Provider마다 다르게 실패할 때

세 provider는 에러 응답 형식이 모두 다르다. OpenAI는 `error.type`과 `error.code`, Anthropic은 `error.type`, Gemini는 `error.status`와 `error.code` 식으로 필드 이름과 위치가 제각각이다.

이 차이를 프론트엔드에 그대로 넘기면 UI마다 분기 코드가 늘어난다. **에러 정규화의 목적은 단 하나다.** provider가 어떤 모양으로 실패하든, 프론트엔드는 `{ provider, kind, message }`라는 같은 형태만 받는다.

`kind`는 `auth·rate_limit·context·server·unknown` 같은 소수의 분류로 정해두고, HTTP status code와 provider별 error type 문자열을 함께 보고 그중 하나로 매핑한다.

오류 매핑은 HTTP 401·403->인증/권한 오류, 429->호출 제한, 5xx->서버 오류, timeout->시간 초과, 파싱 실패->응답 형식 오류로 구분하는 것부터 시작한다. 429는 Retry-After가 있으면 존중하고 최대 재시도 횟수를 둔다. 인증 실패는 키를 바꾸기 전까지 반복 재시도하지 않는다. 화면에는 안전한 설명을, 내부 로그에는 요청 ID와 오류 종류를 남겨 원인을 연결한다.

주의: AI provider의 error message에는 API key 일부나 내부 서비스 이름 같은 민감한 정보가 포함될 수 있다. 사용자에게 보여주거나 로그에 기록하기 전에 반드시 sanitize해야 한다.

운영 단계에서 추가로 고려할 것: AI API는 호출마다 비용이 발생하고, provider별로 “분당 요청 수”와 “분당 토큰 수”가 따로 걸려 있다. 보통 **token bucket** 알고리즘으로 두 한도를 동시에 관리하고, 같은 자리에 모델별 단가를 두어 누적 비용을 추적한다. 가격표와 한도는 자주 바뀌므로 코드에 하드코딩하지 않고 설정으로 분리한다. 그리고 청구 계산에는 사전 추정이 아니라 응답의 usage 숫자를 쓴다. 첫 회독에서는 “요청 수와 토큰 수, 두 한도를 따로 본다”만 기억하면 충분하다. 구현 디테일은 운영 장에서 다룬다.

8.6 응답 정규화—프론트엔드에 통일된 형식 제공

응답 페이로드 형식의 정규화 자체는 8.3에서 이벤트 구조와 함께 정리했다. 여기서는 그 결정이 왜 흔들리면 안 되는지를 한 번 더 짚는다.

페이로드 모양을 흐트러뜨리면 프론트엔드와 백엔드가 불필요하게 엮인다. 다섯 번째 provider를 추가할 때 채팅 UI까지 손대야 한다. 그래서 새 필드 요청은 보수적으로 받는다—“이 필드를 지금 화면 어디에서 쓰는가?”에 답할 수 없으면 일단 보류한다. 정규화의 가치는 형식이 변하지 않는다는 약속 자체에서 나오기 때문이다.

8.7 이 장의 루프 정리

이 장 동안 영역별로 수정한 횟수를 표로 정리해 두었지만, 다음 장 도입부에서 같은 데이터로 더 자세히 분석한다. **여기서 기억할 한 줄은 이것이다. 설계가 잘 잡혀 있으면 대부분 1~2번에 통과했고, provider별 프로토콜 예외에서만 수정 횟수가 늘었다.** 9장이 이 결을 일반화한다.

AI가 코드를 짜는 방식도 여기서 드러난다. **AI가 잘하는 영역(관용 패턴·추상화 설계·예제 있는 데이터 변환)과 헤매는 영역(provider별 프로토콜 예외·종료 신호·인증 위치)의 구분**, 그리고 “약한 영역은

API 문서 요약을 프롬프트에 함께 넘긴다”는 대응법은 다음 장에서 사례와 함께 본격적으로 정리한다. 여기서는 한 줄만 남겨둔다—**요구사항이 명확하고 예제가 충분할 때 AI 코드 생성이 가장 잘 작동한다.**

마무리 박스: 이 장에서 반드시 기억할 것

- 한 문장: **차이는 Rust 한 곳에 모으고, 프론트엔드는 ai_chunk/ai_done/ai_error 세 이벤트만 받는다.**
- 핵심 구조: provider trait(메서드 4개) / Tauri 이벤트 기반 스트리밍(delta 누적) / stream_id 단위 취소(tokio::select!).
- Provider 함정: 이 프로젝트의 OpenAI Chat Completions는 [DONE] 문자열을 별도 처리한다. Anthropic은 event: 줄로 분기하고 누적 출력 토큰의 마지막 값을 쓴다. Gemini REST 스트림은 alt=sse, 인증은 x-goog-api-key 헤더를 쓴다.
- 운영: 사용자 소유 키는 OS keyring에, 개발사 소유 키는 서버에 둔다. rate limit은 요청 수·토큰 수를 따로 보고 가격표는 코드에 박지 않는다.
- 안전: 리스너 등록 후 cleanup(스트림 종료 시·컴포넌트 unmount 시), 취소와 정상 완료가 겹칠 때 첫 신호만 따르기, 에러 메시지는 sanitize 후 사용자에게 노출.
- AI 활용: 관용 패턴·추상화 설계·데이터 변환은 잘 됨. 프로토콜 예외는 약함 -> **요구사항을 먼저 글로 정리하고, 예외 처리에는 API 문서 요약을 함께 넘긴다.**

다음 장인 9장은 이 표에서 보인 결을 일반화한다. “Claude가 잘 처리하는 영역과 헤매는 영역은 어디고, 학습 데이터에 없는 패턴 앞에서 어떻게 대응할 것인가.” 그리고 이 장에서 잡은 **trait 기반 추상화, 세 이벤트 페이로드, stream_id 단위 취소** 패턴은 9장 이후 외부 API를 추가로 붙일 때마다 같은 모양으로 반복해서 등장한다. 일곱 라운드의 흔적은 다음 장의 출발점이 된다.

9장. Mandai 개발 실수—AI가 틀린 곳들

AI와 협업하면 두 가지 결과가 나온다. AI가 맞춘 것과, AI가 틀린 것. 더 많이 배운 쪽은 두 번째였다. 검증 없이 코드를 승인한 곳, 기능을 세 개 쌓고 나서야 구조 문제를 발견한 곳—그 실패들이 다음 시도의 재료가 됐다.

이전 장들은 AI와 협업하는 흐름을 다뤘다. 이 장은 반대편을 보여준다. **AI가 반복해서 틀린 영역들**과 내가 어떻게 그것을 잡아내고 우회했는지의 기록이다. 한두 번 수정으로 끝난 오타나 import 오류는 제외했다. 다섯 번 이상 수정했거나, 요구사항 자체를 다시 써야 했던 문제만 골랐다. 반복해서 나타난 함정은 네 가지였다.

1. **상태 동기화** —프론트엔드와 백엔드가 서로 다른 상태를 본다.
2. **암묵적 규칙** —내 머릿속에만 있어서 AI가 알 수 없는 규칙들.
3. **옛지 케이스 무관심** —AI는 기본적으로 정상 경로(에러 없이 흐르는 일반 흐름)를 선호한다.
4. **버전 혼동** —AI가 오래된 API와 새로운 API 패턴을 섞는다.

용어를 짧게 정리해 두자. **상태 동기화**는 같은 정보를 두 군데(예: 화면과 DB)에 보관할 때 두 값이 항상 일치하도록 맞추는 일이다. **도메인 모델 이탈**은 AI가 우리 프로젝트의 고유한 규칙 대신, 일반적으로 많이 쓰는 패턴을 골라 적용해 버리는 현상이다. 두 용어는 이 장에서 계속 등장한다.

주의: 검증 없이 AI “모범 사례”를 믿지 마라. AI는 튜토리얼에서는 작동하지만 실제 서비스에서는 실패하는 패턴을 자신 있게 제시한다. 항상 전체 동작 흐름을 테스트해라.

이 장은 AI를 비판하려는 것이 아니다. 오히려 반대다. AI가 자주 틀리는 지점을 알면 더 정확한 요청을 보낼 수 있다. “타이머를 만들어줘”와 “앱을 껐다 켜도 진행 중인 타이머가 복원되어야 한다”는 전혀 다른 요청이다. 후자에는 상태의 소유권, 저장 시점, 복원 로직이 모두 담겨 있다. 실제 앱에서 필요한 요구사항은 대부분 이런 식이다. 한 문장이 더 구체적이면 구현 전체의 방향이 달라진다.

읽기 가이드: 이 장은 9.1이 가장 길고 나머지는 짧다. 9.1만 통과하면 뒤는 빠르게 읽힌다. 9.1이 길어진 이유는 거기에 이 장의 핵심 통찰이 다 들어 있어서다.

이 장의 흐름

먼저 가장 무거운 사례를 보고, 점점 가벼운 사례로 내려간다. 마지막에 예방 도구를 한 묶음으로 정리한다.

- **9.1 Pomodoro 타이머** —상태가 두 곳에 살 때 무슨 일이 생기는가
- **9.2 Mandal-Art Cell ID** —일반 패턴이 도메인 규칙을 덮어쓰는 순간
- **9.3 생산성 점수 계산** —엣지 케이스를 명시하면 AI가 개선된다
- **9.4 버전 혼동** —Tauri 1.x와 2.x API를 섞을 때
- **9.5 막는 법: 프롬프트·리뷰·테스트 3종 세트**

학습 목표

- AI 개발에서 반복해서 나타나는 네 가지 실수 패턴(상태 동기화, 암묵적 규칙, 엣지 케이스, 버전 혼동)을 파악한다.
- 각 실수를 어떻게 발견하고 프롬프트로 해결하는지 실제 사례로 배운다.
- AI 실수 방지 체크리스트와 리뷰 프롬프트, 테스트 프로토콜을 직접 만들고 활용하는 방법을 익힌다.

9.1 Pomodoro 타이머—상태가 두 곳에 산다

이 절의 핵심 교훈은 한 줄이다: 진행 중인 타이머의 “남은 시간”을 어디에 저장하든, 항상 `start_at`에서 다시 계산할 수 있어야 한다. 다섯 번의 시행착오 끝에 이 결론에 닿았다. 아래 이야기는 그 과정이다.

Mandai의 Pomodoro(25분 집중 + 짧은 휴식을 반복하는 작업법)는 겉으로는 간단해 보인다. 시작 버튼을 누르고, 25분을 세고, 끝나면 DuckDB에 세션을 저장하면 될 것 같다. 실제로는 그렇지 않다.

화면에 보이는 숫자 뒤에는 여러 정보가 따라붙는다. 시작 시각, 목표 시간, 연결된 태스크, 종료 사유, 알림, 그리고 앱을 다시 켰을 때 어떻게 되돌릴지에 대한 규칙이 모두 필요하다. 게다가 사용자는 타이머가 돌아가는 중에 앱을 닫을 수 있고, 노트북은 갑자기 절전 모드로 들어갈 수 있다.

처음에는 이런 조건을 프롬프트에 제대로 담지 않았다. 그러다 보니 AI가 만든 첫 타이머는 데모 수준 이었고, 실제 조건을 하나씩 넣을 때마다 새 문제가 드러났다. 결국 다섯 번을 고쳐야 했다. 수정할 때마다 ‘타이머’라는 단어가 단순한 카운트다운이 아니라는 점이 명확해졌다. 타이머는 상태를 저장하고, 필요할 때 그 상태를 다시 불러올 수 있어야 하는 작은 시스템이다.

진짜 문제

핵심 문제는 Pomodoro 상태가 두 곳에 나뉘어 있다는 점이다.

- **프론트엔드:** 화면에서 매초 줄어드는 남은 시간.
- **백엔드 (DuckDB sessions 레코드):** 영구 로그. `start_at`, `end_at`, `ended_reason`.

Claude는 처음에 **타이머를 프론트엔드에만** 두고, 세션이 끝난 시점에 DB에 저장하도록 구현했다. 데모에서는 문제가 없어 보인다. 카운트다운이 줄어들고, 0이 되면 완료 기록이 남는다. 하지만 앱을 닫았다가 다시 열면 실행 중이던 세션은 어디에도 남아 있지 않다. 필요한 요구사항은 “앱을 껐다 켜도 진행 중인 타이머가 이어져야 한다”였다. 다시 말해, 타이머 상태는 화면이 아니라 DB에서 읽어 복원할 수 있어야 한다.

잘못된 첫 시도

처음에는 “25분 카운트다운을 표시하고, 끝나면 DB에 저장해라” 정도의 짧은 프롬프트를 보냈다. Claude가 만든 코드는 타이머 상태를 컴포넌트 변수에만 두고, **세션이 끝난 시점에만** DB에 한 줄을 기록했다.

```
// 틀림: 타이머 상태가 컴포넌트에만 존재
let remainingSeconds = 25 * 60;
// 완료 시점에 한 번만 DB INSERT -> 실행 중 지속성 없음
```

데모에서는 작동한다. 하지만 사용자가 세션 중간에 앱을 닫는 순간, 진행 중이던 작업 시간은 기록되지 않는다. 튜토리얼 타이머는 화면이 켜져 있는 동안만 동작하면 되지만, 실제 앱의 타이머는 화면이 꺼진 뒤에도 현재 상태를 어딘가에 보관해야 한다.

이 첫 시도에 이어 네 번의 수정이 더 있었다. 시작 시점에 DB에 미리 INSERT하기, 재시작 시 열린 세션 정리하기, 무조건 정리 대신 시간 기반으로 복구하기, 그리고 타이밍 자체를 Rust 백엔드로 옮기기. 각 수정은 직전 시도의 빈틈에서 시작됐다. 세부는 이 절 끝의 **핵심 교훈**에서 한꺼번에 정리한다. 여기서는 그 과정에서 마주친 함정 세 개만 따로 살펴보자.

함정 1 —CHECK 제약 위반

진행 중인 세션을 시작 시점에 INSERT하려고 했을 때, Claude는 ended_reason='running'을 넣으려 했다. 7장의 sessions 테이블에는 ended_reason 컬럼에 들어올 수 있는 값을 'completed', 'skipped', 'interrupted'로 제한하는 CHECK 제약(특정 값만 허용하는 DB 규칙)이 걸려 있어서 거부당했다.

해결책은 단순하다. 시작 시 ended_reason은 NULL로 두고(= 진행 중), 완료·중단·스킵 시 값을 채워 넣는다. 7장의 스키마를 그대로 살릴 수 있는 방법이다.

함정 2 —과도한 정리 로직과 시간 기반 복구

“재시작 시 ended_reason이 NULL인 세션을 정리해 달라”고 요청했더니, Claude는 앱이 시작될 때 열린 세션을 모두 interrupted로 표시하는 함수를 만들었다. 그런데 이게 너무 공격적이었다. **앱이 잠깐 백그라운드에 갔다가 돌아온 경우에도 타이머가 중단된 것으로 처리됐다.** 사용자가 5분 만에 돌아왔는데 타이머가 이미 죽어 있다.

여기서 깨달았다. 앱을 닫는 것, 최소화하는 것, 노트북이 잠드는 것, 프로세스가 강제 종료되는 것은 모두 다른 상황이다. 그런데 DB에서는 그 차이가 end_at IS NULL 하나로만 나타난다.

해법의 방향을 바꿨다. 무조건 정리하지 말고, **시작 시각을 기준으로 다시 계산**하자. 즉 start_at 이후 흐른 시간이 목표 시간을 넘었으면 자동으로 종료 처리하고, 아직 남았다면 남은 시간을 다시 계산해 타이머를 살린다. 핵심은 한 가지다—저장된 “남은 시간”을 믿지 말고, 매번 start_at에서 다시 계산한다.

확인할 출력: 앱을 닫고 5분 뒤 다시 열면 화면의 남은 시간이 5분 줄어 있어야 한다. 그대로면 저장된 값을 그대로 쓰고 있다는 뜻이다.

합정 3 —백그라운드 스로틀링과 태스크 바인딩

마지막 루프에서 두 가지가 더 드러났다.

백그라운드 스로틀링은 앱이 백그라운드에서 실행 중인 동안 브라우저 엔진(WebView)이 JavaScript 타이머의 실행 빈도를 줄이는 현상이다. `setTimeout`도 똑같이 영향을 받는다. 해법은 타이밍을 **Rust 백엔드로 옮기는 것이다**. Tauri 백엔드는 브라우저의 스로틀링에 영향받지 않는다. 백엔드에서 시간을 재고, 끝나면 프론트엔드에 이벤트를 보내는 구조다.

태스크 바인딩은 다른 종류의 결정 문제다. 사용자가 타이머 도중 태스크를 바꾸면 `session.task_id`도 따라가야 할까? 명시된 규칙이 없으면 AI는 매번 다르게 추측한다. 결정: **세션은 시작 시점의 태스크에 고정한다**. 생산성 보고서를 해석하기 쉬워지고, 통계가 일관된다.

확인할 출력: 앱을 백그라운드로 보내고 25분을 기다리면 알림 사운드가 울려야 한다.

수정된 프롬프트 템플릿

다섯 번의 루프를 거친 뒤, 처음부터 이렇게 물었다면 더 빨랐을 프롬프트다.

이 요구사항들로 Pomodoro 타이머를 만들어라:

지속성:

- 시작 즉시 DuckDB에 세션 저장 (`end_at=NULL`, `ended_reason=NULL`)
- 재시작 시 경과 시간이 `target_minutes` 미만이면 활성 세션 복원
- `target_minutes` 초과한 경우에만 `ended_reason='interrupted'`로 표시

타이밍:

- Rust 백엔드 `tokio::time` 사용 (브라우저 스로틀링 영향 없음)
- 프론트엔드는 표시만, 백엔드가 실제 타이밍 관리
- 백엔드 타이머 만료 시 Tauri 이벤트 `'timer_completed'` 발생

태스크 바인딩:

- 세션 `task_id`는 시작 시점의 활성 태스크에 고정
- 세션 중 태스크 변경은 `session.task_id`에 영향 없음

테스트:

- [] 타이머 시작, 앱 닫기, 다시 열기 -> 올바른 남은 시간으로 복원
- [] 전체 시간 동안 앱 백그라운드 -> 알림 여전히 울림
- [] 타이머 중 앱 크래시 -> 다음 시작 시 `'interrupted'`로 표시

주의: 잘못된 구조로 계속 밀어붙이는 건 매몰비용 오류다. 같은 기능에서 루프를 3~4번 돌아도 해결이 안 된다면, AI의 초기 접근 방향이 근본적으로 틀린 것은 아닌지 점검해라. 제약 조건을 새로 정리하고 처음부터 다시 시작하는 편이 더 빠를 수 있다.

핵심 교훈

상태 동기화란, 프론트엔드 타이머와 DB 데이터가 항상 같은 값을 가리키도록 맞추는 일이다. 이 문제는 AI 혼자서는 해결하기 어렵다. 왜냐하면 세 가지를 먼저 사람이 결정해야 하기 때문이다. 프론트와 백엔드 중 누가 어떤 역할을 맡는지, 앱이 비정상 종료되었을 때 어떻게 복구할지, 그리고 데이터 일관성을 어떻게 보장할지. 이 세 가지를 결정해서 AI에게 알려줘야 한다. 그렇지 않으면 AI는 모든 것이 정상일 때만 작동하는 코드를 작성한다.

다섯 번의 루프를 거친 뒤 **타이머 상태 동기화 체크리스트**를 만들었다.

- ☐ 프론트엔드 타이머와 DB 행이 같은 순간에 시작되는가?
- ☐ 앱 크래시 시 열린 세션들에게 무슨 일이 일어나는가?
- ☐ 절전/백그라운드에서 복귀할 때 사용자가 보는 상태는?
- ☐ 누가 종료 알림을 올리는가: 프론트엔드인가 백엔드인가?
- ☐ 세션 메타데이터(태스크 등)가 실행 중 변경되면 무시하는가 반영하는가?

비슷한 기능을 구현할 때는 프롬프트 맨 위에 이 다섯 항목을 붙여서 시작한다. 한 번의 삽질에서 얻은 교훈이 앞으로 계속 쓸 수 있는 프롬프트 자산이 된다. 다음 절에서는 이“사전 명시”원칙을 다른 형태의 함정—도메인 규칙—에 적용해 본다.

9.2 Mandal-Art Cell ID —일반 패턴이 도메인 규칙을 덮어쓰는 순간

상태 동기화 다음으로 자주 만난 함정이 **도메인 모델 이탈**이다. AI는 우리 프로젝트의 고유한 규칙을 알지 못한 채, 학습 데이터에서 자주 본 일반 패턴을 그대로 적용한다. Cell ID 사례가 가장 직관적이다.

6장에서 cell ID 형식을 {center_idx}-{sub_idx}-{cell_idx}로 정했다. 이 결정은 .cursor/rules/mandai.mdc에도 기록해뒀다. 그런데 AI가 UI 컴포넌트를 작성하면서 **전혀 다른 형식을 만들어냈다**.

어디서 어긋났나

Claude가 작성한 `MandalaGrid.svelte`는 셀을 `id="cell-${row}-${col}"` 형식으로 렌더링했다. `row`와 `col`은 0부터 8까지의 숫자다. 결과적으로 DB의 테이블은 1-3-5 형식을 쓰고, 화면의 DOM은 `cell-3-5` 형식을 쓰게 됐다. 클릭 핸들러가 DOM id를 가지고 DB를 검색하는 순간 두 형식이 맞지 않아 오류가 났다.

원인을 찾는 데 2시간이 걸렸다. 콘솔 로그에 `No cell found with id 'cell-3-5'`라는 메시지가 반복해서 찍혔는데, DB에는 '1-1-1', '1-1-2' 형식만 있었다. **ID 체계가 두 개로 갈렸다.**

왜 이런 일이 일어나는가

내가 보낸 프롬프트가 너무 일반적이었다. “9×9 셀 그리드를 만들어 달라”고만 했지, 이 그리드가 사실 **3×3 블록이 겹쳐 이루어진 구조**라는 사실을 알려주지 않았다. Claude는 자연스럽게 일반 그리드 패턴(`row/col`)을 골랐다. AI 입장에서는 합리적인 선택이었지만, Mandai의 도메인 모델과는 맞지 않았다.

규칙 파일에 형식이 적혀 있긴 했지만, 다른 형식을 금지한다고 명시하지 않았다. 그래서 AI는 더 익숙한 일반 패턴을 골랐다.

고친 방법

수정은 코드가 아니라 규칙 파일에서 먼저 시작했다. `.cursor/rules/mandai.mdc`에 다음 한 줄을 추가했다.

Mandal-Art cell ID는 다음 형식의 문자열이다.

```
`{center_idx}-{sub_idx}-{cell_idx}`
```

DOM `data-cell-id`는 같은 형식을 사용한다. 다른 형식은 허용되지 않는다.

프롬프트도 더 구체적으로 바꿨다. “9×9 그리드”라고만 쓰지 않고, “3×3 서브 그리드를 묶은 구조이고, ID는 `{center_idx}-{sub_idx}-{cell_idx}` 형식을 사용해야 한다”고 명시했다. 수정된 컴포넌트는 DOM의 `data-cell-id`도 같은 형식으로 통일하도록 만들었다.

확인할 출력: 셀을 클릭했을 때 콘솔에 1-3-5 같은 형식이 찍히면 정상이다. `cell-3-5`가 찍히면 옛 코드가 남아 있다.

그 한 줄을 규칙 파일에 추가하고 검증 항목에도 넣은 뒤, 이후 작업 표본에서는 같은 실수가 다시 발견되지 않았다. **규칙이 명시적으로 적혀 있지 않으면 AI는 빈칸을 추측으로 채운다.**

주의: AI는 도메인 모델과 충돌하더라도 “표준” 패턴을 자신 있게 사용한다. 명시적으로 금지하지 않으면, 일반적인 패턴이 도메인 고유의 규칙을 덮어쓴다.

이 경험에서 얻은 결론은 단순하다. AI의 추측은 “일반적으로 많이 쓰이는 것” 쪽으로 흐른다. 도메인이 그 일반적인 패턴에서 벗어날 때마다, 그 차이를 명시해야 한다. 특히 다음 세 가지는 프롬프트나 규칙 파일 맨 위에 두는 것이 안전하다.

- 일반적인 관용구와 다른 ID 방식
- 일반적인 관행과 다른 데이터 흐름
- 표준 라이브러리 규칙과 다른 네이밍 컨벤션

이 세 가지는 취향 차이가 아니라 데이터 모델의 일부다. 다음 절에서는 도메인이 아니라 **데이터 자체의 경계값**에서 같은 패턴이 어떻게 반복되는지 본다.

9.3 생산성 점수 계산—엣지 케이스를 명시하면 AI가 개선된다

Mandai는 “주간 생산성 점수”를 표시한다. 점수 공식은 직접 정했다.

```
score = (
    completed_tasks × 10 + pomodoro_minutes
) / effective_time × correction
effective_time = 작업일 설정에서 "집중 시간"의 합계
```

이 공식을 SQL 쿼리로 만들어달라고 AI에게 요청했다. 첫 프롬프트는 단순했다—“이 공식대로 주별로 계산해라.”

루프마다 드러난 엣지 케이스

Claude가 만든 쿼리는 월요일부터 일요일까지 한 주가 딱 찬 경우에는 잘 동작했다. 하지만 실제 앱을 쓰는 사람은 완성된 한 주만 보지 않는다. 월요일만 지난 시점, 수요일 오후, 휴일이 낀 주처럼 중간에 확인하는 경우가 훨씬 많다.

루프를 돌면서 엣지 케이스가 하나씩 드러났다.

루프	발견한 문제	수정
1	항상 7로 나눔. 수요일에 보면 점수가 너무 낮음	현재 주는 경과된 날 수로 나누기
2	주말에도 일 안 했는데 분모에 포함됨	평일만 합산
3	휴일 설정 무시됨	휴일 테이블 LEFT JOIN
4	유효 시간이 0이면 나눗셈 오류	0일 때 0으로 처리

루프마다 쿼리가 4~5줄씩 늘었다. 가장 큰 차이는 분모를 어떻게 정하는지였다. 처음에는 무조건 7로 나눴지만, 수정 후에는 “지금 이 현재 주라면 경과된 요일까지, 지난 주라면 7일”로 분기 처리했다.

확인할 출력: 새 주의 월요일에 점수를 봤을 때 너무 낮지 않아야 한다. 분모가 1이어서 부풀려진 점수가 나오는지 확인한다.

이 경험에서 얻은 것

패턴은 단순하다. **AI는 명시된 것만 처리한다. 말하지 않으면 빠뜨린다.** 엣지 케이스는 개발자가 직접 찾아서 알려줘야 한다. 한 번 알려주면, AI는 그 패턴을 이후 작업에도 적용한다.

이 사례 이후, 다른 기능을 요청할 때 Claude가 스스로 이런 케이스를 먼저 확인하기 시작했다.

- “만약 분모가 0이면?”
- “만약 날짜 범위가 비어있으면?”
- “만약 설정 값이 null이면?”

엣지 케이스를 한 번 가르치면 AI가 다른 기능에서도 그 교훈을 적용한다. 단, 처음에는 개발자가 먼저 생각해서 던져줘야 한다.

주의: AI는 로직이 맞으면 만족한다. 극단적인 입력값에서도 의미 있는 결과가 나오지는 직접 테스트해야 한다.

다음 절에서는 우리 코드가 아니라 **프레임워크 자체의 변경**이 AI를 헛갈리게 하는 경우를 본다.

9.4 버전 혼동—Tauri 1.x와 2.x API를 섞을 때

AI 학습 데이터에는 Tauri 1.x와 2.x 예제가 모두 섞여 있다. 빠르게 진화하는 프레임워크일수록 인터넷에는 옛 튜토리얼이 그대로 남아 있고, AI는 그것까지 학습한다. 결과적으로 “Tauri 코드”라고만 말하면, 모델은 통계적으로 더 흔하게 본 v1 패턴을 자신 있게 골라 v2 코드 사이에 끼워 넣는다. 이런 코드는 컴파일은 정상적으로 되지만 실행하면 오류가 난다. 더 까다로운 경우는 개발 중에는 잘 돌아가다가 프로덕션 빌드에서만 깨지는 패턴이다.

전형적인 예: 창 관리 API

“최소화하여 트레이로”기능을 요청했더니, Claude가 돌려준 코드는 v1과 v2의 import가 섞여 있었다. v1에서는 `appWindow`라는 전역 객체를 썼지만, v2에서는 `getCurrentWindow()` 함수로 창을 얻는다. AI는 두 방식을 한 파일 안에서 동시에 사용했다.

TypeScript는 임포트를 정상으로 인식했기 때문에 컴파일은 통과했다. 하지만 실행하면 “이런 함수는 없다”는 오류가 났다.

실패하면 볼 것: “function not found”같은 오류는 거의 항상 버전 불일치 신호다. Rust 명령 쪽이 아니라 프론트엔드 import부터 확인해야 한다. 나는 이것 모르고 Rust 쪽을 4시간 들여다봤다.

버전 명시 프롬프트로 해결

해결책은 프롬프트에서 버전을 못박는 것이었다.

Tauri 2.x 사용 (1.x 아님), 트레이로 최소화를 구현하라. 주요 차이점:

- '@tauri-apps/api/window'의 `getCurrentWindow()`로 임포트, `appWindow` 전역 아님
- 기본 창 작업에는 `invoke()` 사용하지 말고, `window` 메서드 직접 사용
- 트레이 API는 `@tauri-apps/api/tray`, 구 `system-tray` 플러그인 아님
- 창 이벤트는 `window.listen()`, `appWindow.listen()` 아님

Tauri 2.x만 표시. API가 확실하지 않으면 추측하지 말고 질문하라.

이후로는 모든 import가 v2 경로로 통일됐다.

확인할 출력: `package.json`의 `@tauri-apps/api` 버전과 import 경로가 일치해야 한다. v1 패턴인 `appWindow` 전역이 남아 있으면 빨간 신호다.

v1 -> v2에서 자주 섞이는 패턴

영역	v1	v2
창 객체	appWindow 전역	getCurrentWindow()
파일 시스템	@tauri-apps/api/fs	@tauri-apps/plugin-fs + 플러그인 등록
HTTP	@tauri-apps/api/http	@tauri-apps/plugin-http 의 fetch()
트레이	system-tray 플러그인	@tauri-apps/api/tray

특히 파일 시스템은 함정이 많다. v2에서는 보통 `src-tauri/capabilities/*.json`에 권한과 경로 scope를 선언하고, Rust 쪽에서도 플러그인을 등록해야 한다. AI가 v1 import를 그대로 제안하면 대개 빌드·타입 검사에서 모듈 오류가 나거나, 권한이 빠졌다면 실행 시 호출 Promise가 거절된다. 오류를 숨기는 빈 `catch`가 있을 때만 사용자에게 조용한 실패처럼 보인다.

모든 Tauri 프롬프트는 다음 한 줄로 시작한다. **“TAURI 2.x 사용 - 모든 API 확인.”** 그리고 위 표의 핵심 항목을 함께 붙여 둔다. 이 가드 한 줄로 버전 혼동의 대부분이 사라졌다.

주의: AI가 학습한 데이터에는 옛 버전과 새 버전이 섞여 있다. 빠르게 진화하는 프레임워크에서 이 혼동이 특히 심하다. 반드시 정확한 버전을 명시하고, `import`가 `package.json`과 일치하는지 확인해라.

지금까지 네 가지 함정을 따로 봤다. 다음 절에서는 이 함정들을 **막는 도구 세 개**를 한꺼번에 정리한다.

9.5 막는 법: 프롬프트·리뷰·테스트 3종 세트

네 가지 실수는 발생 양상이 다르지만 막는 방법은 모두 같은 단계를 거친다. **시작 전에 명시하고**(체크리스트), **AI가 만든 코드를 다시 검토하게 하고**(리뷰 프롬프트), **사람이 직접 동작을 확인한다**(테스트 프로토콜). 세 가지는 같은 층위의 도구다. 어느 하나가 빠지면 나머지 둘이 잡지 못한 실수가 새어나간다.

이 절을 본격적으로 읽기 전에, 짧은 배경 하나가 도움이 된다.

가짜 생산성

속도가 빠르다고 해서 생산성이 높은 건 아니다.

Claude가 30초에 200줄을 생성한다. 빠르게 나아가는 것 같다. 그런데 AI가 **옛지 케이스를 처리하지 않은** 코드를 써냈기 때문에, 나중에 상태 동기화 버그 하나를 디버깅하는 데 7시간이 걸린다. AI는 기술 부채가 쌓이는 동안에도 진행되는 느낌을 준다.

- **전통적 개발:** 천천히 쓰고, 빠르게 디버깅한다.
- **AI 보조 개발:** 빠르게 쓰고, 천천히 디버깅한다.

Mandai 6주 동안 매주 하루 이상을 AI 실수 되돌리기에 썼다. 가드레일 없이 AI를 쓰면 순효과가 마이너스라는 사실을, 다음 표가 보여준다. 시간 수치는 정확한 합계가 아니라 패턴을 보여주는 참고용이다.

실수 유형	빈도	주요 원인
상태 동기화 문제	3개 기능	생명주기 사양 누락
버전 혼동	8회	API 훈련 데이터 혼합
옛지 케이스 맹점	여러 기능	정상 흐름만 테스트
도메인 모델 이탈	4개 기능	일반 패턴이 도메인 규칙 덮어쓰

이 표가 3종 세트의 출발점이다. 세 도구를 각각 보자.

도구 1 —프롬프트 체크리스트

CRUD(생성·조회·수정·삭제)보다 복잡한 기능은 예외 없이 이 체크리스트로 시작한다.

상태 관리:

- [] 프론트엔드와 백엔드 상태 동기화 계획 정의됨
- [] 충돌/절전 시 복구 동작 명시됨
- [] 옛지 케이스 나열: 데이터 없음, 처리 중 중단, 시간대

API 버전 관리:

- [] 프레임워크 버전 명시됨 (Tauri 2.x, SvelteKit 2.x, Svelte 5.x 등)
- [] 임포트 문이 현재 버전 문서와 일치함
- [] 훈련 데이터의 deprecated 패턴 없음

도메인 특수성:

- [] ``.cursor/rules/mandai.mdc``에 커스텀 ID 체계 문서화됨
- [] 비표준 규칙이 명시적으로 기록됨
- [] "일반 패턴"을 덮어쓰는 비즈니스 규칙 나열됨

테스트 시나리오:

- [] 정상 경로 테스트 케이스
- [] 엣지 케이스 시나리오 (비어있음, 부분, 오버플로우)
- [] 실패 복구 시나리오 (네트워크, 충돌, 데이터 손상)

이 체크리스트는 9.1~9.4의 네 가지 함정에 각각 대응된다. 각 항목이 비어 있다는 건, 그 함정의 빌미가 그대로 남아 있다는 뜻이다.

도구 2 —리뷰 프롬프트

체크리스트로 시작했더라도, AI는 여전히 실수한다. 그래서 코드를 받은 뒤에 AI 자신에게 다시 검토를 시킨다. **언제 돌리는가?** AI가 만든 기능을 2~3번 수정한 직후, 그러나 손으로 테스트하기 전이 가장 효율적이다. 너무 일찍 돌리면 아직 안 짠 부분에 대해 평을 늘어놓고, 너무 늦게 돌리면 이미 잡힌 버그들 외엔 잡을 게 없다.

이 기능의 AI 함정을 검토하라:

상태 동기화

- 앱 재시작/충돌 상황에서도 상태를 안전하게 복구하는가?
- 전체 생명주기에서 프론트엔드와 백엔드 상태가 일관되는가?
- 사용자가 처리 중에 앱을 백그라운드/절전 하면 어떻게 되는가?

API 버전 정확성

- 모든 임포트가 [FRAMEWORK_VERSION] API를 사용하는가?
- 구버전처럼 보이는 패턴이 있는가?
- deprecated 메서드나 구식 임포트 경로가 있는가?

엣지 케이스 커버리지

- 데이터가 없을 때/비어있을 때 어떻게 되는가?
- 경계 조건 (자정, 월말 등)을 어떻게 처리하는가?
- 시간대, 공휴일, 업무 일정 변동이 커버되는가?

도메인 모델 정렬

- 코드가 커스텀 ID 체계와 규칙을 따르는가?
- "일반 모범 사례"가 도메인 특화 규칙을 덮어쓰는가?
- Cursor Project Rules 준수 확인

문제가 발견되면, 구체적인 수정 사항과 업데이트된 프롬프트를 제안하라.

리뷰 템플릿이 Claude가 놓친 버그를 잡은 예시 하나만 보자. AI가 만든 타이머 복원 로직은 DB에 저장된 `remaining_minutes` 값을 그대로 믿고 타이머를 다시 시작했다. 문제는 그 값이 절전 시간 동안 흐른 시간을 반영하지 않는다는 점이다. 30분 동안 절전 상태였다면, 실제 남은 시간보다 더 큰 값이 표시된다.

리뷰 프롬프트를 돌리니 곧바로 이 문제를 지적했다. “저장된 값을 신뢰하지 말고, `session.start_at`에서 경과 시간을 다시 계산하라”는 응답이었다. 9.1에서 다섯 번을 해맨 끝에 얻었던 결론과 정확히 같다.

리뷰 도구가 효과를 내려면 한 가지 전제가 필요하다. **AI가 쓴 코드를 사람이 어느 정도는 읽을 수 있어야 한다.** 줄마다 꼼꼼히 검토하는 게 아니라, 위험한 패턴을 알아보는 정도면 충분하다. 다음과 같은 패턴이 보이면 멈춰서 의심한다.

- **누락된 `await`:** `invoke(...)` 같은 비동기 함수 앞에 `await`가 빠지면, 결과 대신 Promise 객체가 들어간다.
- **중요한 경로에 오류 처리 없음:** `save_data`가 실패해도 곧바로 “성공” 메시지를 보여주는 코드.
- **반응형 업데이트 없는 상태 변이:** 객체를 직접 수정하면 화면이 갱신되지 않는 프레임워크(Svelte 등)에서, 배열의 한 원소를 직접 바꾸는 코드.

이 세 가지는 도메인 지식이 없어도 알아볼 수 있고, 가장 자주 등장한다. AI 코드를 줄 단위로 다 읽지 않더라도, 이 패턴을 알아채는 눈이 있어야 리뷰 프롬프트가 가리키는 문제를 판단할 수 있다.

주의: AI 코드 디버깅은 사람이 쓴 코드 디버깅보다 어렵다. AI는 그럴듯해 보이지만 미묘하게 잘못된 로직을 만든다. 프로덕션에 올리기 전에 잡으려면 리뷰 습관을 반드시 만들어야 한다.

도구 3 —테스트 프로토콜

체크리스트와 리뷰 프롬프트를 통과한 코드라도, 사람이 직접 돌려봐야 한다. 다음 13분은 Mandai의 작은 기능에 적용한 최소 점검 타임박스다. 결제·삭제·마이그레이션·보안 경계처럼 위험한 기능에는 더 긴 테스트와 자동화가 필요하다.

1단계: 동작 테스트 (3분). 빌드하고 실행해서 기본 흐름이 작동하는지 확인한다. 안 되면 수정 루프로 돌아간다.

2단계: 엣지 케이스 탐색 (5분). 체크리스트를 따라 테스트한다. 요일을 바꾸고, 시간대를 바꾸고, 데이터를 지우고, 버튼을 빠르게 세 번 클릭한다.

3단계: 상태 복구 테스트 (5분). 앱을 강제 종료한 뒤 재시작한다. 데이터가 복원되는가? 중간 상태에 멈춰 있는가? 그렇다면 앱이 그 상황을 제대로 처리하는가?

Mandai에서 실제로 쓴 시나리오는 이런 식이다. 1단계에서는 포모도로 시작 후 30초 카운트다운 확인, 작업 완료 후 목록 갱신 확인, 만다라 하위 목표 저장 확인. 2단계에서는 포모도로 중 앱 최소화 후 2분 뒤 복원, 빈 제목으로 작업 완료 시도, OS 시간대 변경 후 재시작. 3단계에서는 포모도로 중 앱 강제 종료 후 재시작, 만다라 편집 중 강제 종료, 그리고 DB 파일이 손상된 상태에서의 재시작. 세 단계를 일관되게 적용하면, AI가 만든 버그의 상당수를 사용자보다 먼저 잡아낼 수 있다.

주의: AI가 작성한 코드는 단독으로 테스트할 때는 멀쩡해도, 실제로 사용하다 보면 깨지는 경우가 많다. 요청한 함수 하나만 테스트하지 말고, 전체 사용자 흐름을 테스트해라.

세 도구의 관계

세 도구는 같은 함정을 다른 시점에 막는다.

- **체크리스트** —코드를 쓰기 전에, 빌미를 줄인다.
- **리뷰 프롬프트** —코드가 만들어진 직후, AI 자신에게 한 번 더 검토시킨다.
- **테스트 프로토콜** —사람이 마지막 단계에서 실제 동작을 본다.

세 가지 중 어느 하나만 써도 효과는 있지만, 같이 쓸 때 함정이 가장 줄어든다.

마무리 박스: 이 장에서 반드시 기억할 것

Mandai를 개발하면서 디버깅에 상당한 시간을 썼다. 이 장에서 꼭 기억할 것은 네 가지다.

첫째, AI는 네 가지 실수를 반복한다. 프론트엔드와 백엔드의 상태가 어긋나는 상태 동기화 문제, 옛 API와 새 API를 섞는 버전 혼동, 정상 흐름만 신경 쓰고 예외 상황을 빠뜨리는 옛지 케이스 맹점, 그리고 도메인 고유의 규칙을 무시하고 일반적인 패턴을 적용하는 도메인 모델 이탈. 패턴을 알면 미리 막을 수 있다.

둘째, 막는 방법은 사전에 명시하는 것이다. 코드를 요청하기 전에 생명주기(언제 시작하고, 멈추고, 복원되는지), 정확한 버전, 옛지 케이스 목록, 도메인 규칙을 프롬프트에 적어 둔다. 한 줄의 명시가 몇 시간의 디버깅을 줄여준다.

셋째, 검증은 3종 세트로 만든다. 시작 전 체크리스트, 작성 직후 리뷰 프롬프트, 마지막에 3단계 테스트. 어느 하나만 써도 효과가 있지만, 세 가지를 같이 쓰면 함정이 가장 줄어든다.

넷째, AI로 아낀 시간은 디버그로 돌아온다. 가드레일 없이 AI를 쓰면 코드는 빨리 쌓이지만, 그만큼 시간이 늦게 새어 나간다. 흐름은 이렇게 바뀌어야 한다.

- **전통적 개발:** 생각 -> 코드 -> 디버그
- **AI 보조 개발:** 생각 -> 제약 -> 프롬프트 -> 검증 -> 적게 디버그

“제약”단계 하나를 추가하는 것이 디버그 시간을 줄이는 핵심이다.

Mandai 파트가 끝났다. 다음은 **readbooks.ai**다. 2025년에 Next.js로 만든 PDF 번역 앱을 Tauri 2로 다시 구현한다. 이번 루프는 성격이 다르다. 코드 버그를 잡는 루프가 아니라 **출력 품질**을 검증하는 루프다. 다른 AI로 AI 번역 결과를 검증하는 방식이다. 이 장에서 만든 “사전 명시 + 사후 검증”의 사고틀이, 다음 장에서는 번역 품질이라는 또 다른 형태의 출력에 적용된다.

10 장. readbooks.ai —AI 출력 품질을 설계하는 프롬 프트 엔지니어링

Part III —readbooks.ai (10~11장) Mandai가“코드의 옳고 그름”을 다뤘다면, Part III는 **출력 품질**을 다룬다. AI가 36개 언어로 번역한 결과가 맞는지 어떻게 판단하는가—이 질문이 파트 전체를 이끈다. 10장은 프롬프트 설계, 11장은 검증 파이프라인이다. 첫 회독에서는 36개 언어라는 규모보다“AI 출력도 제품이면 기준과 검증이 필요하다”는 점을 먼저 잡는다.

AI의 **출력**이 제품인 앱은 피드백 방식이 다르다. 코드라면 타입 오류나 런타임 예외가 금방 드러난다. 하지만 번역 결과는 문법적으로 자연스러워 보여도 의미가 틀릴 수 있다. 컴파일러도, 테스트도 그 차이를 잡아주지 못한다. 그래서 이 장의 핵심 질문은 기능 구현이 아니다. **번역 품질을 프롬프트(AI에게 주는 지시문)로 어떻게 설계할 것인가**다.

readbooks.ai는 PDF를 36개 언어로 번역한다. 그런데 결정적인 제약이 하나 있다. **36개 언어 중 내가 직접 읽을 수 있는 언어는 4개뿐이다**. 나머지 32개 언어는 번역 결과가 맞는지 눈으로 확인할 수 없

다. 이 한 가지 사실이 이 장의 모든 결정을 끌고 간다. 검토자 역할의 AI, 자동 형식 검사, 다음 장의 역 번역까지—전부 “읽지 못하는 언어를 제품 품질로 유지하려면 무엇이 필요한가”에 대한 답이다.

프롬프트가 잘못되면 한 언어만 문제가 생기는 게 아니다. 같은 프롬프트를 사용하는 36개 언어 전체가 같은 방식으로 틀어진다. 프롬프트 품질이 곧 제품 품질이다.

배경: readbooks.ai는 2025년에 Next.js로 만든 PDF 번역 앱이다. 현재는 서비스를 운영하지 않는다. 이 책에서 그 소스를 Tauri 2 + SvelteKit 2 + Svelte 5로 재구현했다. 마이그레이션 자체는 이 장의 주제가 아니다. 이 장의 핵심은 재구현 과정에서 설계한 번역 프롬프트 시스템이다.

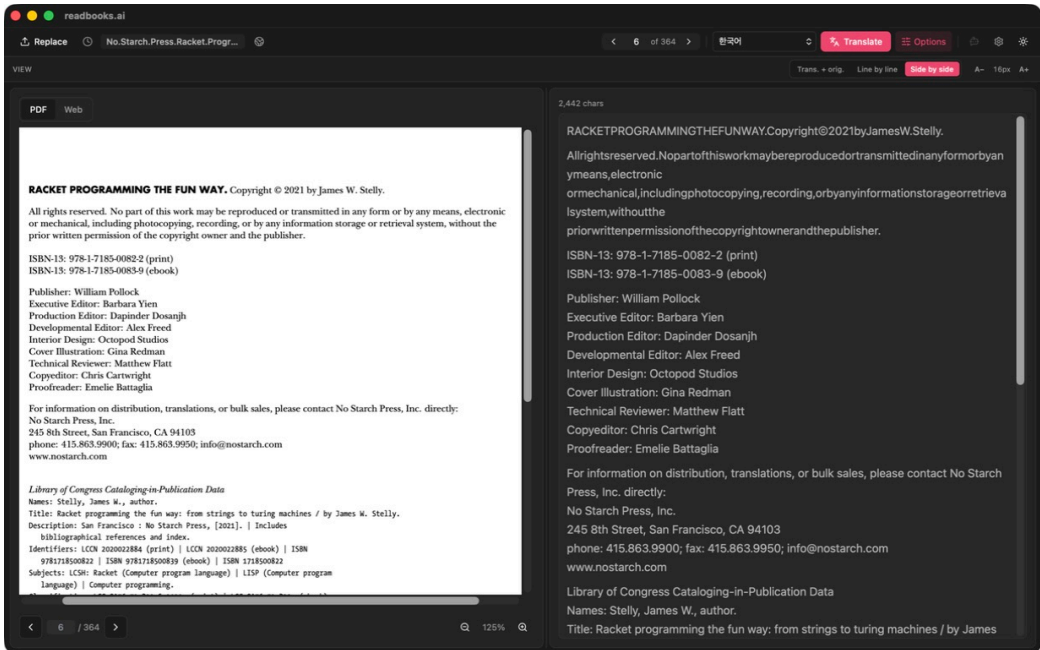


그림 10-1. 실제로 실행한 readbooks.ai의 PDF 원문과 추출 텍스트 양쪽 보기 화면.

이 장의 흐름

이 장은 번역 프롬프트가 어떻게 실패했는지에서 시작해, 품질 기준과 검증 가능한 제약으로 다듬어가는 흐름이다.

- **10.1 배경** —DeepL 한 곳에 의존하던 구조를 왜 LLM 조합으로 바꿨는지.
- **10.2 첫 번째 프롬프트** —“그냥 번역해줘”가 왜 실패하는지.

- **10.3 언어별 분기** —하나의 프롬프트로 36개 언어를 처리할 수 없는 이유.
- **10.4 메타 루프** —AI로 프롬프트 자체를 다듬는 방법.
- **10.5 품질 기준** —“좋은 번역”을 검사 가능한 체크리스트로 바꾸기.
- **10.6 루프 정리와 요약.**

학습 목표

- AI 출력이 제품일 때 개발 루프가 어떻게 달라지는지 이해한다.
- 번역 품질을 결정하는 제약 조건을 어떻게 설계하는지 익힌다.
- 언어별 프로파일을 만드는 과정을 따라간다.
- AI를 번역자, 검토자, 언어 전문가 세 가지 역할로 나눠 쓰는 방법을 배운다.

10.1 배경

readbooks.ai는 처음에 DeepL API 하나로 36개 언어를 번역했다. 결과는 대체로 “읽을 만한” 수준이었다. 하지만 기술서에서 “읽을 만하다”와 “의미가 정확하다”는 다른 말이다.

문장이 자연스러워도 문제는 생긴다. 코드 블록 설명이 틀리거나, API 이름을 현지화해 버리거나, 명령어 일부를 번역해버리면 독자는 원문보다 더 헷갈린다. 이 책에서는 그 구조를 Tauri 2 + SvelteKit 2 + Svelte 5로 다시 구현했다. 다만 여기서 중요한 것은 Next.js에서 Tauri 2로 옮긴 과정 자체가 아니다. 옮기는 과정에서 드러난 문제, 즉 **AI 출력물을 제품 품질로 관리하려면 어떤 기준을 먼저 세워야 하는가**다.

이 장의 핵심 질문은 이것이다. **36개 언어에 걸쳐 번역 품질을 어떻게 일관되게 유지할 것인가**. 이 질문에 답하려면 “좋은 모델을 고른다”는 것만으로는 부족하다. 언어별 프로파일, 보존해야 할 기술 요소, 검토자 역할의 프롬프트까지 함께 설계해야 한다.

번역을 위한 Provider 추상화

처음부터 한 가지 결정을 내렸다. **단일 번역 provider(번역을 처리하는 엔진)에 종속되지 않는다**. Provider는 Claude나 DeepL 같은 외부 서비스일 수도 있고, 사용자의 컴퓨터에서 직접 실행되는 소형 모델일 수도 있다.

번역 품질은 언어와 문서 유형에 따라 달라진다. provider마다 잘하는 영역도 다르다. DeepL이 안정적인 언어가 있고, Claude가 문맥을 더 잘 유지하는 언어가 있다. 그래서 앱은 Claude, OpenAI, Gemini, DeepL과 데스크톱 온디바이스 모델을 하나의 인터페이스 뒤에 두고 바꿔 쓸 수 있게 만들었다.

실제 코드는 readbooks.ai 예제의 `tauri/src-tauri/src/lib.rs`와 `tauri/src-tauri/src/on_device.rs`에서 확인할 수 있다. Svelte 화면은 모든 엔진을 같은 `translate_with_provider` Tauri 명령으로 호출한다. Rust는 선택된 값이 `openai`, `anthropic`, `gemini`, `deepl`, `on_device` 중 무엇인지에 따라 외부 API 또는 기기 내부 런타임으로 분기한다.

구조는 단순하다. 특정 언어에서 품질이 떨어지면 번역 로직 전체를 건드리지 않고, 그 언어에 연결된 provider만 바꿀 수 있게 추상화한다.

주의사항: Provider를 바꾸면 용어 일관성이 깨질 수 있다. 1장을 DeepL로, 2장을 Claude로 번역하면 같은 기술 용어가 다르게 옮겨질 수 있다. Provider가 바뀌어도 용어가 일관되게 유지되도록 캐시를 만들어야 한다.

온디바이스, 온디맨드, 오프라인은 다른 말이다

세 용어는 비슷하게 들리지만 가리키는 대상이 다르다. 이 책에서는 다음처럼 구분한다.

- **온디바이스(on-device):** 추론이 사용자 기기 안에서 실행된다. 문장이 번역 서버로 전송되지 않는다.
- **온디맨드(on-demand):** 모델을 항상 메모리에 두지 않고 필요할 때 다운로드하거나 로드한다. 모델의 배포·수명 주기를 설명하는 말이다.
- **오프라인(offline):** 모델 파일을 준비한 뒤에는 네트워크 연결 없이 추론할 수 있다.

readbooks.ai의 새 옵션은 세 특성을 함께 쓴다. 설정에서 On-device를 선택하고 GGUF 모델 파일을 지정하면 Rust가 앱 프로세스 안에서 `llama.cpp`를 실행한다. 앱을 켤 때는 모델을 읽지 않는다. 사용자가 첫 번역을 요청하는 순간 모델을 메모리에 올리고, 다음 요청부터 같은 인스턴스를 재사용한다. 즉 **온디바이스 추론을 온디맨드로 로드하며, 준비가 끝난 뒤에는 오프라인으로 동작한다**. localhost 서버와 API 키는 필요 없다.

```
[Svelte: On-device] -> invoke -> [Tauri Rust: GGUF 로드/재사용]
                        -> [llama.cpp] -> [결과 + DuckDB]
```

핵심 분기는 짧다. 클라우드 Provider는 기존 HTTP 함수를 호출하지만, on_device는 모델 경로를 확인한 뒤 별도 모듈로 넘긴다.

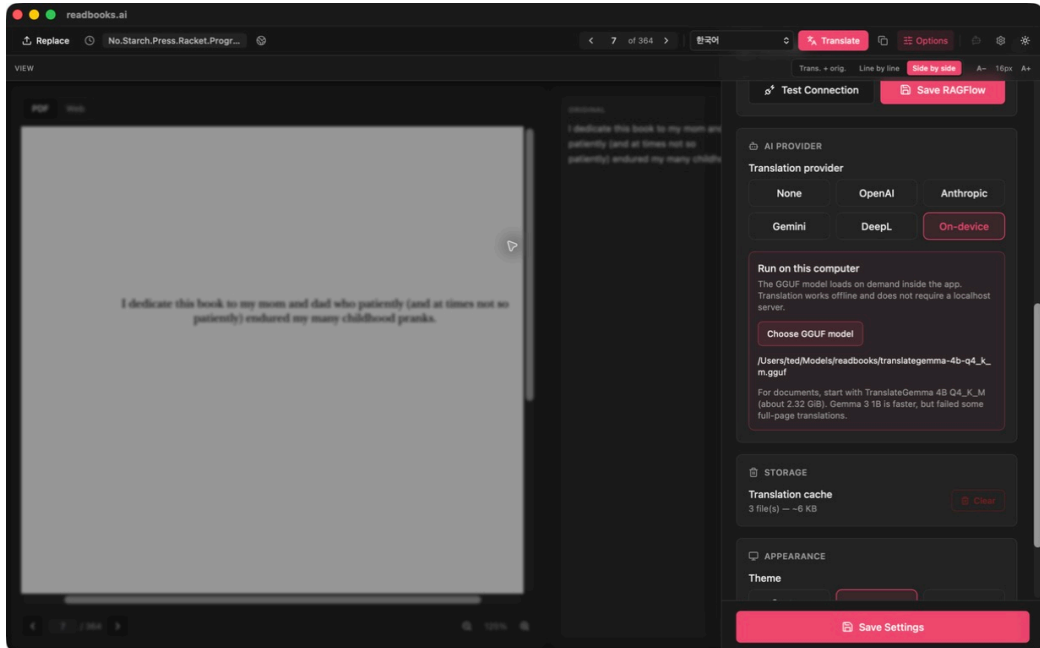


그림 10-2. TranslateGemma 4B Q4_K_M GGUF를 선택한 readbooks.ai 설정 화면. 모델은 앱 안에서 필요할 때 로드되며 별도 로컬 서버를 요구하지 않는다.

```
match provider.as_str() {
  "openai" => do_translate_openai(/* ... */).await,
  "deepl" => do_translate_deepl(/* ... */).await,
  "on_device" => {
    let path = model.ok_or("Settings에서 GGUF 모델을 선택하세요");
    on_device::translate(path, system_prompt, text,
      target_lang).await
  }
  _ => Err("지원하지 않는 Provider".into()),
}
```

`on_device::translate()`는 계산량이 큰 추론을 `spawn_blocking`으로 보낸다. 그렇지 않으면 모델이 토큰을 생성하는 동안 Tauri의 비동기 작업 스레드가 막혀 화면과 다른 명령의 응답성이 떨어질 수 있다. 내부에서는 GGUF의 채팅 템플릿으로 시스템 지시와 원문을 조립한다. 템플릿이 없는 Gemma 계열 GGUF에는 `<start_of_turn>` 형식의 사용자·모델 턴을 직접 구성하는 폴백을 적용한다. 그런 다음 4,096토큰 컨텍스트 안에서 최대 512토큰을 생성한다. 한 페이지가 한도를 넘으면 무리하게 잘라 넣지 않고 더 작은 구간으로 나누라는 오류를 반환한다.

소형 모델이 데스크톱 번역 옵션이 된 이유

예전에는 로컬 LLM을 데스크톱 앱에 넣으려면 모델 크기와 메모리 사용량이 지나치게 컸다. 지금은 4비트 양자화 GGUF가 보편화되면서 4B급 번역 특화 모델도 일반 노트북에서 시험할 수 있다. 이 예제의 문서 번역 시작점은 **TranslateGemma 4B Q4_K_M**이다. 텍스트 전용 GGUF는 2.32GiB이고 검증한 SHA-256은 e7afd56008be424d2ba36ed3b371d40039ae15fa467e8cf71442570d842b1119이다. 특정 모델 파일은 앱이나 Git에 묶지 않는다. 사용자가 모델 저장소의 Gemma 이용 조건과 해시를 확인한 뒤 파일 선택기로 가져온다.

작다고 모두 같은 용도에 맞는 것은 아니다. Gemma 3 270M은 일부 영어 문장을 번역하지 않고 되돌렸고, 1B는 EarLoop의 짧은 회화를 번역했지만 readbooks.ai의 PDF 페이지에서는 한글이 아닌 혼합 문자를 만들었다. TranslateGemma 4B는 Did you want to get a coffee?를 커피 한 잔 어떠세요?로, 한 문장짜리 한정 페이지를 자연스러운 한국어로 번역했다. 이 비교는 파이프라인 점검용 모델, 짧은 회화용 모델, 문서 번역용 모델을 구분해야 한다는 뜻이다.

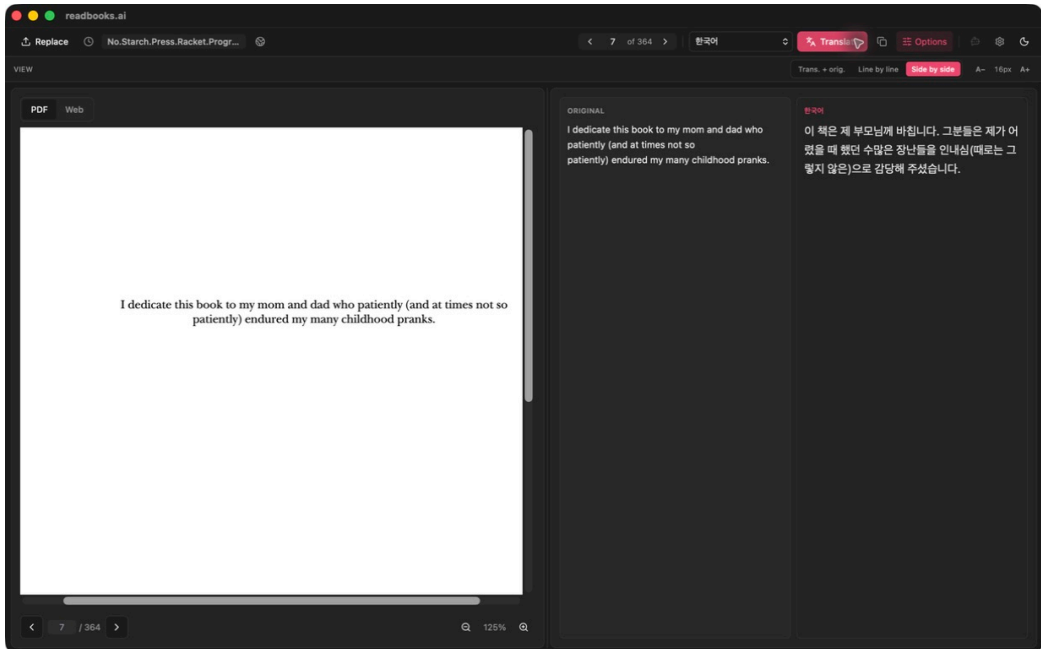


그림 10-3. TranslateGemma 4B를 앱 프로세스에서 직접 실행해 PDF 한정 문장을 한국어로 번역한 화면.

실제 Rust 통합 테스트는 한글 포함 여부만 보지 않고 핵심 의미인 커피가 남았는지도 검사한다. 앱도 한국어 요청에 한글이 하나도 없으면 성공으로 표시하지 않고 더 적합한 GGUF를 고르라고 안내한다.

이 검사는 자연스러움 전체를 보증하지 않으므로 긴 문서는 다음 장의 품질 평가를 거쳐 더 큰 모델이나 클라우드 Provider로 전환할 수 있어야 한다.

Ollama는 모델을 찾고 내려받는 데 편리하지만 앱의 실행 의존성은 아니다. 실제 앱은 llama-cpp-2가 GGUF를 직접 연다. 이번 검증에서 Ollama용 vision blob은 표준 llama.cpp가 요구하는 gemma3.attention.layer_norm_rms_epsilon 메타데이터가 없어 직접 로드되지 않았다. 그래서 텍스트 전용 llama.cpp 변환본을 별도로 내려받아 SHA-256을 확인했다. “GGUF 확장자”만 보고 호환성을 가정하지 말고, 앱이 사용하는 런타임으로 실제 로드와 생성까지 검사해야 한다.

여기서 “작다”는 “항상 충분히 정확하다”는 뜻이 아니다. 소형 모델은 설치 용량, 메모리, 첫 로드 시간을 줄이는 대신 다음 조건에서 품질이 빠르게 떨어질 수 있다.

- 한 번에 긴 PDF 페이지를 넣을 때
- 학습 자료가 적은 언어쌍을 번역할 때
- 코드·표·각주가 섞인 복잡한 문서를 처리할 때
- 용어집과 긴 문맥을 동시에 지켜야 할 때

따라서 설정에는 클라우드 Provider를 그대로 남겨 둔다. 민감한 문서나 네트워크가 없는 환경에서는 온디바이스 모델을 쓰고, 긴 문서나 높은 품질이 필요한 작업에서는 검증된 외부 모델을 선택할 수 있다. Provider 추상화의 목적은 어느 한쪽을 정답으로 정하는 것이 아니라 **개인정보·비용·속도·품질의 우선순위에 따라 실행 엔진을 교체할 수 있게 하는 것이다.**

배포 주의사항: 예제는 임의의 GGUF 파일을 선택하게 한다. 제품 버전이 모델을 직접 내려받는다면 다운로드 전에 크기와 라이선스를 보여 주고, 임시 파일에 받은 뒤 SHA-256을 검증해야 한다. 검증된 파일만 Application Support로 옮기고 모델 ID·버전·해시를 번역 캐시 키에 포함해야 한다.

캐시 키에서 모델 정보를 빼면 모델을 바꾼 뒤에도 이전 모델의 번역이 새 결과처럼 나타날 수 있다. 실제 예제는 PDF 식별자, 페이지, 목표 언어, 원문 해시, provider, 모델 경로를 DuckDB 캐시 키로 사용한다. 예전 localStorage 캐시는 정리용으로만 남겨 두며 번역 결과를 읽어 오는 경로에는 사용하지 않는다. 덕분에 다른 GGUF를 선택하면 같은 페이지도 새 모델로 다시 번역하고, 검증된 결과만 해당 모델의 캐시에 저장한다.

번역 기능 자체는 어렵지 않았다. 진짜 문제는 **품질 판단**이었다. 앞에서 말한 대로 36개 언어 중 32개는 내가 읽을 수 없다. 읽지 못하는 언어의 품질을 제품 수준으로 유지하려면, 프롬프트가 번역자에게 맡길 범위와 절대 건드리면 안 되는 범위를 명확히 나눠야 한다.

10.2 첫 번째 프롬프트—모든 것을 놓친 초안

첫 번째 프롬프트는 짧았다. 번역 작업이니 입력 텍스트와 목표 언어만 주면 모델이 알아서 처리할 것이라 생각했다. 내가 처음 쓴 프롬프트는 이 한 줄이었다.

“다음 영어 텍스트를 {언어}로 번역해줘. 번역만 출력해줘. 설명 없이.”

36개 언어로 실행했을 때 결과는 겉으로는 멀쩡해 보였다. 문단은 자연스러웠고, 대부분의 문장은 목표 언어로 바뀌었다. 그런데 실제 결과물을 열어보면 기술 문서에서 가장 중요한 부분이 반복해서 깨졌다.

- 코드 블록 안의 변수명이 번역됨 (`const userName -> const 사용자이름`)
- 인용부호가 바뀜 (`"foo" -> 「foo」`)
- 숫자 형식이 지역화됨 (`1,000 -> 1.000`)

왜 AI는 이렇게 동작하는가

원인은 하나다. **지시가 없으면 AI는 일반 번역 관례를 따른다.** 일반 번역 관례에서는 모든 텍스트를 자연스럽게 옮기는 게 정답이다. 문학 번역이라면 옳다. 기술 문서에서는 오류가 된다. 파일명, 코드, 명령어, API 이름은 독자가 그대로 복사해서 실행해야 하는 정보이기 때문이다.

이 원리는 이 장 전체를 관통한다. 프롬프트는 “AI에게 무엇을 하라”고 시키는 도구가 아니라, **AI가 기본값으로 따르려는 관례 중 어디서 멈추라고** 알려주는 도구다. 기술 문서 번역 프롬프트는 “자연스럽게 번역하라”에서 출발하면 안 된다. 먼저 보존해야 할 구조를 정의하고, 그 범위 안에서만 문장을 바꾸도록 요구해야 한다.

실제 실패 사례—한 블록에 다섯 가지 오류

다음 영어 발췌를 첫 번째 프롬프트로 한국어 번역에 돌려봤다.

원본: ~markdown Heading: Chapter 3: Database Setup Set up the database. `config.js` file.

```
const config = { host: "localhost", port: 5432, database: "readbooks"
};
```

Connection timeout is 30,000ms. ~

나온 번역: ~markdown 제목: 3장: 데이터베이스 설정 데이터베이스. 설정.js 파일을 구성하세요:

```
const 설정 = { 호스트: "localhost", 포트: 5432, 데이터베이스: "readbooks" };
```

연결 시간 초과는 30.000ms로 설정됩니다. ~

작은 코드 블록 하나에서 다섯 가지 문제가 동시에 나타났다. 파일명·변수명·JSON 키가 모두 번역됐고, 숫자 30,000이 유럽식 30.000으로 바뀌었으며, 결과적으로 코드의 색상 강조(syntax highlighting)까지 깨졌다.

주의사항: 간단한“이것을 번역해줘”프롬프트는 **모든 텍스트를 번역 대상으로 취급한다**. 기술 문서에는 반드시 그대로 유지해야 할 구간들(코드, URL, 명령어)이 있다. 제약조건을 명시하지 않으면 AI는 모든 것을 지역화한다.

1차 수정—보존 범위부터 고정한다

첫 수정은 품질을 더 좋게 만들라는 추상적인 요구가 아니었다. 코드 블록, 인라인 코드, 숫자, URL처럼 보존해야 할 대상을 먼저 고정하고, 그 밖의 산문만 번역하도록 범위를 나눴다. 핵심은 다섯 줄짜리 **시스템 프롬프트**(모든 요청 앞에 항상 따라붙는, AI의 역할과 규칙을 정해주는 지시문)다.

당신은 기술 문서를 번역합니다. 규칙:

- `` fenced 코드 블록 내부의 내용은 번역하지 마세요. 그대로 복사하세요.
- `인라인 코드`는 번역하지 마세요 —원본 그대로 유지하세요.
- 인용부호, 숫자 형식, 날짜는 소스에 있는 대로 보존하세요.
- URL, 이메일, DOI: 원본 그대로 유지하세요.
- 번역만 출력하세요. 설명, 서문 없이.

실행해보니 코드 블록 보존은 작동했다. 그러나 새로운 문제가 생겼다. **인용부호 안의 영어 문장이 소스 언어로 남아 있었다**. AI가“인용부호를 보존하라”를“인용부호 안의 모든 것을 보존하라”로 해석했기 때문이다.

2차 수정—“기호”와“내용”을 분리한다

내가 원하는 것은 그 반대에 가깝다. 인용 구두점은 남아 있어야 하지만, 인용된 내용은 일반 문장처럼 번역되어야 한다. 그래서 다음 두 줄을 추가했다.

- 구두점 기호 (" , ' , «», „", 등)는 그대로 보존합니다.
하지만 인용부호 안의 텍스트는 번역합니다.
- 숫자 형식 (십표, 소수점)은 소스에 있는 대로 보존합니다.

같은 문장을 다시 돌려보자.

원본: The system returns:

```
{"status": "success", "message": "File uploaded"}
```

한국어: 시스템은 다음 값을 반환합니다.

```
{"status": "success", "message": "파일이 업로드되었습니다"}
```

JSON 구조와 키는 그대로 유지되었고, 사람이 읽는 메시지만 번역되었다. 정확히 원했던 동작이다.

여기서 얻은 교훈은 단순하다. **“X를 변경하지 마라”는 제약조건은 X의 정확한 범위를 함께 명시해야 한다.** 범위가 모호하면 AI는 자신만의 해석을 만들고, 그 해석은 대체로 일반 번역 관례를 따른다.

10.3 언어별 분기—“36개 언어를 위한 하나의 프롬프트”환상

한 언어(한국어)에서 제약을 잡는 데도 두 번의 수정이 필요했다. 그렇다면 36개 언어를 동시에 다뤄야 한다면 어떻게 될까. 처음에는 프롬프트 하나로 모두 처리할 수 있다고 생각했다. 실제로 해보니 달랐다. 같은 영어 문장을 번역해도 독일어, 러시아어, 아랍어, 중국어는 서로 다른 방식으로 틀린다. 언어군마다 제약조건을 따로 정해야 했다.

먼저 어떤 종류의 문제가 언어별로 생기는지 정리해두자.

- 한국어/일본어: 영어 고유명사는 관례상 영어로 유지 (예: “React”는 “React”로)
- 독일어: 명사 첫 글자 대문자 규칙을 LLM이 일관되게 지키지 못함.
- 러시아어: 정관사가 없는 언어인데 LLM이 없던 지시어(“이”, “그”)를 끼워 넣음.
- 아랍어/히브리어: 오른쪽에서 왼쪽으로 읽는 언어(RTL)라 markdown 렌더링에 영향을 줌.
- 중국어: 간체와 번체에서 같은 영어 단어가 다른 용어로 번역되어야 함.

예를 들어 영어“The react component handles state management through hooks.”를 독일어로 번역하면 AI가 react, komponent, hooks를 모두 소문자로 출력하는 경우가 있다. 독일어 맞춤법 대신 영어의 대문자 규칙을 그대로 따른 것이다. 러시아어의 경우 원문에 없던 지시어(“этот”)가 끼어들어 “특정 **혹**”이라는 의미가 새로 생긴다. 자연스러움을 높이려다 원문 의미를 바꿔버린다.

주의사항: 언어군을 묶어서 생각하면 개별 언어 수준에서 규칙이 깨진다. 포르투갈어와 스페인어처럼 가까운 언어도 기술 용어 선호가 다르다.

해결—기본 프롬프트 + 언어별 프로파일

해결책은 시스템 프롬프트 하나에 **언어별 추가 블록**을 붙이는 방식이었다. 공통 규칙은 기본 프롬프트에 두고, 언어마다 달라지는 관례만 짧은 프로파일로 분리한다.

```
const base = `당신은 기술 문서를 번역합니다. 규칙: ...`;

const languageProfiles: Record<string, string> = {
  ko: `~ 영어 고유명사(React, npm, API)는 영어 그대로 유지`,
  de: `~ 독일어 명사 대문자화 필수\n~ 복합명사는 한 단어로 유지`,
  ar: `~ RTL 텍스트 방향 적용. 코드 블록은 LTR 유지`,
};
```

각 프로파일은 10줄 이하로 유지했다. 규칙이 길어지면 모델이 중요한 제약을 빠뜨리거나 서로 충돌하는 지시를 동시에 따르려 한다.

실제 프로파일은 어떻게 생겼는가

위의 한 줄짜리 예시만 보면 감을 잡기 어렵다. 실제로 사용한 독일어 프로파일을 전부 보여주면 다음과 같다.

독일어 (de) 프로파일:

- 명사 첫 글자는 항상 대문자로 표기한다.
영어 외래어도 명사로 쓰이면 대문자.
예: Hook → Hook, component → Komponente.
- 영어 고유명사(React, npm, API)는 그대로 유지한다.
명사로 쓰일 때는 대문자 규칙을 적용한다.
- 복합명사는 한 단어로 붙여 쓴다 (Datenbankverbindung, 하이픈 ×).
- 격식체 (Sie/Ihnen) 사용. du 형태 금지.
- "you"는 문맥에 따라 Sie 또는 수동태로.

모든 언어 파일을 외우기보다 같은 템플릿을 적용한다. 각 언어에 (1) 문체 (2) 금지할 오역 (3) 숫자·단위 표기 (4) 고유명사 규칙 (5) 출력 형식을 적고, 실제로 틀린 문장 하나와 올바른 문장 하나를 붙인다. 위 다섯 줄의 예가 이 형식이다. 언어별 규칙은 해당 언어 검토자가 확인한 범위만 사용하고 36개 언어의 정확성을 일괄 보장하지 않는다.

언어 프로파일 구축: AI를 연구 어시스턴트로 쓰기

36개 언어를 혼자 조사하는 대신, Claude를 도메인 전문가처럼 활용했다. 핵심은 “프로파일을 만들어 줘”라고 통째로 맡기지 않은 것이다. 먼저 내가 필요한 기준을 정했다(타이포그래피, 기술 용어 관례, 형식 차이). 그다음 언어별로 자주 틀리는 지점을 물어보고, 돌아온 답변에서 실제 프롬프트에 넣을 항목만 골랐다.

이 방식으로 2시간 안에 36개 언어 프로파일을 정리할 수 있었다. 단, 결과를 그대로 믿지는 않았다. 언어마다 짧은 샘플을 직접 돌려보고, 코드 블록·숫자·용어가 잘 유지되는지 확인했다. **AI는 코드 작성뿐 아니라 도메인 지식 조사에도 쓸 수 있다. 하지만 최종 기준을 정하고 결과를 검증하는 일은 여전히 제품을 만드는 사람이 해야 한다.**

주의사항: 복잡한 언어 프로파일은 규칙끼리 충돌을 일으킨다. “격식체 사용”과 “자연스럽게 번역”을 함께 명시하면 AI가 흐름보다 격식을 택한다. 규칙에 우선순위를 매겨라.

10.4 AI와 함께 프롬프트 설계—메타 루프

이 장에서 가장 중요한 내용이 여기에 있다. 번역에만 AI를 쓰는 것이 아니라, **AI로 프롬프트 자체를 설계한다**. 한 번 만든 프롬프트를 그대로 쓰지 않고, 실패 사례를 모아 계속 다듬어 나간다.

메타 프롬프트란

먼저 용어를 정리한다. **메타 프롬프트**는 “다른 프롬프트를 만들거나 고치기 위한 프롬프트”다. 보통의 프롬프트는 AI에게 일을 시킨다(“이 문장을 번역해줘”). 메타 프롬프트는 AI에게 프롬프트 자체에 대해 일을 시킨다(“이 번역 프롬프트의 약점을 찾아줘”). 같은 모델이지만 입력이 달라지면 출력 성격도 완전히 달라진다.

쓰는 방식은 두 레벨로 나뉜다.

1. **레벨 1:** 좋은 번역을 만드는 프롬프트를 작성한다.
2. **레벨 2:** 더 나은 번역 프롬프트를 만드는 프롬프트(메타 프롬프트)를 작성한다.

레벨 2의 형태는 이렇다.

“당신은 프롬프트 엔지니어링 전문가입니다. 기술 문서용 이 번역 프롬프트를 개선해야 합니다. 현재 버전: [현재 프롬프트 텍스트] 내가 보고 있는 문제들: [구체적인 실패 예시] 당신의 작업: 프롬프트에 대한 3가지 구체적인 개선사항을 제안하세요. 총 200단어 이하로.”

36개 언어를 하나씩 직접 테스트하는 것보다, 이 메타 프롬프트 패턴으로 프롬프트 자체를 반복 개선하는 쪽이 훨씬 빠르다.

같은 AI를 검토자로 다시 쓰기

같은 모델에게 역할을 바꿔 물어볼 수도 있다.

“당신은 다음 프롬프트를 받는 번역 AI가 될 예정입니다. 읽고 **지시사항이 모호한 곳 세 곳**을 찾아주세요. 고치지는 마세요.”

Claude가 찾은 모호한 부분 세 가지는 모두 실제 번역 출력에 영향을 줬다. 예를 들어“숫자 형식 보존”이 날짜도 포함하는지, “인라인 코드”가 백틱 하나인지 둘인지가 분명하지 않았다. 이를 반영해 프롬프트를 다듬으니 품질이 올라갔다. **같은 AI 모델이지만 역할이 달라지면 다른 관점으로 바라본다.**

이중 에이전트—번역자와 검토자 분리

여기서 **에이전트**라는 단어를 쓰는데, 이 책에서는“특정 역할을 부여받아 한 가지 일만 하는 AI 인스턴스”라는 좁은 뜻으로 쓴다. 같은 모델(예: Claude)이라도 어떤 프롬프트로 호출하느냐에 따라 다른 에이전트가 된다.

핵심 패턴은 단순하다. **한 에이전트의 출력을 다른 에이전트가 검토하게 한다.**

- **Agent 1 (번역자):** 번역 프롬프트를 따라 출력을 생성한다.
- **Agent 2 (검토자):** 같은 출력을 받아 코드 보존, 용어 일관성, 형식 보존, 의미 보존을 검사한다. 문제가 있으면 목록으로 출력하고, 깨끗하면‘No issues’를 반환한다.

예를 들어 독일어 번역에서 파일명 `app.config.js`가 `anwendungs.konfig.js`로 번역되면, 검토자가“파일명이 번역됨—코드 보존 규칙 위반”이라고 짚어준다. 단독 번역에서는 그냥 넘어가는 오류를 잡아낼 수 있다. 32개의 읽지 못하는 언어에 대해 이 패턴이 특히 유용했다.

실패한 메타 프롬프트들

모든 메타 프롬프트 접근법이 성공한 건 아니다. 막연한 요청은 막연한 답을 부른다.

- “더 좋게 만들어”-> “더 구체적으로 하세요”같은 일반론만 돌아온다. **구체적인 실패 사례를 함께 줘야** 한다.
- “이 프롬프트 점수를 매겨”-> “8/10”같은 숫자가 나오지만 실제 성능과 무관하다. **점수 대신 테스트 사례에서 실패를 식별하라고** 요청해야 한다.

주의: 메타 프롬프트도 일반 프롬프트처럼 구체적인 기준이 필요하다.

프롬프트 버전 관리

프롬프트를 여러 차례 수정하다 보면 변경 이력 관리가 필요해진다. git처럼 관리했다.

```
prompts/
v1.0-base-naive.md           # "이 텍스트를 번역하라"
v1.1-add-code-preservation.md
v1.2-fix-quote-handling.md
v2.0-language-profiles.md
v2.1-ambiguity-fixes.md
current.md -> v2.1-ambiguity-fixes.md
```

각 버전에는 무엇이 바뀌었고 왜 바뀌었는지 기록을 남겼다. 덕분에 **프롬프트 버전을 비교**하고, 개선이 오히려 퇴보였을 때 바로 되돌릴 수 있었다. 실제로 Claude가 자연스러움을 높이려고 “번역에서 창의적이 되라”를 추가하자고 제안했는데, 적용하자 기술 용어가 들쭉날쭉해져 바로 되돌린 사례가 있다.

10.5 번역 품질 기준—AI에게 전달하는 방법

“번역 품질이 높아야 한다”는 식의 선언은 통하지 않는다. AI는 구체적인 지시를 받아야 움직인다. **검사 가능한 항목들로 잘게 나눠야** 한다.

최종 프롬프트에서 발췌한 검사 항목들이다.

품질 기준 (출력하지 말 것 —내부 검사용):

1. 의미 보존: 문장이 생략되거나 조작되지 않음.
2. 용어 일관성: 같은 원문 용어 -> 같은 대상 용어로 문서 전체에서.
3. 어조 일치: 기술 문서는 격식을 갖춘 채로, 대화체가 아님.
4. AI 메타 토크 없음: "번역은 다음과 같습니다" 등이 금지됨.
5. Markdown 구조 동일: 같은 제목 레벨, 목록, 코드 펜스.

다섯 항목을 “내부 검사”로 묶으면 출력이 달라진다. 모델이 실제 내부에서 어떤 절차를 밟는지는 알 수 없지만, 프롬프트에 체크리스트를 넣으면 결과가 더 일관되게 나오는 경향이 있다.

각 기준이 어떤 실패에서 나왔는가

기준 다섯 개는 모두 실제 실패 사례에서 만들어졌다. 대표적인 예시 두 가지만 보자.

- **의미 보존**: “The function returns null if no user is found.”가 “사용자가 없으면 함수가 null을 찾습니다.”로 번역됐다. “반환”이 “찾는다”로 바뀌었다. “정확하게 번역하라”만으로는 부족하고, **의미 보존을 명시적으로 요구**해야 한다.
- **어조 일치**: 영어의 “Execute the following command”가 프랑스어로 너무 캐주얼한 *tu* 형태 (“Va exécuter...”)로 바뀐 적이 있다. 격식체 *vous*를 명시하지 않으면 AI는 대화체를 기본값으로 쓴다.

용어 일관성, AI 메타 토크 차단, Markdown 구조 보존도 비슷한 사례가 쌓이며 만들어진 규칙이다.

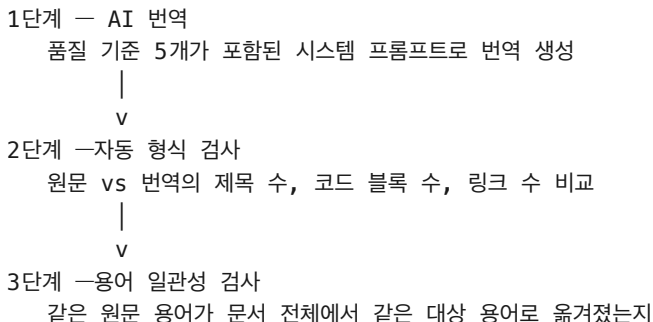
도메인별 추가 기준

기본 5개로 잡히지 않는 엣지 케이스가 있어, 보존 규칙 묶음을 추가했다. API 엔드포인트 경로, 오류 코드, 설정 키, 명령줄 플래그, 파일 확장자—이런 식별자들은 절대 번역하지 않도록 명시했다. 예를 들어 러시아어 번역이 `.config.json`을 `.конфиг.json`으로 현지화하려 한 사례가 있었다. 러시아어로는 논리적인 번역이지만, 코드 예제가 전부 깨진다.

주의: 품질 기준끼리 충돌할 수 있다. 대표적인 예가 “의미 보존” vs “자연스러운 표현”이다. 우선순위를 미리 정해둔다. **기술적 정확성이 첫 번째, 자연스러움이 두 번째다.**

검증 파이프라인—자동 검사 + 위험 기반 사람 검토

품질 기준만으로는 부족하다. 출력이 정말 그 기준을 지켰는지 확인하는 단계가 필요하다. 다음 네 단계를 순서대로 거치도록 했다.



|
v

4단계 —사람 검토 (모든 경고 항목 + 초기 표본)
문화적 뉘앙스, 비표준 기술 용어 등 자동으로 못 잡는 항목

4단계가 특히 중요했다. 자동 기준은 코드·숫자·URL 보존 같은 구조적 문제를 많이 잡았지만, 문화적 뉘앙스나 해당 언어에서 쓰지 않는 기술 용어는 원어민 또는 전문 검토가 필요했다. 출시 초기에는 무작위 표본도 함께 보고, 기준선이 쌓인 뒤에는 REVIEW/FAIL과 변경 영향 언어를 우선 검토한다. 이 4단계 흐름이 다음 11장의 검증 파이프라인으로 이어진다.

10.6 이 장의 루프 정리

지금까지의 반복 과정을 한 표로 정리하면 이렇다.

라운드	내용	결과
1	초기 제약 조건 (코드 블록, URL)	두 번째 패스 필요 (따옴표)
2	따옴표에 대한 “기호 vs 내용” 분리	완료
3	언어 프로필 디자인	내가 주도, AI가 지원
4	언어 전문가로서의 Claude	36개 언어 프로필 생성
5	Claude를 프롬프트 검토자로 재사용	3개의 모호성 발견
6	자기 재작성 실험	비용 대비 효과 낮음, 폐기
7	Vite의 ?raw로 .md를 문자열로 импорт	한 번에 해결

사이드노트—라운드 6 비용 분석: 프롬프트에 “5개 항목이 모두 만족될 때까지 다시 써라”를 추가하는 실험도 해봤다. 토큰 사용량이 거의 두 배로 늘었지만 품질 개선 폭은 매우 작았다. 내부 체크리스트 방식은 토큰을 절반만 쓰면서 품질 개선 효과의 대부분을 낸다.

사이드노트—Vite 문자열 импорт: `import prompt from "./prompt.md?raw";`는 파일 내용을 문자열로 가져온다. 비밀번호를 프롬프트 파일에 넣으면 번들에 포함될 수 있으므로 비밀번호는 두지 않는다. TypeScript가 이 문법을 모르면 프로젝트의 Vite 타입 선언이 포함돼 있는지 확인한다. Markdown을 문자열로 가져오는 것과 안전한 HTML로 렌더링하는 것은 별개의 작업이다.

반복된 세 가지 패턴

모든 루프에서 세 가지 패턴이 반복됐다.

1. **제약 특정성**: “잘 번역하라”처럼 막연한 지시는 품질이 들쭉날쭉했다. “코드를 정확히 보존하라”처럼 구체적인 제약을 주면 결과가 훨씬 안정적이었다.
2. **AI 역할 전환**: 같은 AI를 번역자, 검토자, 언어 전문가 등 다른 역할로 쓰면 각각 다른 시각의 결과가 나왔다.
3. **반복 개선**: 어떤 프롬프트도 처음부터 완벽하지 않았다. 체계적으로 테스트하고 고치는 과정을 거쳐야 품질이 올라간다.

이 세 가지는 번역 프롬프트에만 해당하지 않는다. 정밀한 결과가 필요한 AI 작업이라면 어디에나 적용된다.

이 장의 결론: AI는 코드를 짜주는 도구만이 아니다. 상황에 따라 **검토자**, **전문가**, **실험 파트너**로도 활용할 수 있다.

이 장에서 만든 산출물은 세 가지다. 첫째, 코드·숫자·URL을 보존하는 시스템 프롬프트. 둘째, 36개 언어별 프로파일. 셋째, 번역 결과를 검사할 품질 기준 5개. 다음 장의 검증 파이프라인은 바로 이 세 가지를 입력으로 사용한다.

프롬프트가 견고해도 한 가지 문제가 남는다. **출력이 정말 정확한지 실제로 검증할 수 있을까?** 36개 언어 중에서 나는 서너 개만 읽을 수 있다. 나머지 32개 언어는 눈으로 확인이 안 된다. 다음 장이 그 문제를 다룬다. **다른 AI로 역번역하고 자동으로 점수를 매기는 파이프라인**이다.

마무리 박스: 이 장에서 반드시 기억할 것

이 장은 **프롬프트 엔지니어링을 인간과 AI의 협업 과정**으로 다룬다. 프롬프트는 한 번 쓰고 버리는 문장이 아니라, **계속 관리하고 개선하는 품질 시스템**이어야 한다.

이 장에서 반드시 기억할 것:

- **제약 특정성**: AI는 말을 글자 그대로 받아들인다. “따옴표 보존”과 “따옴표 기호는 보존하되 내용은 번역”은 전혀 다른 결과를 만든다.
- **언어별 프로파일**: 하나의 프롬프트로 36개 언어를 모두 처리할 수 없다. 언어마다 별도 규칙이 필요하다.

- **메타 프롬프팅:** AI로 AI용 프롬프트를 더 잘 만든다. 같은 모델을 번역자, 검토자, 언어 전문가 역할로 나눠 활용할 수 있다.
- **코드로서의 품질 기준:** “높은 품질”처럼 막연한 목표 대신, 검증 가능한 조건 5개로 나눈다. AI는 주관적인 목표보다 명확한 체크리스트를 훨씬 잘 따른다.

이를 코드로 구현할 때는 두 가지가 필요하다. **Provider 추상화**(여러 번역 서비스를 하나의 인터페이스로 묶기), 그리고 **프롬프트 관리**(버전 관리, 부분별 조립, 자동 테스트).

실용적으로는 다음 다섯 가지를 기억하면 된다.

1. **간단하게 시작하고 제약 추가:** 기본 번역부터 시작해, 문제가 생기면 구체적인 규칙을 하나씩 추가한다.
2. **AI를 여러 전문가로 활용:** 같은 모델도 역할에 따라(번역자/검토자/전문가) 다른 시각을 준다.
3. **대상 언어별 테스트:** 출력 언어를 직접 읽을 수 없다면, 자동 검증 시스템을 만들어 둔다.
4. **프롬프트 버전 관리:** 코드처럼 변경 내역을 추적하고, 이전 버전으로 되돌릴 수 있게 관리한다.
5. **측정 및 모니터링:** 실제 운영 중에 프롬프트 성능을 추적해, 이후 개선에 활용한다.

다음 장 미리보기: 출력 언어를 읽을 수 없을 때 번역 품질을 어떻게 검증할까. 역번역 파이프라인으로 자동 번역 품질 점수를 매기는 방법을 다룬다.

11장. 번역 검증—AI 출력을 어떻게 신뢰할 것인가

readbooks.ai는 36개 언어를 지원한다. 그중 32개는 대부분의 개발자가 직접 읽을 수 없는 언어다. 번역이 자연스러운지, 용어가 맞는지, 문단이 빠지지 않았는지 눈으로 확인할 수 없다면 품질 관리는 다른 방식으로 설계해야 한다.

프롬프트는 잘 짜여 있고 출력도 그럴듯해 보인다. 그런데 어디가 잘못됐는지 알 방법이 없다. 이 문제는 번역에만 있지 않다. AI가 사람이 따라잡기 어려운 속도로 전문 콘텐츠를 쏟아내는 모든 분야에서 같은 상황이 생긴다.

해결책은 **두 번째 AI를 쓰는 검증 파이프라인**이다. 먼저 실제로 자주 발생하는 실패 유형을 목록으로 만든다. 그다음, 각 실패가 실제로 일어났는지 AI가 자동으로 확인하게 한다.

완벽한 방법은 아니다. 미리 예상한 실패 유형만 잡을 수 있고, 예상 밖의 오류는 여전히 사람이 직접 봐야 한다. 그래서 검증은 생성 단계에 묻히지 않고 별도 단계로 분리된다. 코드 리뷰어를 작성자와 분리하는 이유와 같다. 같은 사람이 작성하고 리뷰하면 자기가 빠뜨린 부분을 똑같이 빠뜨리기 쉽다. 생성 AI가 만든 결과를 바로 신뢰하는 게 아니라, 독립된 검증 절차를 통과해야 제품으로 낸다.

이 장의 흐름

큰 단계만 보면 네 걸음이다.

1. **문제를 정의한다** —“그렇듯하지만 틀린”번역의 네 가지 모양.
2. **역번역을 한 번 해본다** —검증의 가장 단순한 형태.
3. **세 축으로 확장한다** —의미·길이·용어로 나눠 본다.
4. **사람의 피드백을 다시 모은다** —자동 검증이 놓친 것을 다시 프롬프트로 돌려보낸다.

운영 단계의 디테일(합성 테스트 케이스 보정, 어족별 회귀 테스트, 카나리 배포)은 장 뒤쪽에 짧게 정리만 해두고, 첫 회독에서는 위 네 걸음을 따라가는 데 집중한다.

학습 목표

- AI 번역 결과를 직접 읽을 수 없을 때 검증이 왜 어려운지, 어떻게 접근해야 하는지 이해한다.
- 역번역(번역문을 다시 원래 언어로 돌려 비교하는 방법)으로 검증하는 흐름과 점수 기준 잡는 방법을 배운다.
- 36개 언어를 자동으로 테스트하는 큰 그림을 익힌다.
- 사용자 피드백을 프롬프트 개선에 다시 반영하는 패턴을 파악한다.

이 장에서 만들 최소 산출물: `verify_translation` Tauri 명령 한 개와, 세 축 점수(의미·길이·용어)를 보여주는 화면 하나.

11.1 문제—“그렇듯하지만 틀린”번역

LLM이 번역에서 저지르는 가장 까다로운 실수는, 문장은 자연스럽지만 의미가 틀린 경우다. 겉으로는 멀쩡해 보이기 때문에 사용자가 잘 눈치채지 못한다.

기술 문서에서는 특히 위험하다. 문장이 자연스럽게 읽혀도 코드 설명이나 API 이름이 잘못됐다면, 독자는 틀린 절차를 그대로 따라 하게 된다.

자주 나타나는 실패 유형은 네 가지다.

- **누락**: 문단에서 문장 전체가 빠진다. 예를 들어 데이터베이스 연결 풀을 설명하는 두 문장 중 두 번째 문장이 통째로 없어진다.
- **날조 (hallucination)**: 원문에 없던 예시가 추가된다. 영어 원문이 fetch API 만 언급했는데 스페인어 번역에 XMLHttpRequest가 끼어든다.
- **용어 표류**: 같은 단어를 문서 앞에서는“컴포넌트”로, 뒤에서는“구성요소”로 다르게 번역한다. 독자는 두 단어가 같은 개념인지 확신할 수 없다.
- **문체 붕괴**: 기술 문서 중간에 갑자기 캐주얼한 대화체가 섞여 들어온다. 격식체로 쓰던 가이드 중간에 친근한 반말이 튀어나오는 식이다.

이 중 **누락**과 **날조**는 원문을 같이 보지 않으면 사용자가 발견하기 어렵다.

우려: 이런 오류는 컴파일 오류처럼 바로 드러나지 않는다. 누군가 눈치채기 전까지 수천 명에게 그대로 전달된다.

11.2 역번역—손으로 한 번 해보기

첫 번째 전략은 고전적이다. **번역 결과를 다시 원래 언어로 번역해서 원문과 비교하는 것이다**. 대상 언어를 읽지 못해도 역번역 결과는 영어로 돌아오므로 직접 비교할 수 있다.

먼저 코드 없이 손으로 한 번 따라가 보자.

1. 영어 원문 한 문단을 적는다. 예: “Use the fetch API to make HTTP requests.”
2. 이걸 스페인어로 번역한다(생성 AI에게 맡긴다). 결과: “Use the fetch API y el método XMLHttpRequest para hacer solicitudes HTTP.”
3. 그 스페인어를 다시 영어로 번역한다(검증 AI에게 맡긴다). 결과: “Use the fetch API and XMLHttpRequest method to make HTTP requests.”
4. 1번과 3번을 사람이 눈으로 비교한다. XMLHttpRequest가 어디서 나왔는지 바로 보인다.

핵심은 **2단계에서 어떤 언어로 갔는지 몰라도, 4단계에서는 영어끼리 비교한다**는 점이다. 36개 언어를 모두 읽을 줄 몰라도 검증이 가능한 이유다.

단, 완전한 방법은 아니다. 역번역 과정에서 오류가 가려질 수도 있다. 그래서 이 방법을 자동화하면서 점수 기준을 함께 잡아야 한다.

11.3 검증 명령 만들기—가장 단순한 골격

이제 위 흐름을 Tauri 명령으로 옮긴다. 처음에는 완성된 품질 평가기가 아니라 가장 작은 골격을 먼저 얻고 싶었다. 원문과 번역문을 넣으면 다시 영어로 되돌리고 점수 하나를 반환하는 구조다.

프롬프트:

“Translation 검증 command. 입력: source (영어), translation, target language. 내부적으로 영어로 back-translate. source와 유사도 점수. translate_with_provider command 재사용. 골격만, 10 줄 미만.”

Claude가 돌려준 첫 골격은 `verify_translation`이라는 Tauri 명령을 만들고, 안에서 10장에서 만든 `translate_with_provider`를 다시 불러 이미 받은 번역문을 영어로 되돌린다. 그리고 원문과 비교한 점수 하나를 돌려준다.

10장의 `translate_with_provider`는 “어떤 provider(OpenAI/Anthropic/온디바이스 등)와 모델로, 어떤 언어에서 어떤 언어로 옮길지”를 인자로 받는 범용 함수였다. 전체 번역·검증 파이프라인에서는 이 함수를 **두 번 호출한다**: 생성 단계에서 원문 -> 대상 언어로 한 번, 검증 단계에서 대상 언어 -> 영어로 한 번이다. 다만 `verify_translation` 명령의 입력에는 번역문이 이미 들어오므로, **이 명령 내부의 호출은 역번역 한 번뿐**이다.

오프라인 검증에서 독립성을 지키는 방법

온디바이스 Provider도 같은 인터페이스를 사용하므로 네트워크 없이 역번역할 수 있다. 그러나 정방향 번역과 역번역을 같은 소형 모델에 맡기면 두 단계가 같은 오해를 반복할 수 있다. 결과가 원문과 비슷하게 돌아왔다고 해서 첫 번역이 정확하다고 단정할 수 없는 이유다.

오프라인만 허용되는 환경에서는 검증을 세 층으로 나눈다.

1. 코드 블록, URL, 숫자, 문단 수는 모델을 쓰지 않는 규칙 검사로 비교한다.
2. 역번역은 가능하면 생성 모델과 다른 계열의 로컬 모델로 실행한다.
3. 기기 메모리 때문에 모델 하나만 쓸 수 있다면 생성·검증 프롬프트와 평가 기준을 분리하고, 사람이 검토할 회귀 표본을 유지한다.

두 로컬 모델을 동시에 메모리에 올릴 필요는 없다. 번역 모델을 내리고 검증 모델을 필요할 때 로드하는 온디맨드 방식을 쓰면 메모리를 아낄 수 있다. 대신 모델 교체 시간이 생기므로 문장마다 번갈아 로드하지 말고, 한 문서의 번역을 마친 뒤 검증 작업을 묶어서 처리한다.

11.4 단일 점수의 한계—BLEU와 cosine 유사도

골격이 돌아가니 점수 부분이 다음 문제로 떠올랐다. 어떤 숫자로 “비슷하다”를 표현할 것인가?

처음에는 BLEU(번역 품질을 단어 조합 겹침으로 재는 고전 지표)를 시도했지만, **역번역 검증에는 맞지 않았다**. BLEU는 같은 단어 조합이 얼마나 겹치는지 세기 때문에, 의미가 똑같아도 다른 단어로 표현되면 점수가 낮게 나온다.¹

그래서 방향을 바꿔 **의미 유사도 점수를 받는 API**를 쓰기로 했다. 원문과 역번역을 각각 임베딩 API에 넣으면 두 문장이 의미적으로 얼마나 가까운지 0~1 사이 숫자로 돌려준다. 의미가 비슷하면 1에 가까운 값이 나온다.

이걸로 끝이면 좋겠지만, 이 방법에도 한계가 있다. 의미 유사도는 **길이 차이에 둔감하다**. 번역에서 문단 하나가 통째로 빠져도 남은 내용이 원문과 가깝다면 0.95 이상이 나올 수 있다.

실제로 React 컴포넌트 생명주기를 설명하는 영어 문단에서 “mounting” 설명이 통째로 빠진 독일어 번역을 만들어 테스트했다. 빠진 채로 역번역해도 유사도가 0.95로 나왔다. **단일 점수로는 부족하다**는 게 여기서 드러난다.

11.5 세 개의 축으로 나누기

번역 품질은 숫자 하나로 표현하기 어렵다. 의미는 비슷하지만 문단이 빠질 수 있고, 길이는 비슷하지만 핵심 용어가 틀릴 수 있다. 그래서 검증 기준을 **세 개 축**으로 나눴다.

- **의미적 유사성**: 임베딩 API가 돌려주는 0~1 점수.
- **길이 비율**: 번역된 단어 수 ÷ 원문 단어 수. 0.85~1.2는 이 프로젝트가 처음 실험한 임시 경고 범위일 뿐이며, 정상 범위는 언어쌍·문서 종류·토큰 계산 방식에 맞춰 회귀 표본으로 보정한다.
- **용어 일관성**: 핵심 용어(고유명사, API 이름)의 빈도가 얼마나 일치하는가.

각 지표를 별도 함수로 구현하고, 어떤 지표든 기준값을 벗어나면 해당 번역에 플래그(오류 표시)를 한다.

결과 타입을 프론트엔드와 맞추기

프론트엔드가 검증 결과를 받아 화면에 표시하려면 결과 모양을 양쪽에서 똑같이 정의해야 한다.

¹BLEU는 본래 번역 모델끼리 같은 정답 문장에 얼마나 가깝게 재현하는지 비교하기 위한 지표라, “의미는 같고 단어는 다른” 역번역에는 부적합하다.

- TypeScript 쪽에는 `VerifyReport` 타입을 만든다. 의미 점수, 길이 비율, 용어 일관성, 플래그 배열을 담는다.
- Rust 쪽에는 같은 모양의 구조체를 만든다. 그리고 `serde::Serialize`를 붙인다.

`serde::Serialize`를 붙이는 이유는 단순하다. Rust 구조체는 그대로 자바스크립트에 넘길 수 없다. Tauri는 구조체를 일단 JSON으로 바꾼 다음 프론트엔드로 보낸다. 흐름은 다음 세 줄로 정리된다.

1. Rust가 `VerifyReport` 구조체를 만든다.
2. `serde`가 그것을 JSON 문자열로 직렬화한다.
3. `invoke('verify_translation', ...)`의 반환값으로 자바스크립트에서 `VerifyReport` 객체가 도착한다.

확인할 출력: 프론트엔드에서 받은 객체의 `flags` 배열에 어떤 키워드가 들어왔는지가 핵심 신호다. 비어 있으면 GOOD, 한두 개 있으면 REVIEW, 여러 개 있으면 FAIL로 처리한다.

당시 회귀 테스트에서 세 지표는 다수의 누락 사례를 걸러냈지만, 원문에 없는 내용을 자연스럽게 덧붙인 날조 오류는 빠져나갔다. `flags` 배열에 자리는 비워뒀지만 탐지 로직은 아직 없었다. **검증 파이프라인은 처음부터 완성되지 않는다.** 실패 사례를 만날 때마다 테스트와 지표를 하나씩 추가하며 강해진다.

우려: 길이 비율은 언어에 따라 오탐지를 낼 수 있다. 독일어 복합어는 단어 수가 줄어들고, 인도네시아어는 조사가 추가된다. 언어별로 허용 범위를 따로 조정해야 한다.

날조 탐지 추가

날조란 원문에 없던 내용이 번역에 끼어드는 오류다. 의미 유사성과 길이 비율만으로는 잡기 어렵다.

탐지 방법은 단순하다. LLM에게 원문과 번역문 각각에서 고유명사·제품명·API 식별자를 한 줄에 하나씩 뽑아 달라고 요청한다. 그 다음 두 결과를 비교해서, **번역문에만 있고 원문에는 없는 단어**를 날조 후보로 본다. 완벽하지 않지만 충분히 실용적이다.

11.1에서 들었던 `XMLHttpRequest` 사례가 그대로 이 방법으로 잡힌다.

3단계 배지로 정리

임계값을 잡는 데도 시행착오가 있었다. 처음에는 너무 엄격해서 모든 페이지에 “이 번역이 이상해 보입니다”배지가 떴다. 너무 느슨하게 바꾸니 이번에는 거의 발동하지 않았다. 결국 **3단계 배지**로 정리했다.

- **GOOD**: 모든 지표가 범위 내.
- **REVIEW**: 한 지표가 경고 범위.
- **FAIL**: 두 개 이상 지표가 경고.

3단계로 나눈 뒤 오탐지 불만이 줄었다. **자동 검증으로 모든 오류를 잡을 수는 없다.** 현실적인 목표는 사람이 봐야 할 번역과 그렇지 않은 번역을 분리하는 데 있다.

11.6 운영하며 다듬는 이야기

여기까지가 첫 회독에서 꼭 알아야 할 내용이다. 이 절은 실제 서비스로 가져갈 때 추가로 거치는 단계를 짧게 짚고 넘어간다. 깊이 들어가지 않고, “이런 단계가 더 있다” 정도로만 보면 된다.

- **합성 테스트로 임계값 보정**: AI에게 “한 축에서만 문제 있는 케이스”같은 시나리오를 만들게 하고, 그 결과로 GOOD/REVIEW/FAIL의 경계선을 미세 조정한다.
- **언어별 테스트 매트릭스**: 36개 언어를 어족(로망스어·게르만어·교착어·표의문자)으로 묶고, 어족마다 자주 틀리는 오류 유형을 골라 자동 생성한다. 게르만어는 복합어가, 교착어는 길이 비율이 까다롭다는 식의 특성이 드러난다.
- **언어별 회귀 테스트**: 공통 프롬프트를 고쳤더니 일부 언어의 품질이 떨어지는 교차 영향이 생길 수 있다. 언어마다 자체 기준선을 JSON으로 분리하고, 공통 규칙을 바꾸면 대표 언어 전체를, 언어별 규칙을 바꾸면 해당 언어를 다시 검증한다.

검증표는 언어·입력 유형·기대 조건·실제 결과·판정·검토자/도구 버전을 열로 둔다. 예를 들어 “숫자 12.5%가 보존됨”, “문단 3개가 유지됨”, “번역문만 출력됨”, “고유명사 표기가 일관됨”을 각각 판정한다. 합성 문장 통과율과 실제 문서의 사람 평가를 분리하고, 시험하지 않은 언어의 정확도를 추정값 없이 적지 않는다. 이 표를 채우면 운영 검증을 시작할 수 있다.

11.7 사용자 피드백 루프

자동 검증이 아무리 정교해도 실제 사용자가 지적하는 문제를 완전히 대신할 수는 없다. Tauri 2로 재 구현한 버전에 피드백 버튼을 추가했다.

기능

- 각 문단 끝에 작은 “이 번역이 이상해 보입니다”버튼.
- 클릭하면 문단 내용, 원문, 대상 언어, provider 정보를 로그 DB로 전송.
- 2주마다 로그 검토.

실제 데이터로 프롬프트 업데이트

3주차 로그를 분석하니 다음 문제가 나왔다.

- 독일어 “복합명사 분할” 12건
- 프랑스어 “성별 일치 오류” 8건
- 스페인어 “vos/tu 혼용” 5건

이 데이터를 Claude에게 넘겨 언어별 프로파일을 업데이트했다. 예를 들어 독일어 프로파일에는 “독일어 복합명사는 단일 단어로 유지(Datenbankverbindung, Datenbank-Verbindung이 아니라). 원문에서 명확히 하이픈을 사용하지 않는 한 분할하지 마세요.”같은 규칙을 추가했다.

확인할 출력: 같은 문제가 다음 2주간 다시 신고되는지가 검증 신호다. 이 한 줄로 같은 문제 12건이 다시 나타나지 않았다. **피드백 -> 프롬프트 업데이트**는 비용 대비 효과가 명확한 개선 방법이다.

피드백을 단순한 세 단계로 정렬

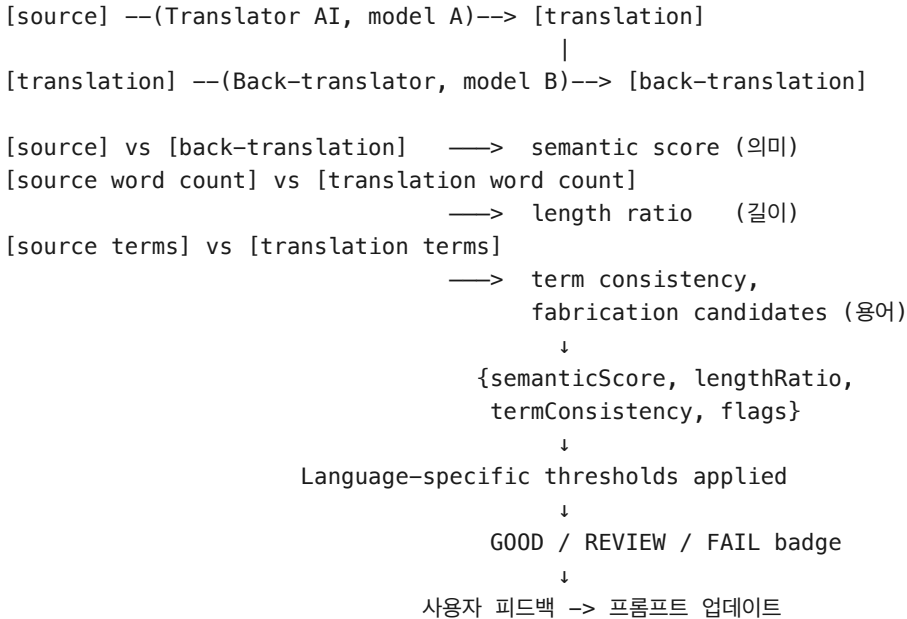
사용자 피드백이 모두 같은 우선순위를 갖지는 않는다. 영향 범위로만 묶어 세 단계로 다룬다.

- **즉시 수정:** 의미 반전(enable를 disable로 옮기는 식), 보안 관련 오역.
- **주간 배치 수정:** 용어 불일치, 문화적으로 부적절한 톤.
- **월간 검토:** 기술적으로는 맞지만 어색한 표현, 단순한 선호 차이.

분류 자체도 Claude에게 맡긴다. 분류하지 않으면 사소한 스타일 선호도가 쌓여 정작 심각한 오류를 늦게 발견하게 된다.

11.8 “AI 출력을 다른 AI로 검증하기”시각화

이 장의 핵심 패턴을 한 그림으로 모았다. 위에서 아래로 흐름이 흐른다.



생성과 검증의 실패가 서로 강하게 겹치지 않도록 독립성을 높이는 것이 핵심이다. 다른 provider나 모델을 쓰면 같은 계열의 편향을 함께 반복할 가능성을 줄일 수 있지만, 정확성을 자동으로 보장하지는 않는다. Claude로 번역하고 GPT로 검증하거나, 로컬 모델 A로 번역한 뒤 로컬 모델 B로 검증할 수 있다. 같은 모델만 쓸 수 있다면 검증 프롬프트·표본·온도와 평가 기준을 생성 단계와 분리한다.

11.9 독자 앱에 적용하는 팁

이 패턴은 번역에만 쓰는 게 아니다. AI 출력이라면 어디든 적용할 수 있다.

팁 1. 단일 점수 금지. 축을 쪼개라.

품질을 점수 하나로 표현하면 무엇이 문제인지 알 수 없다. 의미·길이·형식·일관성처럼 평가 기준을 항목별로 나눠야 어느 부분이 나쁜지 바로 파악된다.

팁 2. 생성 단계와 검증 단계를 독립적으로.

가능하면 Claude가 생성한 결과를 GPT가, OpenAI가 생성한 결과를 Gemini가 검증하는 식으로 모델 계열을 나눈다. 서로 다른 모델도 같은 오류를 낼 수 있으므로, 사람 검토용 표본과 규칙 기반 누락 검사까지 함께 둔다.

팁 3. 임계치는 3단계로.

합격과 불합격, 두 가지로만 나누면 기준이 불필요하게 엄격해진다. 좋음·검토 필요·실패, 세 단계로 나누면 사람이 직접 확인해야 하는 양을 현실적으로 조절할 수 있다.

팁 4. 사용자 피드백을 프롬프트에 반영.

자동 검증이 잡지 못한 문제는 사용자가 발견한다. 2주마다 피드백 로그를 모아 언어별 프롬프트 프로파일을 갱신한다. 적은 비용으로 출력 품질을 꾸준히 높일 수 있는 방법이다.

11.10 Part II와 Part III를 돌아보며

Part II(Mandai)와 Part III(readbooks.ai), 두 앱이 공통으로 보여준 것이 있다. **AI가 구현 속도를 높여 쥐도 방향을 내가 잡지 않으면, 빠른 속도로 잘못된 방향으로 가게 된다.** 요청·구현·검증 루프는 속도와 방향을 함께 잡는 장치다.

- Part II에서 루프는 주로 **코드의 옳고 그름**을 다뤘다. 컴파일되는지, 동작하는지, 엣지 케이스는 없는지.
- Part III에서 루프는 **출력의 품질**을 다뤘다. 번역에는 정답이 없다. 평가 기준을 직접 설계해야 했다.

다음 장에서는 번역 품질 검증을 넘어 대시보드 규모의 프로젝트로 이동한다. 검증의 초점도 언어적 정확성에서 데이터 정확성과 실시간 렌더링으로 바뀐다.

마무리 박스: 이 장에서 반드시 기억할 것

쉬운 말로 다시 정리하면 다음 네 가지다.

1. **품질은 점수 하나로 못 잡는다.** 의미·길이·용어, 최소 세 가지 축으로 나눠서 봐야 한다.
2. **생성과 검증은 독립성을 높인다.** 가능하면 모델을 나누고, 어렵다면 프롬프트와 평가 기준이라도 분리한다.
3. **합격/불합격 두 단계는 너무 거칠다.** GOOD / REVIEW / FAIL 세 단계가 사람이 실제로 쓰기 좋다.
4. **자동 검증은 끝이 아니다.** 사용자 피드백을 모아 프롬프트를 정기적으로 고쳐야 품질이 유지된다.

이 장의 운영 원칙은 다음과 같다.

패턴: AI 출력의 편향을 잡으려면 다른 AI 모델로 검증해라

아키텍처: 복잡한 출력에는 단일 품질 지표보다 다축 점수가 낫다

테스트: AI로 테스트 케이스를 생성해 전 범위를 자동 검증하고, 임계치를 바꾸기 전에는 반드시 회귀 테스트를 먼저 실행해라

운영: 언어별 기준선으로 교차 언어 간섭을 막고, 실제 사용자 피드백으로 지속 보정해라